Politécnico
Castelo Branco
Escola Superior
de Tecnologia

Análise de Cesto de Compras
Projeto II

Bruno Daniel Tomás Alves

João José Valente Coelho Tomé

Orientadores

Professora Adjunta Doutora Ana Paula Neves Ferreira da Silva

Professor Adjunto Doutor Arlindo Ferreira da Silva

Trabalho de Projeto apresentado à Escola Superior de Tecnologia da Universidade Politécnica de Castelo Branco para cumprimento dos requisitos necessários à obtenção do grau de Licenciado em Engenharia Informática, realizado sob a orientação científica da Professora Adjunta Doutora Ana Paula Neves Ferreira da Silva e coorientação do Professor Adjunto Doutor Arlindo Ferreira da Silva, da Universidade Politécnica de Castelo Branco.

Outubro 2026

Composição do júri

Presidente do júri

Doutor Osvaldo Santos

Professor Adjunto, Escola Superior de Tecnologia da Universidade Politécnica de Castelo Branco

Vogais

Doutor Fernando Ribeiro

Professor Adjunto, Escola Superior de Tecnologia da Universidade Politécnica de Castelo Branco

Doutora Ana Paula Neves Ferreira da Silva

Professora Adjunta, Escola Superior de Tecnologia da Universidade Politécnica de Castelo Branco

Doutor Arlindo Ferreira da Silva

Professor Adjunto, Escola Superior de Tecnologia da Universidade Politécnica de Castelo Branco

Resumo

O presente projeto insere-se na área da Análise de Cesto de Compras e tem como objetivo estudar e aplicar técnicas de extração de regras de associação a dados transacionais, de modo a sugerir produtos adicionais ao utilizador durante a construção de uma lista de compras.

No âmbito do Projeto II, foi desenvolvido um sistema composto por três componentes: uma API REST, em FastAPI, que integra o motor de recomendação baseado no algoritmo Apriori, a autenticação dos utilizadores e a persistência dos dados em SQLite; uma aplicação móvel em Flutter, na qual o utilizador cria conta, constrói e guarda listas de compras e recebe até cinco recomendações, calculadas a partir das regras de associação e do seu próprio histórico; e uma aplicação de backoffice em Streamlit, na qual o administrador consulta estatísticas sobre as regras e configura os parâmetros do motor. O sistema foi modelado segundo o processo ICONIX e a API foi verificada com 14 testes automáticos, que cobrem a autenticação, as listas de compras, as recomendações e os serviços do backoffice. Numa avaliação offline, em que se esconde um item de cada cesto de teste, o motor recuperou esse item nas cinco recomendações em 26,5% dos casos, um valor igual ao de uma recomendação baseada apenas na popularidade dos itens.

Na componente experimental, retomada do Projeto I, os algoritmos Apriori e FP-Growth foram aplicados a dois conjuntos de dados públicos, Groceries e Online Retail II. Ambos produziram os mesmos conjuntos frequentes e as mesmas regras, como é esperado em métodos exatos; o desempenho dependeu dos dados e da implementação, tendo o Apriori sido o mais rápido no conjunto de 169 itens que o motor de recomendação utiliza. A análise mostrou ainda a importância do pré-processamento, em particular da definição da unidade de transação e da limpeza dos registos.

O protótipo demonstra a viabilidade técnica da abordagem, mas tem limitações: as regras não superaram a popularidade na avaliação offline, a utilidade das recomendações não foi avaliada com utilizadores reais, os testes automáticos cobrem apenas a API e a versão atual não inclui autenticação no backoffice, expiração das sessões nem comunicação cifrada.

Palavras-chave

Análise de Cesto de Compras; Regras de Associação; Aprendizagem Computacional; Apriori; FP-Growth.

Abstract

This project focuses on Market Basket Analysis and aims to study and apply association rule mining techniques to transactional data, in order to suggest additional products to the user while building a shopping list.

In Project II, a system composed of three components was developed: a REST API, built with FastAPI, which integrates the recommendation engine based on the Apriori algorithm, user authentication and data persistence in SQLite; a mobile application in Flutter, in which users create an account, build and save shopping lists and receive up to five recommendations, computed from the association rules and their own history; and a back-office application in Streamlit, in which the administrator consults statistics on the rules and configures the engine parameters. The system was modelled following the ICONIX process, and the API was verified with 14 automated tests covering authentication, shopping lists, recommendations and the back-office services. In an offline evaluation, in which one item is hidden from each test basket, the engine recovered that item among its five recommendations in 26.5% of cases, the same value as a recommendation based only on item popularity.

In the experimental component, carried over from Project I, the Apriori and FP-Growth algorithms were applied to two public datasets, Groceries and Online Retail II. Both produced the same frequent itemsets and the same rules, as expected from exact methods; performance depended on the data and on the implementation, with Apriori being the fastest on the 169-item dataset used by the recommendation engine. The analysis also showed the importance of preprocessing, in particular the definition of the transaction unit and the cleaning of records.

The prototype demonstrates the technical feasibility of the approach, but it has limitations: the rules did not outperform popularity in the offline evaluation, the usefulness of the recommendations was not evaluated with real users, the automated tests cover only the API, and the current version does not include authentication in the back office, session expiry or encrypted communication.

Keywords

Market Basket Analysis; Association Rules; Machine Learning; Apriori; FP-Growth.

Índice

1. Introdução 1
1.1 Enquadramento 1
1.2 Objetivos do projeto 2
1.3 Planeamento do projeto 4
1.4 Estrutura do documento 5
2. Fundamentação teórica 7
2.1 Aprendizagem computacional 7
2.2 Mineração de dados 9
2.3 Análise de Cesto de Compras 11
2.4 Apriori 13
2.5 FP-Growth 15
2.6 Síntese crítica do estado da arte e enquadramento do projeto 17
3. Trabalho relacionado 19
3.1 Aplicações da Análise de Cesto de Compras em diferentes setores 19
3.2 Estudos no retalho e em supermercados 20
3.3 Abordagens avançadas e integração com aprendizagem computacional 21
3.4 Síntese dos trabalhos relacionados 23
4. Software e ferramentas utilizados 24
4.1 Python 24
4.2 Git e GitHub 24
4.3 Mendeley 24
4.4 pandas 25
4.5 NumPy 25
4.6 Bibliotecas dos algoritmos 25
4.7 Matplotlib 26
4.8 openpyxl 26
4.9 Streamlit 27
4.10 Seaborn 27
4.11 FastAPI 27
4.12 Uvicorn 28
4.13 Pydantic 28
4.14 SQLAlchemy 28
4.15 SQLite 28
4.16 bcrypt 29

4.17 Requests 29
4.18 Flutter 29
4.19 Dart 29
4.20 Bibliotecas da aplicação móvel 29
4.21 pytest 30
5. Metodologia experimental e resultados 31
5.1 Metodologia experimental 31
5.2 Dataset 1: Groceries 32
5.3 Dataset 2: Online Retail II 37
5.4 Discussão comparativa 43
5.5 Implicações para o motor de recomendação 44
6. Análise, desenho e implementação do sistema 46
6.1 Arquitetura geral 46
6.1.1 Camadas e componentes 47
6.1.2 Padrões aplicados 47
6.1.3 Fluxo de dados 48
6.1.4 Interface REST 50
6.1.5 Interface da aplicação móvel 51
6.2 Atores e casos de uso 52
6.2.1 Requisitos 52
6.2.2 Atores 54
6.2.3 Diagrama de casos de uso 54
6.2.4 Fichas de casos de uso 55
6.3 Diagramas de robustez 71
6.3.1 UC01 Criar conta 71
6.3.2 UC02 Autenticar-se 72
6.3.3 UC03 Terminar sessão 74
6.3.4 UC04 Consultar histórico de listas 75
6.3.5 UC05 Construir lista de compras 77
6.3.6 UC06 Consultar itens recomendados 78
6.3.7 UC07 Guardar lista de compras 79
6.3.8 UC08 Consultar estatísticas 80
6.3.9 UC09 Configurar parâmetros do motor 82
6.4 Diagramas de sequência 83
6.4.1 UC01 Criar conta 83
6.4.2 UC02 Autenticar-se 84

6.4.3 UC03 Terminar sessão 85
6.4.4 UC04 Consultar histórico de listas 86
6.4.5 UC05 Construir lista de compras 87
6.4.6 UC06 Consultar itens recomendados 88
6.4.7 UC07 Guardar lista de compras 89
6.4.8 UC08 Consultar estatísticas 90
6.4.9 UC09 Configurar parâmetros do motor 91
6.5 Testes e avaliação 92
6.5.1 Testes automáticos da API 92
6.5.2 Avaliação offline das recomendações 93
6.6 Instalação e execução 95
6.7 Interfaces implementadas 97
6.7.1 Aplicação móvel 97
6.7.2 Backoffice 98
6.7.3 Documentação da API 100
6.8 Síntese do capítulo 101
7. Conclusão 102
8. Bibliografia 105

Índice de figuras

Figura 1 – Cronograma de Gantt do planeamento do Projeto I e do Projeto II 5
Figura 2 – Regressão linear com correlação positiva (adaptado de Montgomery, Peck & Vining, 2021) 8
Figura 3 – Processo KDD (Knowledge Discovery in Databases) (adaptado de Fayyad, Piatetsky-Shapiro & Smyth, 1996) 9
Figura 4 – Algoritmo Apriori (elaboração própria) 14
Figura 5 – Construção de uma FP-tree a partir de quatro transações (suporte mínimo de 2 transações; elaboração própria) 15
Figura 6 – Suporte versus confiança das 161 regras (Dataset 1; resultados idênticos com Apriori e FP-Growth) 36
Figura 7 – Suporte versus elevação das 161 regras (Dataset 1; resultados idênticos com Apriori e FP-Growth) 36
Figura 8 – Suporte versus confiança das 31 regras (Dataset 2) 42
Figura 9 – Suporte versus elevação das 31 regras (Dataset 2) 42
Figura 10 – Arquitetura de componentes do sistema 49
Figura 11 – Classes de domínio principais 50
Figura 12 – Mockups da aplicação móvel: (a) autenticação, (b) criação de conta, (c) ecrã inicial, (d) construção da lista 52
Figura 13 – Diagrama geral de casos de uso 55
Figura 14 – Diagrama de robustez do UC01 Criar conta 72
Figura 15 – Diagrama de robustez do UC02 Autenticar-se 73
Figura 16 – Diagrama de robustez do UC03 Terminar sessão 74
Figura 17 – Diagrama de robustez do UC04 Consultar histórico de listas 76
Figura 18 – Diagrama de robustez do UC05 Construir lista de compras 77
Figura 19 – Diagrama de robustez do UC06 Consultar itens recomendados 78
Figura 20 – Diagrama de robustez do UC07 Guardar lista de compras 79
Figura 21 – Diagrama de robustez do UC08 Consultar estatísticas 81
Figura 22 – Diagrama de robustez do UC09 Configurar parâmetros do motor 82
Figura 23 – Diagrama de sequência do UC01 Criar conta 83
Figura 24 – Diagrama de sequência do UC02 Autenticar-se 84
Figura 25 – Diagrama de sequência do UC03 Terminar sessão 85
Figura 26 – Diagrama de sequência do UC04 Consultar histórico de listas 86
Figura 27 – Diagrama de sequência do UC05 Construir lista de compras 87
Figura 28 – Diagrama de sequência do UC06 Consultar itens recomendados 88
Figura 29 – Diagrama de sequência do UC07 Guardar lista de compras 89
Figura 30 – Diagrama de sequência do UC08 Consultar estatísticas 90
Figura 31 – Diagrama de sequência do UC09 Configurar parâmetros do motor 91
Figura 32 – Aplicação móvel: (a) autenticação, (b) criação de conta, (c) histórico de listas 97

Figura 33 – Aplicação móvel: construção de uma nova lista, (a) sem itens e (b) com itens e recomendações..... 98
Figura 34 – Backoffice, página "Estatísticas": (a) regras de associação e totais de itens, (b) itens mais frequentes e itens que mais aparecem nas regras..... 99
Figura 35 – Backoffice: (a) itens sem regras associadas, na página "Estatísticas", (b) página "Opções"..... 100
Figura 36 – Documentação interativa da API gerada pelo FastAPI (Swagger UI) 101

Índice de equações

Equação 1 – Teorema de Bayes 10
Equação 2 – Cálculo do valor de suporte 12
Equação 3 – Cálculo do valor de confiança 12
Equação 4 – Cálculo do valor de elevação 12

Índice de tabelas

Tabela 1 – Síntese dos trabalhos relacionados 23
Tabela 2 – Versões das ferramentas utilizadas nas experiências 26
Tabela 3 – Estatísticas do Dataset 1 32
Tabela 4 – Produtos mais frequentes (Dataset 1) 33
Tabela 5 – Regras com maior confiança (Dataset 1) 34
Tabela 6 – Regras com maior elevação (Dataset 1) 34
Tabela 7 – Equivalência dos resultados e tempo de extração dos conjuntos frequentes (Dataset 1) 35
Tabela 8 – Estatísticas do Dataset 2 38
Tabela 9 – Produtos mais frequentes (Dataset 2) 38
Tabela 10 – Regras com maior confiança (Dataset 2) 39
Tabela 11 – Regras com maior elevação (Dataset 2) 40
Tabela 12 – Equivalência dos resultados, tempo e memória na extração dos conjuntos frequentes (Dataset 2, max_len = 2) 43
Tabela 13 – Endpoints da API 50
Tabela 14 – Requisitos funcionais 53
Tabela 15 – Requisitos não funcionais 53
Tabela 16 – Atores do sistema 54
Tabela 17 – Ficha do caso de uso UC01 Criar conta 55
Tabela 18 – Ficha do caso de uso UC02 Autenticar-se 57
Tabela 19 – Ficha do caso de uso UC03 Terminar sessão 58
Tabela 20 – Ficha do caso de uso UC04 Consultar histórico de listas 60
Tabela 21 – Ficha do caso de uso UC05 Construir lista de compras 62
Tabela 22 – Ficha do caso de uso UC06 Consultar itens recomendados 63
Tabela 23 – Ficha do caso de uso UC07 Guardar lista de compras 65
Tabela 24 – Ficha do caso de uso UC08 Consultar estatísticas 67
Tabela 25 – Ficha do caso de uso UC09 Configurar parâmetros do motor 69
Tabela 26 – Testes automáticos da API 92
Tabela 27 – Resultados da avaliação offline das recomendações (Dataset 1, 80/20, cinco divisões) 94
Tabela 28 – Versões utilizadas no desenvolvimento e nos testes do sistema 95

Lista de abreviaturas, siglas e acrónimos

API – *Application Programming Interface* (interface de programação de aplicações)

CSV – *Comma-Separated Values* (valores separados por vírgulas)

DTO – *Data Transfer Object* (objeto de transferência de dados)

FP – *Frequent Pattern* (Padrão frequente)

HTTP – *Hypertext Transfer Protocol*

HTTPS – *Hypertext Transfer Protocol Secure*

JSON – *JavaScript Object Notation*

KDD – *Knowledge Discovery in Databases* (Descoberta de Conhecimento em Bases de Dados)

MBA – *Market Basket Analysis* (Análise de Cesto de Compras)

OLTP – *Online Transaction Processing* (Processamento de Transações em Linha)

ORM – *Object-Relational Mapping* (mapeamento objeto-relacional)

REST – *Representational State Transfer*

RF – Requisito funcional

RNF – Requisito não funcional

SKU – *Stock Keeping Unit* (unidade de manutenção de stock)

UC – *Use Case* (caso de uso)

UML – *Unified Modeling Language* (linguagem de modelação unificada)

1. Introdução

Uma das principais competências de qualquer organização consiste na capacidade de identificar oportunidades de crescimento, riscos, ameaças e, simultaneamente, prever resultados e tendências de mercado. No setor do retalho, nomeadamente em supermercados e minimercados, a análise dos dados de compra permite compreender o comportamento dos consumidores e identificar padrões de consumo relevantes. A utilização de técnicas de análise de dados e aprendizagem computacional torna possível transformar grandes volumes de dados transacionais em informação útil para apoio à decisão.

Num contexto de crescente disponibilidade de dados transacionais, a capacidade de analisar e extrair conhecimento a partir dos mesmos constitui uma vantagem competitiva relevante para as organizações. Neste sentido, a Análise de Cesto de Compras (*Market Basket Analysis*) assume um papel importante, permitindo identificar produtos frequentemente adquiridos em conjunto e apoiar estratégias de recomendação, organização de produtos e campanhas promocionais.

Este relatório apresenta de forma integrada o trabalho desenvolvido nas unidades curriculares de Projeto I e Projeto II, de modo a constituir um documento autocontido. Por esse motivo, os Capítulos 2 a 5 retomam e atualizam conteúdos desenvolvidos em Projeto I, enquanto o Capítulo 6 documenta sobretudo o sistema implementado em Projeto II. O relatório de Projeto I foi elaborado por João Tomé; em Projeto II, as três componentes do sistema e o presente relatório foram desenvolvidos em conjunto pelos dois autores. A versão do código correspondente a esta entrega é a disponibilizada no repositório indicado na secção 4.2. O presente capítulo encontra-se organizado nas secções Enquadramento, Objetivos do projeto, Planeamento do projeto e Estrutura do documento.

1.1 Enquadramento

O presente trabalho insere-se na área da Análise de Cesto de Compras e tem como finalidade explorar a utilização de técnicas de mineração de dados para identificar padrões de coocorrência entre produtos. Estas técnicas permitem extrair regras de associação a partir de grandes volumes de dados transacionais, possibilitando a identificação de itens que tendem a ser adquiridos em conjunto.

Neste âmbito, o presente trabalho enquadra-se na implementação de uma aplicação dirigida ao utilizador final, capaz de sugerir automaticamente produtos adicionais durante a construção de uma lista de compras. Para tal, recorre-se a modelos de

aprendizagem computacional baseados em regras de associação, nomeadamente os algoritmos Apriori e FP-Growth, amplamente utilizados neste domínio.

Através da análise dos dados de transações, torna-se possível identificar padrões frequentes e gerar regras com valores elevados de confiança e de elevação. Estas regras permitem inferir, para uma determinada seleção de produtos, quais os itens com maior probabilidade de serem adquiridos em conjunto, com potencial para melhorar a experiência do utilizador e apoiar estratégias de vendas.

Importa clarificar que a aplicação desenvolvida não está associada a nenhuma cadeia de supermercados específica, tratando-se antes de uma solução genérica de recomendação para listas de compras, estudada com dados de dois contextos distintos: um supermercado físico e uma plataforma de comércio eletrónico; o motor de recomendação implementado (Capítulo 6) utiliza o primeiro. Do ponto de vista do vendedor, o benefício potencial assenta sobretudo nas vendas cruzadas (*cross-selling*): ao sugerir, no momento da compra, produtos frequentemente adquiridos em conjunto com os já selecionados, a aplicação poderá aumentar o valor médio por transação e apoiar decisões de organização de produtos e de campanhas promocionais. Estes benefícios são potenciais e não foram avaliados neste trabalho; em particular, um eventual aumento das vendas não implica, por si só, aumento do lucro, cuja avaliação exigiria considerar margens, custos e um critério de otimização explícito.

1.2 Objetivos do projeto

O objetivo principal deste projeto consiste em estudar e aplicar técnicas de extração de regras de associação a dados transacionais reais e em integrá-las numa aplicação que sugira produtos durante a construção de uma lista de compras.

De forma mais específica, pretende-se:

- Estudar os conceitos fundamentais da Análise de Cesto de Compras e das regras de associação;
- Analisar e comparar algoritmos de extração de padrões frequentes, nomeadamente Apriori e FP-Growth, quanto aos resultados e ao desempenho computacional;
- Aplicar os algoritmos a diferentes conjuntos de dados, avaliando o impacto da estrutura e qualidade dos dados nos resultados obtidos;
- Desenvolver uma aplicação que permita sugerir produtos com base nos padrões identificados;
- Avaliar a utilidade das recomendações geradas para apoio à decisão e melhoria da experiência do utilizador.

A aplicação desenvolvida tem como objetivo apoiar o consumidor durante a criação de uma lista de compras, sugerindo automaticamente novos itens que apresentam elevada probabilidade de serem adquiridos em conjunto com os produtos já selecionados.

A utilidade das recomendações (quinto objetivo) poderia ser avaliada de duas formas complementares: através de uma avaliação offline, ocultando itens de cestos de teste não utilizados na extração das regras e verificando se o sistema os recomenda (por exemplo, com a precisão e a revocação nas cinco primeiras recomendações, comparadas com uma recomendação baseada apenas na popularidade dos itens), e através de uma avaliação com utilizadores reais, que medisse a utilidade percebida. As métricas das regras (suporte, confiança e elevação) e os testes funcionais da API não medem, por si só, essa utilidade. A avaliação offline foi realizada e é apresentada na secção 6.5.2; o Capítulo 7 discute em que medida este objetivo foi alcançado.

1.3 Planeamento do projeto

O planeamento do projeto, apresentado no cronograma da Figura 1, encontra-se organizado em duas fases principais, correspondentes ao Projeto I, de outubro de 2025 a janeiro de 2026, e ao Projeto II, de março a setembro de 2026. O cronograma representa as etapas tal como foram efetivamente executadas.

Na primeira fase (Projeto I), o trabalho foi estruturado nas seguintes etapas:

- **Pesquisa bibliográfica**: levantamento inicial de fontes relevantes na área da Análise de Cesto de Compras e da aprendizagem computacional;
- **Estado da arte**: estudo aprofundado dos conceitos, algoritmos e trabalhos relacionados na área;
- **Escolha de tecnologias**: seleção das linguagens, bibliotecas e ferramentas utilizadas no desenvolvimento;
- **Escolha da metodologia**: definição da abordagem metodológica a seguir ao longo do projeto;
- **Conjuntos de dados e algoritmos**: análise dos conjuntos de dados disponíveis e estudo dos algoritmos Apriori e FP-Growth;
- **Implementação e testes**: implementação prática dos algoritmos e realização dos testes experimentais sobre os conjuntos de dados escolhidos;
- **Redação do relatório I**: redação contínua do relatório ao longo de toda esta fase, acompanhando o desenvolvimento do trabalho;
- **Revisão e compilação**: revisão final e compilação do documento entregável do Projeto I.

Na segunda fase (Projeto II), dedicada ao desenvolvimento da aplicação de recomendações, as etapas foram as seguintes:

- **Definição de requisitos**: levantamento e definição dos requisitos funcionais e não funcionais da aplicação;

- ***Mockups* e diagramas**: elaboração dos *mockups* da interface e dos diagramas de modelação (casos de uso, robustez e sequência);
- **Desenvolvimento da API**: implementação da API REST em FastAPI, incluindo a autenticação, a persistência em SQLite e a integração do motor de recomendação;
- **Desenvolvimento das aplicações cliente**: implementação da aplicação móvel em Flutter, destinada aos utilizadores, e da aplicação de *backoffice* em Streamlit, destinada ao administrador;
- **Sistema de recomendação**: implementação da lógica de geração de recomendações com base nas regras de associação extraídas;
- **Testes e validação**: testes funcionais da aplicação e validação dos resultados obtidos;
- **Redação do capítulo 6**: redação contínua do capítulo relativo à aplicação desenvolvida, com uma pausa no mês de agosto e retoma em setembro;
- **Revisão final**: revisão final do relatório, como última etapa do projeto.

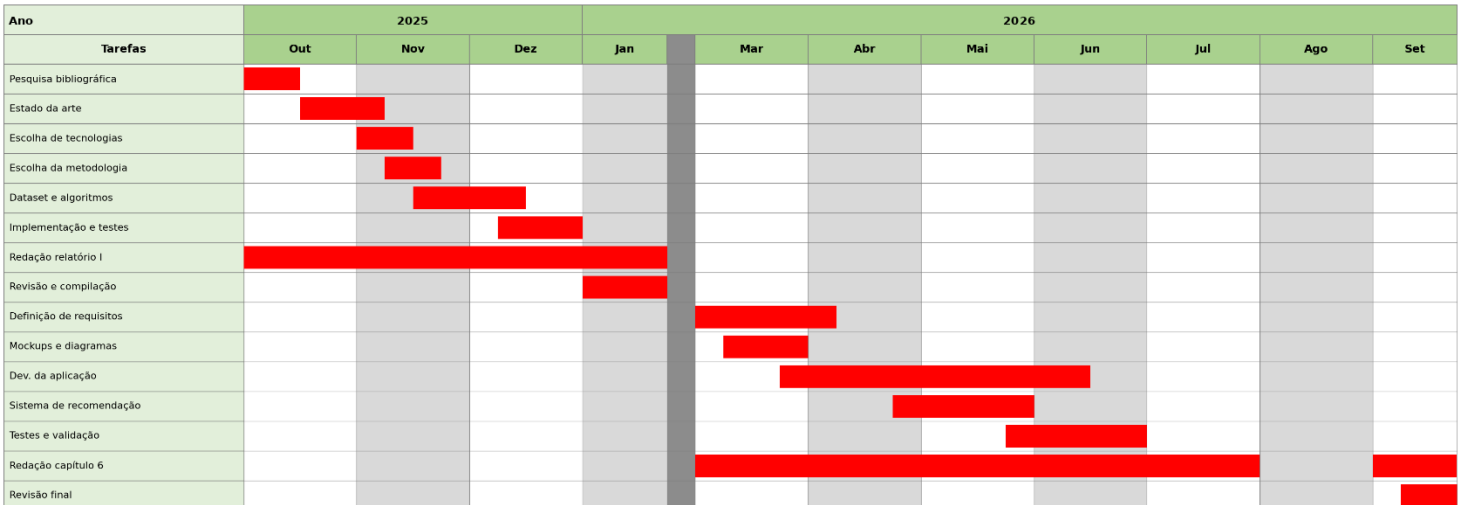


Figura 1 – Cronograma de Gantt do planeamento do Projeto I e do Projeto II

Este planeamento permitiu estruturar o desenvolvimento do trabalho de forma progressiva. Em relação ao planeado no início do Projeto II, a principal reformulação foi de arquitetura: o cálculo das regras de associação, que no protótipo inicial era feito pela própria aplicação de backoffice, passou a ser feito exclusivamente pela API, partilhada pela aplicação móvel e pelo backoffice (Capítulo 6). Na revisão final foram ainda repetidas e alargadas as medições de desempenho apresentadas no Capítulo 5.

1.4 Estrutura do documento

O presente relatório encontra-se organizado nos seguintes capítulos:

- **Capítulo 1 – Introdução:** Apresenta o enquadramento do projeto, os objetivos, o planeamento e a estrutura do documento.

- **Capítulo 2 – Fundamentação teórica:** Introduz os conceitos fundamentais de aprendizagem computacional, mineração de dados e Análise de Cesto de Compras, bem como os principais algoritmos utilizados neste domínio, terminando com uma síntese crítica.
- **Capítulo 3 – Trabalho relacionado:** Analisa trabalhos científicos e aplicações existentes relacionados com a temática abordada.
- **Capítulo 4 – Software e ferramentas utilizados:** Descreve as tecnologias, bibliotecas e ferramentas utilizadas na componente experimental e no desenvolvimento do sistema.
- **Capítulo 5 – Metodologia experimental e resultados:** Descreve a metodologia experimental, os conjuntos de dados utilizados e o respetivo tratamento, a aplicação dos algoritmos Apriori e FP-Growth e os resultados obtidos, terminando com uma discussão comparativa e com as implicações para o motor de recomendação.
- **Capítulo 6 – Análise, desenho e implementação do sistema:** Apresenta a arquitetura do sistema desenvolvido (API, aplicação de backoffice e aplicação móvel), as decisões do motor de recomendação, os requisitos, os casos de uso, os diagramas de robustez e de sequência, os testes, a avaliação offline das recomendações, as instruções de instalação e as interfaces implementadas.
- **Capítulo 7 – Conclusão:** Apresenta uma síntese do trabalho desenvolvido, limitações identificadas e propostas de evolução futura.

Por fim, é apresentada a **Bibliografia**, contendo as referências utilizadas.

2. Fundamentação teórica

Neste capítulo apresenta-se uma breve introdução à aprendizagem computacional, à sua origem e às diferentes abordagens, seguindo-se a mineração de dados e a técnica utilizada na implementação deste projeto.

2.1 Aprendizagem computacional

A aprendizagem computacional (*machine learning*) é uma área da Inteligência Artificial dedicada ao desenvolvimento de modelos computacionais capazes de aprender automaticamente a partir de dados, ajustando o seu comportamento com base na experiência adquirida. Ao contrário da programação tradicional, em que todas as regras são explicitamente definidas, os sistemas de aprendizagem computacional constroem modelos que generalizam padrões existentes nos dados, permitindo realizar previsões ou tomar decisões em cenários novos (Mitchell, 1997).

Segundo Bishop (2006), a aprendizagem computacional baseia-se fortemente em métodos estatísticos. Em muitos métodos, em particular na aprendizagem supervisionada, o processo de aprendizagem é formulado como um problema de otimização, no qual se procura minimizar o erro entre as previsões do modelo e os dados observados. Esta formulação não é, contudo, universal: na descoberta de padrões frequentes, a tarefa realizada neste trabalho, o objetivo não é prever um valor, mas identificar conjuntos de itens que ocorrem frequentemente em conjunto. O aumento da quantidade de dados disponíveis e da capacidade computacional impulsionou a adoção de ambas as abordagens em contextos reais.

Atualmente, a aprendizagem computacional é utilizada numa grande diversidade de aplicações, incluindo sistemas de recomendação, deteção de padrões, reconhecimento de imagem e análise de comportamento do consumidor. No setor do comércio e dos serviços digitais, estas técnicas permitem analisar grandes volumes de dados transacionais e identificar relações complexas entre eventos, fornecendo suporte à tomada de decisão e à personalização de produtos e serviços (Alpaydin, 2020).

Os métodos de aprendizagem computacional podem ser classificados em diferentes categorias, consoante o tipo de informação disponível durante o treino. Na aprendizagem supervisionada, o modelo é treinado com exemplos rotulados, sendo amplamente utilizada em tarefas de classificação e regressão. A Figura 2 ilustra este último caso: um conjunto de observações com uma reta de regressão ajustada, evidenciando uma correlação positiva entre a variável independente e a variável dependente, característica de um problema de regressão linear simples (adaptado de Montgomery, Peck & Vining, 2021). Em contraste, a aprendizagem não supervisionada trabalha com dados não rotulados, tendo como principal objetivo a descoberta de

estruturas subjacentes, tais como agrupamentos ou associações entre variáveis (Hastie, Tibshirani & Friedman, 2009).

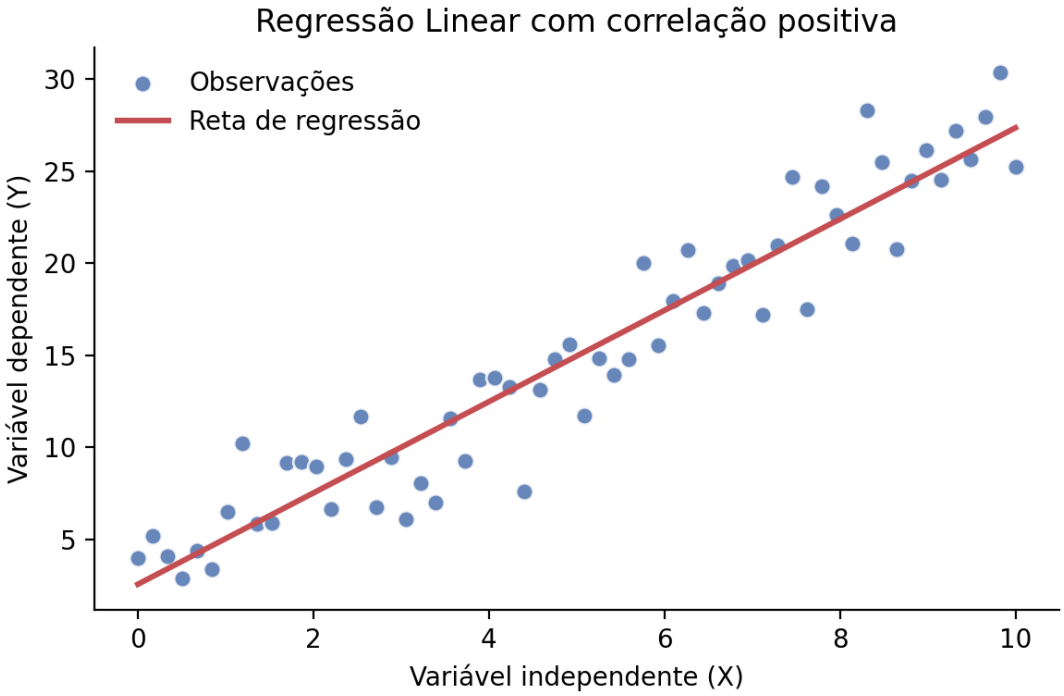


Figura 2 – Regressão linear com correlação positiva (adaptado de Montgomery, Peck & Vining, 2021)

A aprendizagem não supervisionada assume particular relevância em contextos onde não existe conhecimento prévio sobre as relações entre os dados. Um exemplo típico é a análise de dados transacionais, onde se pretende identificar padrões de compra recorrentes sem que exista uma classificação previamente definida. Neste tipo de problemas, os algoritmos procuram relações estatisticamente relevantes entre os itens presentes nos dados, permitindo extrair conhecimento útil a partir de grandes volumes de informação (Tan et al., 2019).

Existe ainda a aprendizagem por reforço, na qual um agente aprende através da interação com um ambiente dinâmico, ajustando a sua estratégia com base em recompensas ou penalizações associadas às ações executadas. Embora esta abordagem seja amplamente utilizada em áreas como robótica e jogos, não se enquadra diretamente no âmbito deste projeto, cujo foco está na análise exploratória de dados e na descoberta de padrões de associação (Sutton & Barto, 2018).

No contexto deste trabalho, a aprendizagem computacional é aplicada através de técnicas de aprendizagem não supervisionada, com o objetivo de identificar padrões de coocorrência entre itens em dados de compras. Estes padrões constituem a base para a análise de cesto de compras, permitindo a extração de regras de associação que podem ser utilizadas em sistemas de recomendação e apoio à decisão no domínio do retalho.

2.2 Mineração de dados

A mineração de dados é uma área da ciência de dados que se dedica à extração de padrões relevantes, relações e conhecimento implícito a partir de grandes volumes de dados. Esta área insere-se no processo mais abrangente designado por *Knowledge Discovery in Databases* (KDD), que engloba várias etapas sequenciais desde a seleção dos dados até à interpretação dos resultados obtidos. Segundo Fayyad, Piatetsky-Shapiro e Smyth (1996), a mineração de dados corresponde à fase central deste processo, na qual são aplicados algoritmos capazes de descobrir automaticamente informação útil e previamente desconhecida.

A Figura 3 esquematiza este processo, segundo Fayyad, Piatetsky-Shapiro e Smyth (1996). O processo KDD inicia-se com a seleção dos dados relevantes a partir de diferentes fontes, seguindo-se o pré-processamento, que inclui tarefas como limpeza, integração e transformação dos dados. Após esta fase, aplica-se a mineração de dados propriamente dita, recorrendo a métodos estatísticos, algoritmos de aprendizagem computacional e técnicas de reconhecimento de padrões. Por fim, os resultados obtidos são avaliados e interpretados, de modo a garantir que o conhecimento extraído é válido, compreensível e útil para o domínio em questão (Han, Kamber & Pei, 2012).

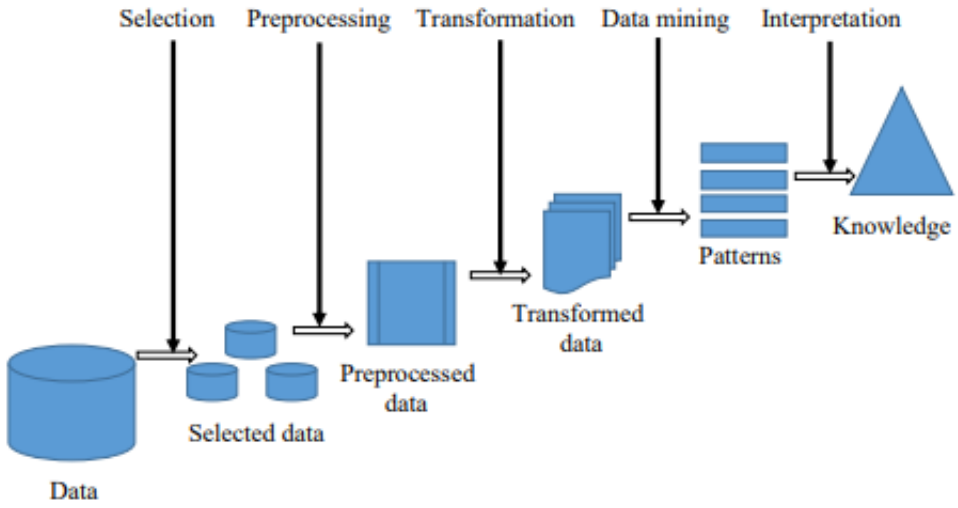


Figura 3 – Processo KDD (Knowledge Discovery in Databases) (adaptado de Fayyad, Piatetsky-Shapiro & Smyth, 1996)

O crescente interesse pela mineração de dados está diretamente relacionado com o aumento exponencial da quantidade de dados gerados por sistemas informáticos modernos. Aplicações empresariais, plataformas de comércio eletrónico, redes sociais e sistemas transacionais produzem diariamente grandes volumes de dados, cuja análise manual se torna impraticável. Neste contexto, a mineração de dados surge como uma ferramenta essencial para transformar dados brutos em informação acionável, apoiando a tomada de decisão estratégica (Witten et al., 2016).

Do ponto de vista histórico, várias técnicas que hoje integram a mineração de dados têm origem em métodos estatísticos. O Teorema de Bayes (Equação 1), que relaciona probabilidades condicionais, é um desses fundamentos (Murphy, 2022). Neste trabalho, a sua relevância é apenas conceptual: a confiança de uma regra de associação corresponde a uma probabilidade condicional, P(B|A), estimada diretamente a partir das frequências observadas, sem recurso a inferência bayesiana.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Equação 1 – Teorema de Bayes

Com o avanço da computação, tornou-se possível aplicar estas técnicas a conjuntos de dados de grande dimensão. A introdução dos sistemas de gestão de bases de dados relacionais, na década de 1970, permitiu armazenar e consultar dados de forma estruturada, enquanto o desenvolvimento de sistemas transacionais (*Online Transaction Processing* – OLTP) possibilitou o processamento de grandes volumes de transações em tempo real (Silberschatz, Korth & Sudarshan, 2019). Estes avanços tecnológicos criaram as condições necessárias para a consolidação da mineração de dados como área autónoma de investigação e aplicação prática (Fayyad, Piatetsky-Shapiro & Smyth, 1996).

Inicialmente, a mineração de dados incidia maioritariamente sobre dados estruturados, organizados em tabelas relacionais. Contudo, com a evolução tecnológica e a diversidade crescente dos dados disponíveis, surgiram novas áreas como a mineração de texto, a mineração de imagens e a mineração de grafos. Estas abordagens permitem analisar dados não estruturados ou semiestruturados, ampliando significativamente o campo de aplicação das técnicas de mineração de dados (Aggarwal, 2015).

No contexto dos dados transacionais, a mineração de dados é frequentemente utilizada para identificar padrões de comportamento dos utilizadores, nomeadamente em ambientes de retalho e comércio eletrónico. A análise destas transações permite descobrir associações entre itens adquiridos em conjunto, padrões de consumo recorrentes e relações que não são imediatamente evidentes através de análises tradicionais. Estas descobertas são particularmente relevantes para estratégias de marketing, organização de produtos e sistemas de recomendação (Tan et al., 2019).

Neste projeto, a mineração de dados é aplicada à análise de cesto de compras, utilizando dados transacionais para identificar associações entre produtos. Através da aplicação de algoritmos específicos de descoberta de padrões frequentes, é possível extrair regras de associação que constituem a base para a recomendação de itens adicionais, podendo contribuir para a melhoria da experiência do utilizador e para o aumento do valor das transações comerciais.

2.3 Análise de Cesto de Compras

A Análise de Cesto de Compras (*Market Basket Analysis* – MBA) é uma técnica de mineração de dados, com origem no trabalho de Agrawal, Imieliński e Swami (1993), utilizada para identificar padrões de coocorrência entre itens em bases de dados transacionais. O seu objetivo principal consiste em descobrir associações significativas entre produtos frequentemente adquiridos em conjunto, permitindo compreender o comportamento de compra dos consumidores e apoiar a tomada de decisões estratégicas em diversos contextos, nomeadamente no retalho e no comércio eletrónico (Tan et al., 2019).

Esta técnica baseia-se na extração de regras de associação do tipo $A \Rightarrow B$, em que A (antecedente) e B (consequente) são conjuntos de itens não vazios e disjuntos ($A \cap B = \emptyset$); a regra indica que, nas transações que contêm A, B também ocorre com uma determinada frequência. A descoberta destas regras permite identificar relações implícitas nos dados, que não são facilmente detetáveis através de métodos de análise tradicionais, tornando a Análise de Cesto de Compras uma ferramenta relevante para análise exploratória de dados (Han, Kamber & Pei, 2012). Uma regra de associação descreve uma coocorrência observada e não estabelece uma relação de causalidade. Nas experiências do Capítulo 5, as regras foram restringidas a um único item no antecedente e no consequente; o motor de recomendação (Capítulo 6) não aplica esta restrição.

A Análise de Cesto de Compras tem sido amplamente aplicada em sistemas de recomendação, campanhas promocionais e organização de produtos em espaços comerciais. No contexto dos sistemas de recomendação, esta abordagem permite sugerir itens adicionais aos consumidores com base nos produtos já selecionados, contribuindo para o aumento do valor médio por transação e para a melhoria da experiência do utilizador (Kaur & Kang, 2016). Além disso, a Análise de Cesto de Compras pode ser utilizada para otimizar a disposição dos produtos em loja, identificando itens que devem ser colocados em proximidade para estimular compras conjuntas (Han, Kamber & Pei, 2012).

A avaliação da relevância das regras de associação é realizada com base em métricas estatísticas que permitem quantificar a força e a utilidade das associações descobertas. As métricas mais utilizadas são o suporte, a confiança e a elevação (*lift*), as quais permitem filtrar regras pouco significativas e destacar associações relevantes do ponto de vista prático (Tan et al., 2019).

O **suporte** (Equação 2) mede a frequência relativa com que um conjunto de itens ocorre no conjunto total de N transações. O suporte de uma regra $A \Rightarrow B$ corresponde ao suporte da união $A \cup B$, isto é, à proporção $n(A \cup B)/N$ de transações que contêm simultaneamente todos os itens de A e de B, em que n(X) designa o número de

transações que contêm X; distinguem-se ainda o suporte do antecedente e o do consequente, usados no cálculo da confiança e da elevação. O suporte é uma medida de frequência e não um teste de significância estatística; o limiar de suporte mínimo serve para filtrar conjuntos de itens com baixa frequência observada.

$$\text{suporte}(A \Rightarrow B) = \frac{n(A \cup B)}{N}$$

Equação 2 – Cálculo do valor de suporte

A **confiança** (Equação 3) expressa a probabilidade condicional de ocorrência do item consequente B, dado que o antecedente A ocorreu numa transação. Esta métrica indica o grau de fiabilidade de uma regra de associação, refletindo a frequência com que B ocorre quando A está presente (Aggarwal, 2015).

$$\text{confiança}(A \Rightarrow B) = \frac{\text{suporte}(A \cup B)}{\text{suporte}(A)}$$

Equação 3 – Cálculo do valor de confiança

A **elevação** (*lift*) (Equação 4) compara a confiança da regra com o suporte do consequente. Valores superiores a 1 indicam que, na amostra analisada, A e B ocorrem juntos com maior frequência do que seria esperado se fossem independentes (associação positiva); valores inferiores a 1 indicam associação negativa e valores próximos de 1 ausência de associação (Han, Kamber & Pei, 2012). Um valor de elevação superior a 1 não demonstra causalidade nem exclui que a associação seja fortuita, sobretudo quando o suporte é reduzido.

$$\text{elevação}(A \Rightarrow B) = \frac{\text{confiança}(A \Rightarrow B)}{\text{suporte}(B)}$$

Equação 4 – Cálculo do valor de elevação

Apesar das vantagens da Análise de Cesto de Compras, esta técnica apresenta desafios significativos, nomeadamente a geração de um elevado número de regras, muitas das quais podem ser redundantes ou pouco relevantes. Este problema é particularmente evidente em bases de dados de grande dimensão, onde a escolha inadequada dos valores mínimos de suporte e confiança pode resultar numa explosão combinatória de regras (Aggarwal, 2015). Assim, a seleção criteriosa dos parâmetros e dos algoritmos de extração assume um papel fundamental na obtenção de resultados úteis.

No âmbito deste projeto, a Análise de Cesto de Compras é utilizada como base para a construção de um sistema de recomendação de produtos, recorrendo a algoritmos de descoberta de padrões frequentes. A aplicação desta técnica permite extrair conhecimento a partir de dados reais de transações, possibilitando a sugestão automática de itens adicionais aos consumidores com base em padrões de compra previamente observados.

2.4 Apriori

O algoritmo Apriori é um dos métodos clássicos e mais difundidos para a descoberta de conjuntos de itens frequentes e regras de associação em bases de dados transacionais. Foi proposto por Agrawal e Srikant (1994) no contexto da análise de grandes bases de dados comerciais, tendo como principal objetivo identificar padrões de compra recorrentes de forma sistemática e eficiente.

O funcionamento do algoritmo Apriori (Figura 4) baseia-se no chamado **princípio Apriori**, que estabelece que, se um conjunto de itens não for frequente, então nenhum dos seus superconjuntos poderá ser frequente. Este princípio permite reduzir significativamente o espaço de pesquisa, eliminando antecipadamente conjuntos de itens que não satisfazem o valor mínimo de suporte definido, tornando o processo de mineração mais eficiente (Han, Kamber & Pei, 2012).

O algoritmo opera de forma iterativa, começando pela identificação dos conjuntos de itens frequentes de tamanho 1 (*1-itemsets*). Nesta fase inicial, o algoritmo percorre a base de dados para calcular o suporte de cada item individual, eliminando aqueles cujo suporte seja inferior ao valor mínimo estabelecido. Na iteração seguinte (k = 2, 3, …), os conjuntos frequentes de tamanho k−1 são combinados entre si para gerar candidatos de tamanho k, num processo designado por *join*. São depois eliminados os candidatos que contenham algum subconjunto de tamanho k−1 não frequente (passo *prune*), por aplicação do princípio Apriori. Só então a base de dados é percorrida para contar o suporte dos candidatos restantes, mantendo-se apenas os que atingem o suporte mínimo (Aggarwal, 2015).

Este processo repete-se de forma iterativa, aumentando progressivamente o tamanho dos conjuntos de itens até que não seja possível gerar novos conjuntos frequentes. Após a obtenção dos conjuntos de itens frequentes, o algoritmo procede à geração das regras de associação, calculando a confiança de cada regra candidata e mantendo apenas as que atingem a confiança mínima; métricas adicionais, como a elevação, são normalmente calculadas *a posteriori*, para ordenar ou filtrar as regras (Tan et al., 2019).

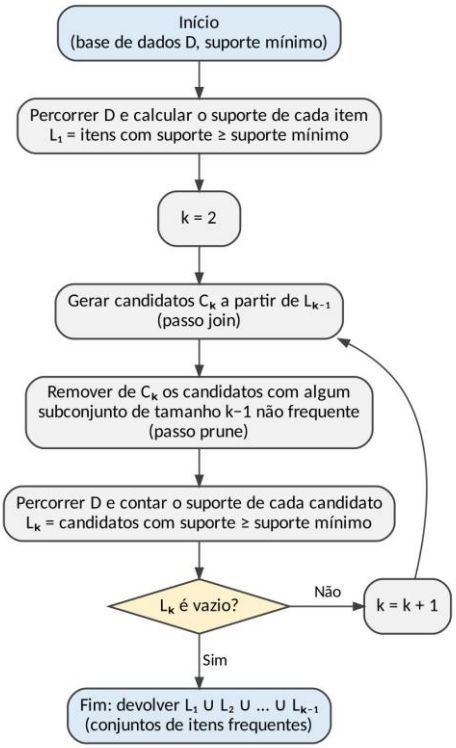


Figura 4 – Algoritmo Apriori (elaboração própria)

Uma das principais vantagens do algoritmo Apriori reside na sua simplicidade conceptual e facilidade de implementação, o que o torna particularmente adequado para fins educativos e para conjuntos de dados de pequena ou média dimensão. Além disso, o algoritmo produz regras de associação facilmente interpretáveis, o que facilita a sua utilização em contextos de apoio à decisão (Han, Kamber & Pei, 2012).

Contudo, o algoritmo Apriori apresenta limitações significativas em termos de desempenho, especialmente quando aplicado a bases de dados de grande dimensão ou com um elevado número de itens distintos. A necessidade de realizar múltiplas passagens pela base de dados e a geração de um grande número de conjuntos candidatos resultam num elevado custo computacional e consumo de memória. Estas limitações motivaram o desenvolvimento de algoritmos mais eficientes, como o FP-Growth, que procuram mitigar estes problemas (Aggarwal, 2015).

No contexto deste projeto, o algoritmo Apriori é utilizado como uma abordagem de referência para a descoberta de regras de associação, permitindo comparar os resultados obtidos com os do FP-Growth quando aplicados a dados reais de cestos de compras. É também o algoritmo utilizado pelo motor de recomendação do sistema (Capítulo 6).

2.5 FP-Growth

O algoritmo FP-Growth (*Frequent Pattern Growth*) foi proposto por Han, Pei e Yin (2000) como uma alternativa mais eficiente ao algoritmo Apriori para a descoberta de conjuntos de itens frequentes em bases de dados transacionais. O seu principal objetivo é reduzir o custo computacional associado à geração de candidatos e às múltiplas passagens pela base de dados, problemas que limitam a escalabilidade do algoritmo Apriori em conjuntos de dados de grande dimensão.

Ao contrário do Apriori, o FP-Growth não gera explicitamente conjuntos candidatos. Em vez disso, constrói uma estrutura de dados compacta denominada **FP-*tree*** *(Frequent Pattern Tree)* (Figura 5), que representa as transações da base de dados de forma comprimida. Esta árvore armazena apenas os itens frequentes, organizados de acordo com a sua frequência, permitindo que padrões frequentes sejam extraídos diretamente da estrutura sem necessidade de reprocessar repetidamente a base de dados original (Han, Kamber & Pei, 2012).

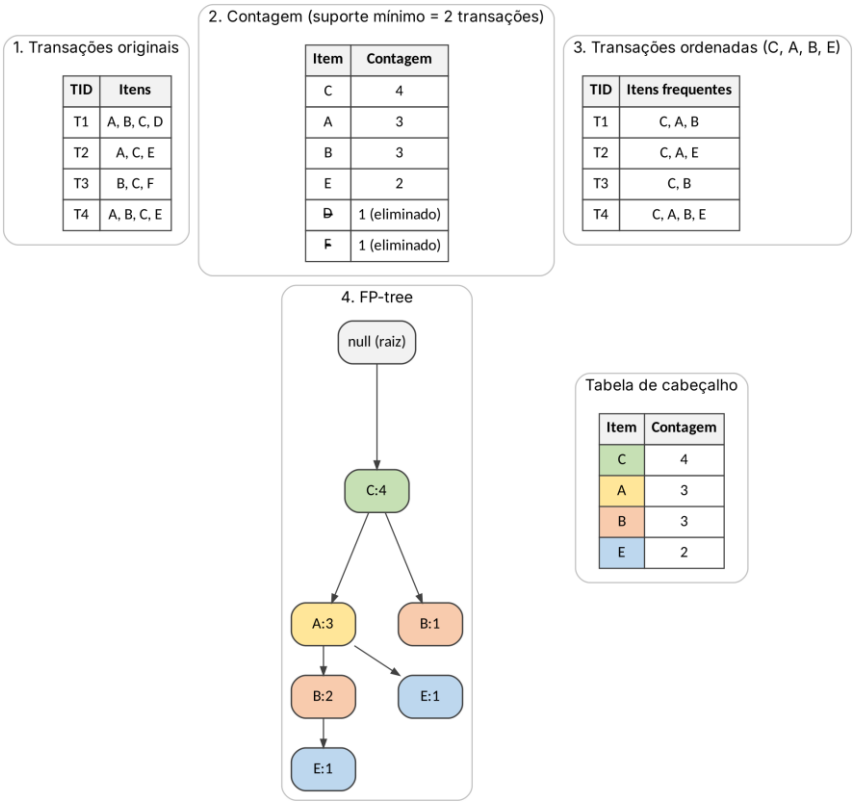


Figura 5 – Construção de uma FP-tree a partir de quatro transações (suporte mínimo de 2 transações; elaboração própria)

O funcionamento do algoritmo FP-Growth pode ser dividido em duas fases principais. Na primeira fase, a base de dados é percorrida para calcular a frequência de

cada item, sendo eliminados os itens que não atingem o valor mínimo de suporte. Em seguida, as transações são reorganizadas de acordo com a ordem decrescente de frequência dos itens e inseridas na *FP-tree*, onde cada caminho da raiz até um nó representa um conjunto de itens partilhado por várias transações, permitindo que padrões comuns sejam armazenados de forma compacta e sem repetição de informação. No exemplo da Figura 5, com um suporte mínimo de 2 transações, os itens D e F são eliminados e os restantes são ordenados por frequência decrescente (C, A, B, E) antes de serem inseridos na árvore.

Na segunda fase, o algoritmo explora recursivamente a *FP-tree* para extrair conjuntos de itens frequentes. Para cada item, é construída uma base de padrões condicionais, a partir da qual é gerada uma *FP-tree* condicional. Este processo repete-se até que todos os padrões frequentes sejam identificados. Esta abordagem permite decompor o problema original em subproblemas menores, tornando o processo de mineração mais eficiente (Han, Pei & Yin, 2000).

Uma das principais vantagens do FP-Growth é evitar a geração de candidatos e reduzir o número de passagens pela base de dados, o que tende a torná-lo mais eficiente em bases de dados grandes e densas. Han, Pei e Yin (2000) mostraram experimentalmente que o FP-Growth pode ser substancialmente mais rápido do que o Apriori em bases de dados de grande dimensão, sobretudo quando existem muitos padrões frequentes; a vantagem depende, contudo, das características dos dados e da implementação utilizada, como se verifica no Capítulo 5.

No entanto, o FP-Growth também apresenta algumas limitações. A construção e o armazenamento da *FP-tree* podem tornar-se dispendiosos em bases de dados extremamente grandes, e a complexidade da estrutura pode dificultar a interpretação direta do processo de mineração. Além disso, a implementação do FP-Growth é, em geral, mais complexa do que a do Apriori, o que pode representar um obstáculo em contextos educativos ou em sistemas com recursos limitados (Aggarwal, 2015).

No âmbito deste projeto, o algoritmo FP-Growth é utilizado em conjunto com o Apriori. Como ambos são métodos exatos, para o mesmo conjunto de transações, o mesmo suporte mínimo e os mesmos critérios de geração e filtragem de regras, devem produzir os mesmos conjuntos frequentes e as mesmas regras; a sua aplicação conjunta serve, por isso, para verificar a equivalência dos resultados e para comparar o seu desempenho computacional (tempo de execução e memória), apresentado no Capítulo 5.

2.6 Síntese crítica do estado da arte e enquadramento do projeto

A análise da literatura demonstra que a Análise de Cesto de Compras é uma técnica consolidada e amplamente utilizada para a descoberta de padrões de compra em dados transacionais, com aplicações relevantes em sistemas de recomendação, estratégias de marketing e organização de produtos em ambientes físicos e digitais (Agrawal, Imieliński & Swami, 1993; Kaur & Kang, 2016). A utilização de regras de associação permite transformar dados históricos de vendas em conhecimento útil para apoio à decisão, sendo particularmente eficaz em contextos onde se pretende compreender comportamentos de consumo recorrentes.

Os estudos analisados evidenciam que os algoritmos Apriori e FP-Growth continuam a ser as abordagens mais utilizadas para a extração de padrões frequentes, apesar das diferenças significativas em termos de desempenho computacional. O Apriori destaca-se pela sua simplicidade e facilidade de interpretação, sendo adequado para conjuntos de dados de pequena ou média dimensão e para fins educativos. No entanto, apresenta limitações claras em termos de escalabilidade, devido à necessidade de múltiplas passagens pela base de dados e à geração de um elevado número de conjuntos candidatos (Aggarwal, 2015; Han, Kamber & Pei, 2012).

Por outro lado, o algoritmo FP-Growth surge como uma solução mais eficiente para bases de dados de grande dimensão, eliminando a geração explícita de candidatos e recorrendo a uma estrutura de dados compacta para representar transações comprimidas. Han, Pei e Yin (2000) mostraram que este algoritmo pode ser substancialmente mais rápido do que o Apriori em bases de dados grandes e densas; a vantagem depende, contudo, dos dados e da implementação, como se verifica no Capítulo 5. Além disso, a sua implementação é mais complexa e a compreensão do processo de extração pode ser menos intuitiva quando comparada com a abordagem do Apriori.

Apesar da eficácia comprovada das regras de associação, vários autores apontam limitações relacionadas com a geração excessiva de regras, muitas das quais são redundantes ou pouco relevantes do ponto de vista prático. Este fenómeno exige uma escolha criteriosa dos valores mínimos de suporte e confiança, bem como a utilização de métricas adicionais, como a elevação (*lift*), para selecionar as associações mais informativas e evitar correlações triviais causadas por itens muito populares (Aggarwal, 2015; Han, Kamber & Pei, 2012).

Nos trabalhos consultados (Capítulo 3), predomina o foco na extração de regras, sendo menos frequentes a integração dos resultados em aplicações orientadas ao utilizador final e a avaliação da utilidade das recomendações em contexto real. Esta

observação limita-se à amostra de trabalhos analisada e não pretende generalizar para toda a literatura.

Face a estas limitações, verifica-se uma tendência crescente para integrar a Análise de Cesto de Compras em sistemas mais abrangentes, combinando técnicas de mineração de dados com modelos de aprendizagem computacional e sistemas de recomendação personalizados. Esta integração permite evoluir de uma análise meramente exploratória para aplicações práticas orientadas ao utilizador, capazes de fornecer sugestões relevantes em tempo real com base em padrões históricos de compra (Tan et al., 2019).

Neste contexto, o presente projeto enquadra-se na aplicação prática da Análise de Cesto de Compras para suporte a recomendações automáticas de produtos, utilizando dados transacionais reais e recorrendo aos algoritmos Apriori e FP-Growth, que, sendo exatos, produzem as mesmas regras com os mesmos parâmetros; a sua comparação incide, por isso, na verificação dessa equivalência e no desempenho computacional (tempo de execução e memória).

Adicionalmente, o projeto procura aproximar a análise teórica da aplicação prática, integrando os resultados da mineração de dados numa aplicação funcional orientada ao utilizador. Pretende-se, assim, demonstrar a viabilidade técnica desta integração; a avaliação da utilidade das recomendações junto de utilizadores reais fica para trabalho futuro.

3. Trabalho relacionado

Neste capítulo apresenta-se o trabalho relacionado com a Análise de Cesto de Compras. A secção 3.1 descreve a aplicação desta técnica em diferentes setores, a secção 3.2 analisa estudos realizados no retalho e em supermercados e a secção 3.3 aborda propostas mais recentes que combinam regras de associação com outras técnicas, terminando com o posicionamento do presente trabalho.

3.1 Aplicações da Análise de Cesto de Compras em diferentes setores

A Análise de Cesto de Compras tem sido amplamente aplicada em diversos setores, devido à sua capacidade de identificar padrões de coocorrência entre itens e eventos em dados transacionais. Embora tenha surgido no contexto do retalho, esta técnica tem vindo a ser adaptada a múltiplos domínios, onde a descoberta de associações permite compreender comportamentos, otimizar processos e apoiar sistemas de recomendação (Tan et al., 2019).

No setor do retalho, a análise de cesto de compras é utilizada para identificar produtos frequentemente adquiridos em conjunto, permitindo otimizar a disposição de produtos em loja, definir estratégias de vendas cruzadas e planear campanhas promocionais direcionadas. A colocação estratégica de produtos associados pode contribuir para aumentar o valor médio por transação e tornar as estratégias de marketing mais eficazes (Kaur & Kang, 2016). Além disso, as regras de associação permitem prever padrões de procura e apoiar decisões de gestão de inventário, reduzindo ruturas de stock e excesso de armazenamento (Han, Kamber & Pei, 2012).

No comércio eletrónico, a Análise de Cesto de Compras desempenha um papel central em sistemas de recomendação personalizados. Plataformas digitais utilizam padrões de coocorrência entre produtos para sugerir itens complementares durante o processo de compra, aumentando a probabilidade de aquisição adicional e melhorando a experiência do utilizador. Este tipo de recomendação baseada em associação é frequentemente combinado com técnicas de filtragem colaborativa e modelos de aprendizagem computacional para aumentar a precisão das sugestões apresentadas ao cliente (Aggarwal, 2016).

Na área da saúde, a mineração de dados tem sido aplicada a registos clínicos para apoiar o diagnóstico, o prognóstico e a definição de tratamentos (Bellazzi & Zupan, 2008). Neste contexto, a descoberta de padrões frequentes pode ajudar a identificar associações entre diagnósticos, tratamentos e medicações que não são imediatamente evidentes através de análises tradicionais.

No setor financeiro, a análise de cesto de compras é utilizada para identificar padrões de utilização conjunta de produtos bancários, como contas, cartões de crédito,

seguros e investimentos. Estas associações permitem desenvolver estratégias de *cross-selling* e segmentação de clientes, aumentando a fidelização e o valor do cliente ao longo do tempo. A mineração de padrões transacionais também é utilizada para deteção de comportamentos anómalos e prevenção de fraude, ao identificar combinações de transações que divergem dos padrões habituais (Aggarwal, 2015).

Em plataformas de serviços digitais e de conteúdos, como serviços de streaming e aplicações móveis, a análise de padrões de consumo é utilizada para recomendar conteúdos e funcionalidades com base no histórico de utilização dos utilizadores. A identificação de associações entre conteúdos consumidos permite prever preferências futuras e personalizar interfaces, aumentando o tempo de utilização e a satisfação do utilizador. Estas abordagens são frequentemente integradas em sistemas de recomendação híbridos, combinando regras de associação com modelos preditivos mais avançados (Ricci, Rokach & Shapira, 2022).

Neste contexto, a aplicação da Análise de Cesto de Compras no presente projeto visa não apenas a descoberta de padrões de compra, mas também a sua integração numa aplicação de apoio à criação de listas de compras, permitindo demonstrar a aplicação prática das regras de associação num sistema orientado ao utilizador final. Esta abordagem procura aproximar a análise teórica dos dados à implementação de sistemas de recomendação interativos, alinhando-se com as tendências atuais de integração entre mineração de dados e aplicações inteligentes.

3.2 Estudos no retalho e em supermercados

A aplicação da Análise de Cesto de Compras no setor do retalho tem sido amplamente explorada, sobretudo devido à disponibilidade de grandes volumes de dados transacionais e à relevância económica da otimização das estratégias de vendas. Diversos estudos analisam o comportamento de compra dos consumidores em supermercados e lojas de conveniência, recorrendo principalmente a algoritmos de regras de associação como o Apriori e o FP-Growth.

Kurniawan et al. (2018) desenvolveram uma aplicação para computador que aplica o algoritmo Apriori a dados de transações de um estabelecimento comercial, com o objetivo de identificar padrões de compra dos clientes. Os autores obtiveram 30 regras de associação, com valores médios de confiança e de suporte de 46,69% e 1,78%, respetivamente. O estudo ilustra a aplicação do Apriori a bases de dados de pequena dimensão.

Mustakim et al. (2018) aplicaram os algoritmos Apriori e FP-Growth a dados de transações do minimercado Berkah Mart, em Pekanbaru (Indonésia), com o objetivo de analisar padrões de consumo. Segundo os autores, o FP-Growth permitiu obter regras de associação informativas de forma rápida e eficiente. Importa notar que, para os mesmos dados, limiares e critérios de filtragem, ambos os algoritmos produzem os mesmos padrões frequentes; eventuais diferenças no número de regras reportadas

resultam de diferenças de implementação, de filtragem ou das condições experimentais, e não de uma superioridade do FP-Growth na qualidade das regras.

Santos (2020), num trabalho de projeto realizado na NOVA Information Management School, aplicou regras de associação e técnicas de segmentação de clientes a dados de um supermercado, identificando relações entre departamentos e subcategorias de produtos. Com base nesses resultados, são propostas a reorganização de categorias numa aplicação móvel de compras e a sugestão de produtos relacionados durante o processo de compra.

Outros estudos procuram enriquecer as regras com fontes de dados adicionais. Kutuzova e Melnik (2018) analisaram dados provenientes de múltiplas fontes heterogéneas para melhorar sistemas de recomendação baseados em padrões de compra. Os autores concluíram que a combinação de dados transacionais com outros tipos de informação permite enriquecer as regras de associação e melhorar a relevância das recomendações apresentadas aos clientes.

De forma geral, os estudos no domínio do retalho confirmam a utilidade da Análise de Cesto de Compras na descoberta de padrões de compra relevantes e no apoio a estratégias comerciais. No entanto, nos trabalhos consultados, verifica-se que muitos se limitam à extração e análise das regras, sem proceder a uma avaliação sistemática da utilidade das recomendações em ambientes de utilização real. Além disso, nem sempre é realizada uma comparação detalhada entre diferentes algoritmos quanto ao desempenho computacional e à qualidade das regras geradas, o que dificulta a seleção da abordagem mais adequada para aplicações práticas.

Neste contexto, o presente projeto procura contribuir para colmatar estas lacunas, ao comparar explicitamente os algoritmos Apriori e FP-Growth utilizando dois conjuntos de dados reais e ao integrar os resultados num sistema de recomendação orientado ao utilizador. Esta abordagem permite comparar os resultados dos dois algoritmos e demonstrar a aplicação prática das recomendações geradas, alinhando-se com as necessidades reais de aplicações no domínio do retalho.

3.3 Abordagens avançadas e integração com aprendizagem computacional

Embora as regras de associação clássicas continuem a ser amplamente utilizadas na Análise de Cesto de Compras, a literatura recente tem explorado abordagens mais avançadas que combinam técnicas de mineração de dados com modelos de aprendizagem computacional, visualização de redes e métodos de segmentação de clientes. Estas abordagens visam ultrapassar algumas limitações das regras tradicionais, como a geração excessiva de padrões e a dificuldade em interpretar grandes volumes de regras.

Uma das abordagens emergentes consiste na utilização de grafos para representar padrões de coocorrência entre produtos. Em vez de gerar listas extensas de regras, os

produtos são representados como nós e as relações de compra conjunta como arestas, permitindo a identificação visual de comunidades de produtos frequentemente adquiridos em conjunto. Gino et al. (2023) modelaram dados de cestos de compras como redes temporais e mostraram que a sua análise exploratória permite identificar grupos de produtos relacionados, sugestivos de compras temáticas, e examinar a relação entre o rendimento dos clientes e o seu comportamento de compra. Apesar das vantagens em termos de interpretabilidade visual, esta abordagem não substitui totalmente as regras de associação quando são necessárias recomendações automáticas e quantitativamente avaliáveis.

Outra linha de investigação foca-se na melhoria estatística das regras de associação, procurando reduzir correlações espúrias causadas por produtos altamente populares. Dogan (2024) propôs complementar as métricas tradicionais com testes de significância estatística das correlações encontradas nas transações de venda e com atributos específicos dos produtos, como a taxa de desconto e o número de favoritos, obtendo regras condicionadas a essas características para recomendação em comércio eletrónico. No entanto, esta abordagem implica maior complexidade computacional e requer bases de dados mais ricas em atributos descritivos.

Alguns estudos propõem ainda sistemas híbridos de recomendação que combinam regras de associação com filtragem colaborativa e modelos baseados em conteúdo. Esta combinação permite explorar simultaneamente padrões globais de compra e preferências individuais dos utilizadores, mitigando problemas como o *cold start* e aumentando a diversidade das recomendações apresentadas (Aggarwal, 2016; Ricci, Rokach & Shapira, 2022). Estes sistemas são particularmente relevantes em plataformas de grande escala, onde a personalização desempenha um papel central na experiência do utilizador.

Para além do Apriori e do FP-Growth, existem outros algoritmos exatos para a extração de conjuntos frequentes, como o Eclat (Zaki, 2000), que utiliza uma representação vertical dos dados — para cada item, a lista das transações em que ocorre — e calcula o suporte por interseção dessas listas. Também ao nível das métricas existem alternativas ao suporte, à confiança e à elevação, como a convicção (Brin et al., 1997) ou a medida de Piatetsky-Shapiro, também designada *leverage* (Tan et al., 2019), que procuram captar melhor a força e a direção das associações. Neste trabalho optou-se pelo Apriori e pelo FP-Growth, por serem os algoritmos mais referidos nos estudos analisados e por estarem disponíveis na mesma biblioteca, com a mesma interface, e pelas três métricas clássicas, por serem as mais utilizadas e as mais fáceis de interpretar.

Apesar das vantagens destas abordagens avançadas, a sua implementação implica maior complexidade técnica, maior volume de dados e custos computacionais superiores. Para projetos de menor escala ou com objetivos pedagógicos, as regras de associação clássicas continuam a representar uma solução eficaz, permitindo obter resultados interpretáveis com menor esforço de implementação. Assim, a escolha da

abordagem deve considerar o contexto de aplicação, os recursos disponíveis e os objetivos do sistema (Tan et al., 2019).

Neste projeto, opta-se por uma abordagem baseada em regras de associação clássicas, utilizando os algoritmos Apriori e FP-Growth, de forma a garantir simplicidade de implementação, interpretabilidade dos resultados e viabilidade computacional. No entanto, a revisão da literatura evidencia que existe potencial para evolução futura do sistema, através da integração com técnicas de segmentação de clientes e modelos de recomendação híbridos, o que poderá ser explorado como trabalho futuro.

3.4 Síntese dos trabalhos relacionados

A Tabela 1 resume os trabalhos analisados e a sua relação com o presente projeto.

Tabela 1 – Síntese dos trabalhos relacionados

Trabalho	Dados	Técnica	Resultado principal
Kurniawan et al. (2018)	Transações de um estabelecimento comercial	Apriori (aplicação para computador)	30 regras; confiança média de 46,69% e suporte médio de 1,78%
Mustakim et al. (2018)	Minimercado Berkah Mart (Indonésia)	Apriori e FP-Growth	Regras informativas; FP-Growth descrito como rápido e eficiente
Santos (2020)	Supermercado	Regras de associação e segmentação de clientes	Relações entre departamentos; propostas para uma aplicação móvel
Kutuzova e Melnik (2018)	Várias fontes de dados heterogéneas	Regras de associação	A combinação de fontes melhora a relevância das recomendações
Gino et al. (2023)	Cestos modelados como redes temporais	Visualização de redes	Grupos de produtos relacionados; relação entre rendimento e compras
Dogan (2024)	Comércio eletrónico com atributos dos produtos	Regras de associação e testes de significância	Regras condicionadas a atributos dos produtos
Este trabalho	**Groceries (9 835 transações) e Online Retail II (20 573 faturas)**	**Apriori e FP-Growth (mlxtend); sistema com API, aplicação móvel e backoffice**	**Equivalência verificada; tempo e memória medidos; recomendações integradas numa aplicação**

4. Software e ferramentas utilizados

Neste capítulo são abordadas as ferramentas e o software utilizados no desenvolvimento deste projeto.

As secções 4.1 a 4.8 descrevem as ferramentas utilizadas na componente experimental (Capítulo 5). As secções 4.9 a 4.21 descrevem as tecnologias do sistema desenvolvido em Projeto II: a API que calcula as recomendações, a base de dados, a aplicação de *backoffice*, a aplicação móvel e os testes automáticos.

4.1 Python

Python é uma linguagem de programação interpretada, de alto nível e de tipagem dinâmica, muito utilizada em análise de dados e aprendizagem computacional pela disponibilidade de bibliotecas científicas maduras (Python Software Foundation, s.d.).

Neste projeto, a linguagem Python foi utilizada nas experiências do Capítulo 5, na API e na aplicação de *backoffice*, por dispor de bibliotecas que facilitam a preparação e a mineração de dados e o desenvolvimento de serviços web.

4.2 Git e GitHub

O Git é um sistema de controlo de versões distribuído, que regista as alterações efetuadas ao código e permite consultá-las e revertê-las. O GitHub é uma plataforma de alojamento de repositórios Git, que facilita a colaboração e a partilha do código (GitHub, s.d.).

Neste projeto, o código foi versionado com Git e alojado num repositório GitHub (https://github.com/JoaoJTome/Projeto), o que permitiu rever o desenvolvimento em qualquer momento e reverter alterações quando necessário. O repositório contém os *scripts* utilizados nas experiências do Capítulo 5, incluindo os de verificação da equivalência e de medição do tempo e da memória, o código da API, da aplicação de *backoffice* e da aplicação móvel, e as ligações para os conjuntos de dados utilizados.

4.3 Mendeley

Mendeley é um gestor de referências gratuito que recolhe referências, organiza citações e cria bibliografias (Mendeley, s.d.).

Neste projeto, o Mendeley foi utilizado para gerir as referências e gerar as citações e a bibliografia.

4.4 pandas

O pandas é uma biblioteca de código aberto para a linguagem Python que disponibiliza estruturas de dados tabulares (*DataFrame*) e funções de leitura, limpeza, transformação e agregação de dados (Pandas, s.d.).

Neste projeto, o pandas é utilizado para carregar, limpar e estruturar os dados transacionais em *DataFrames*, constituindo a base para o pré-processamento dos dados antes da aplicação dos algoritmos de Análise de Cesto de Compras.

4.5 NumPy

O NumPy é um projeto de código aberto que tem como objetivo suportar a computação numérica em Python. Foi criado em 2005, tendo como base as bibliotecas Numeric e Numarray. O NumPy é distribuído como software de código aberto e gratuito (NumPy, s.d.).

Neste projeto, o NumPy é utilizado para suportar as operações vetoriais e matriciais subjacentes à representação binária dos itens, essencial ao funcionamento dos algoritmos de mineração de regras de associação.

4.6 Bibliotecas dos algoritmos

Neste projeto, a principal biblioteca utilizada para a extração de conjuntos de itens frequentes e geração de regras de associação é a **mlxtend** (*Machine Learning Extensions*). Esta biblioteca disponibiliza implementações dos algoritmos Apriori e FP-Growth com a mesma interface, bem como funções auxiliares para o cálculo de métricas de avaliação das regras de associação, como suporte, confiança e elevação (*lift*) (Raschka, 2018).

A biblioteca mlxtend integra-se diretamente com estruturas de dados do **NumPy** e do pandas, permitindo que os dados transacionais sejam facilmente transformados em matrizes booleanas e manipulados como *DataFrames*. Esta compatibilidade facilita o pré-processamento dos dados e a análise posterior dos resultados, reduzindo a complexidade do código e aumentando a legibilidade do processo de mineração (Harris et al., 2020; McKinney, 2010).

Para a transformação de listas de transações em formato adequado aos algoritmos de mineração, é utilizada a classe **TransactionEncoder**, disponibilizada pela própria biblioteca mlxtend. Esta classe converte listas de itens em matrizes binárias, onde cada coluna representa um item e cada linha corresponde a uma transação, indicando a presença ou ausência de cada item. Esta representação é necessária para a aplicação direta das funções apriori() e fpgrowth() (Raschka, 2018).

Após a obtenção dos conjuntos de itens frequentes, a função **association_rules()** permite gerar automaticamente as regras de associação e calcular as métricas relevantes, como suporte, confiança e elevação. Esta funcionalidade simplifica

significativamente o processo de análise, permitindo filtrar regras de acordo com critérios estatísticos definidos e ordenar os resultados por métricas de interesse, como a elevação ou a confiança.

Foi considerada também a plataforma Orange, que oferece um ambiente visual para mineração de dados e aprendizagem computacional, incluindo módulos para regras de associação (Demšar et al., 2013). A escolha recaiu sobre a mlxtend por três critérios concretos deste projeto: o motor de recomendação tinha de ser integrado num serviço web escrito em Python (a API), o pré-processamento dos dados tinha de ser controlado em código e as experiências do Capítulo 5 tinham de ser executadas de forma automatizada e repetível, com medição de tempo e de memória. O Orange está orientado para a construção interativa de fluxos de análise, pelo que satisfazer estes critérios exigiria passos adicionais; não se trata de uma limitação demonstrada da ferramenta, mas de uma preferência de engenharia face aos requisitos do projeto.

Assim, a escolha da biblioteca mlxtend justifica-se por disponibilizar implementações de ambos os algoritmos, apriori() e fpgrowth(), com a mesma interface e o mesmo formato de entrada, o que permite aplicá-los nas mesmas condições, e pela sua integração com o pandas. A reprodutibilidade dos resultados não decorre da biblioteca em si: depende de documentar o ambiente de execução, as versões utilizadas, os ficheiros de dados e os parâmetros de cada experiência; as versões são indicadas na Tabela 2 e os parâmetros na secção 5.1.

4.7 Matplotlib

O Matplotlib é uma biblioteca de visualização de dados para Python, que permite produzir gráficos estáticos de vários tipos (Hunter, 2007).

Neste projeto, o Matplotlib foi utilizado para produzir os gráficos de suporte, confiança e elevação apresentados no Capítulo 5 e, através do Seaborn, o gráfico do *backoffice*.

4.8 openpyxl

O openpyxl é uma biblioteca para Python que permite ler e escrever ficheiros Excel no formato xlsx (openpyxl, s.d.).

Neste projeto, o openpyxl foi utilizado, através do pandas, para ler o ficheiro do Dataset 2.

A Tabela 2 indica as versões das ferramentas utilizadas nas experiências do Capítulo 5.

Tabela 2 – Versões das ferramentas utilizadas nas experiências

Ferramenta	Versão	Utilização
Python	3.9.5	Linguagem de programação das experiências
pandas	2.2.3	Leitura, limpeza e agregação dos dados
NumPy	1.24.4	Operações vetoriais sobre a matriz transação × item
mlxtend	0.23.4	Algoritmos Apriori e FP-Growth e geração das regras

Ferramenta	Versão	Utilização
Matplotlib	3.5.3	Gráficos de suporte, confiança e elevação
openpyxl	3.1.5	Leitura do ficheiro Excel do Dataset 2

As versões das bibliotecas utilizadas pelo sistema desenvolvido em Projeto II são indicadas na Tabela 28 (secção 6.6).

4.9 Streamlit

O Streamlit é uma biblioteca de código aberto para Python que permite construir aplicações web interativas orientadas a dados, sem necessidade de conhecimentos de desenvolvimento *front-end* (HTML, CSS ou JavaScript). Cada elemento de interface — botões, menus, campos de seleção ou *sliders* — é criado através de uma chamada de função Python (Streamlit Inc., s.d.).

Neste projeto, o Streamlit foi utilizado para desenvolver a aplicação de *backoffice*, destinada ao administrador. Esta aplicação não acede diretamente ao motor de recomendação: consome a API através de pedidos HTTP para apresentar estatísticas sobre as regras de associação e os itens mais frequentes e para alterar os parâmetros do motor (suporte mínimo, confiança mínima e peso das escolhas do utilizador).

4.10 Seaborn

O Seaborn é uma biblioteca de visualização estatística para Python, construída sobre o Matplotlib, que disponibiliza funções de alto nível para desenhar gráficos estatísticos diretamente a partir de *DataFrames* do pandas (Waskom, 2021).

Neste projeto, o Seaborn é utilizado na aplicação de backoffice para desenhar o gráfico de dispersão entre o suporte e a confiança das regras de associação, através da função scatterplot().

4.11 FastAPI

O FastAPI é uma *framework* web para Python destinada à construção de APIs, baseada nas anotações de tipos da linguagem. A partir dessas anotações, valida os pedidos, injeta dependências e gera automaticamente a documentação interativa da API segundo a especificação OpenAPI (Ramírez, s.d.).

Neste projeto, o FastAPI é utilizado para implementar a API *backend*, que passou a ser o único ponto do sistema onde são calculadas as regras de associação e as recomendações. A API disponibiliza os serviços de autenticação, listas de compras e recomendações usados pela aplicação móvel, e os serviços de estatísticas e de parâmetros usados pela aplicação de backoffice.

4.12 Uvicorn

O Uvicorn é um servidor web para Python que implementa a especificação ASGI (*Asynchronous Server Gateway Interface*), a interface utilizada por *frameworks* assíncronas como o FastAPI (Encode, s.d.).

Neste projeto, o Uvicorn é o servidor que executa a API durante o desenvolvimento e a demonstração, através do comando uvicorn app.main:app.

4.13 Pydantic

O Pydantic é uma biblioteca de validação de dados para Python, na qual a estrutura dos dados é definida através de classes com anotações de tipos (Pydantic, s.d.).

Neste projeto, o Pydantic é utilizado para definir e validar o formato dos dados trocados com a API, como o email e a palavra-passe no registo, a lista de itens a guardar ou os limites dos parâmetros do motor de recomendação. Os pedidos com dados inválidos são rejeitados antes de chegarem à lógica da aplicação.

4.14 SQLAlchemy

O SQLAlchemy é um conjunto de ferramentas SQL e um mapeador objeto-relacional (ORM) para Python, que permite representar as tabelas de uma base de dados como classes e manipular os seus registos como objetos (SQLAlchemy, s.d.).

Neste projeto, o SQLAlchemy é utilizado para definir e aceder às tabelas de utilizadores, sessões, listas de compras e parâmetros do motor de recomendação. O ORM reduz a dependência do código em relação ao motor de base de dados, mas uma migração para outro sistema não se resume a alterar o endereço de ligação: diferenças de dialeto SQL, de tipos de dados, de restrições e de concorrência, bem como a ausência de um sistema de migrações do esquema, exigiriam adaptações e testes. Essa migração não foi testada neste trabalho.

4.15 SQLite

O SQLite é um motor de base de dados relacional embebido, que guarda toda a base de dados num único ficheiro e não necessita de um servidor separado (Gaffney et al., 2022; SQLite, s.d.).

Neste projeto, o SQLite é utilizado como base de dados da API, onde ficam guardadas as contas de utilizador, as sessões, o histórico de listas de compras e os parâmetros do motor de recomendação. Esta opção é suficiente para a fase de desenvolvimento e demonstração, podendo ser substituída por um sistema de gestão de bases de dados mais robusto caso a aplicação venha a ser escalada.

4.16 bcrypt

O bcrypt é uma função de *hash* para palavras-passe, concebida para ser computacionalmente custosa e para incluir um *salt* aleatório em cada *hash*, o que dificulta ataques por força bruta (Provos & Mazières, 1999).

Neste projeto, a biblioteca bcrypt para Python é utilizada para gerar os hashes das palavras-passe dos utilizadores da aplicação móvel, que nunca são armazenadas em texto simples, e para as verificar no momento da autenticação.

4.17 Requests

O Requests é uma biblioteca para Python que simplifica o envio de pedidos HTTP (Requests, s.d.).

Neste projeto, o Requests é utilizado pela aplicação de backoffice para consultar a API, tanto para obter as estatísticas como para ler e atualizar os parâmetros do motor de recomendação.

4.18 Flutter

O Flutter é uma *framework* de código aberto, desenvolvida pela Google, para construir aplicações com interface gráfica para várias plataformas, como Android, iOS e web, a partir de um único código-fonte (Flutter, s.d.).

Neste projeto, o Flutter é utilizado para desenvolver a aplicação móvel destinada ao consumidor final, que permite criar conta, autenticar-se, construir e guardar listas de compras e receber recomendações. A aplicação está configurada para ser executada no emulador Android.

4.19 Dart

O Dart é uma linguagem de programação orientada a objetos, desenvolvida pela Google, otimizada para o desenvolvimento de interfaces de utilizador e com suporte nativo a programação assíncrona (Dart, s.d.).

Neste projeto, o Dart é a linguagem em que é escrita a aplicação móvel, uma vez que é a linguagem utilizada pelo Flutter.

4.20 Bibliotecas da aplicação móvel

Para além do Flutter, a aplicação móvel utiliza duas bibliotecas disponibilizadas no repositório oficial de pacotes do Dart, o pub.dev.

A biblioteca **http** permite enviar pedidos HTTP a partir de código Dart e é utilizada em todas as comunicações da aplicação com a API (Dart Team, s.d.).

A biblioteca shared_preferences permite guardar pequenos valores de forma persistente no dispositivo e é utilizada para guardar o *token* de sessão do utilizador, para que a sessão se mantenha entre utilizações da aplicação (Flutter Team, s.d.). A própria documentação do pacote indica que não deve ser usado para dados críticos, uma vez que os valores não são cifrados. Num cenário de produção, o *token* deveria ser guardado num armazenamento seguro do sistema operativo (como o Keystore, no Android), em conjunto com a expiração das sessões e a comunicação por HTTPS, indicadas como trabalho futuro no Capítulo 7.

4.21 pytest

O pytest é uma *framework* de testes para Python que permite escrever testes automáticos sob a forma de funções simples (pytest-dev, s.d.).

Neste projeto, o pytest é utilizado para testar automaticamente a API, em conjunto com o cliente de testes do FastAPI, que permite enviar pedidos à API sem arrancar o servidor (Ramírez, s.d.). Os testes cobrem o registo, a autenticação e o fim de sessão, a gravação de listas, as recomendações e os serviços do backoffice, recorrendo a uma base de dados temporária e a um conjunto de dados de teste.

5. Metodologia experimental e resultados

Neste capítulo descreve-se a metodologia experimental, caracterizam-se os conjuntos de dados escolhidos e o respetivo tratamento, apresentam-se os resultados obtidos com os algoritmos utilizados e discutem-se as diferenças entre os dois casos.

5.1 Metodologia experimental

Os dois algoritmos foram aplicados com a biblioteca mlxtend (secção 4.6). As transações de cada conjunto de dados foram convertidas numa matriz binária transação × item com a classe TransactionEncoder, em que cada item conta uma única vez por transação. Os conjuntos de itens frequentes foram obtidos com as funções apriori() e fpgrowth(), com os mesmos parâmetros, e as regras com a função association_rules(), filtradas pela confiança mínima.

Para o Dataset 1 utilizaram-se um suporte mínimo de 0,005 e uma confiança mínima de 0,25; para o Dataset 2, de maior dimensão, um suporte mínimo de 0,02 e uma confiança mínima de 0,40. Em ambos os casos, os conjuntos de itens foram limitados a dois itens (max_len = 2), pelo que todas as regras têm um único item no antecedente e no consequente. Os limiares foram escolhidos de forma a obter um número de regras que pudesse ser analisado manualmente. No Dataset 1, um suporte de 0,005 (cerca de 49 transações) e uma confiança de 0,25 produzem 161 regras; com suportes de 0,01 e 0,02, o número desce para 96 e 43 (Tabela 7). No Dataset 2, em que os cestos são maiores e as associações entre produtos semelhantes são mais fortes, um suporte de 0,02 (cerca de 411 faturas) e uma confiança de 0,40 produzem 31 regras, enquanto um suporte de 0,01 já produz 263 (Tabela 12). A restrição max_len = 2 limita as regras a pares de produtos, que são mais fáceis de interpretar e correspondem à recomendação de um produto a partir de outro. Para cada conjunto apresentam-se as dez regras com maior confiança e as dez com maior elevação; os empates na confiança são desempatados pela elevação. O suporte e a confiança são apresentados em proporção (entre 0 e 1), com quatro casas decimais, e a elevação com duas.

Para comparar os algoritmos, verificou-se primeiro a equivalência dos resultados: em cada configuração, compararam-se os conjuntos de itens frequentes e as regras obtidos com apriori() e fpgrowth(), incluindo os respetivos valores de suporte, confiança e elevação. Mediu-se depois o tempo de extração dos conjuntos de itens frequentes, a fase em que os dois algoritmos diferem, para vários valores de suporte mínimo: no Dataset 1, com e sem a restrição max_len = 2, tomando a mediana de cinco execuções; no Dataset 2, apenas com max_len = 2 e com a mediana de três execuções, por ser mais demorado. No Dataset 2 mediu-se também o pico de memória, com o módulo tracemalloc do Python, e incluiu-se a variante do Apriori com

low_memory=True, que avalia os candidatos um a um. As medições foram efetuadas num computador com processador AMD Ryzen 7 3700X (8 núcleos, 3,60 GHz) e 16 GB de RAM, com Windows 10, Python 3.9.5, pandas 2.2.3, NumPy 1.24.4 e mlxtend 0.23.4.

5.2 Dataset 1: Groceries

O primeiro conjunto de dados utilizado neste projeto corresponde ao *Groceries Dataset*, um conjunto de dados públicos que reúne um mês de registos reais de transações de ponto de venda de uma mercearia local (Hahsler, Hornik & Reutterer, 2006). O conjunto de dados é disponibilizado através do pacote arules da linguagem R, da autoria de Michael Hahsler, Kurt Hornik e Thomas Reutterer, e é amplamente utilizado na literatura como referência para experiências de Análise de Cesto de Compras. O ficheiro utilizado neste trabalho (dataSort.csv) corresponde ao conjunto Groceries do pacote arules (Hahsler, Hornik & Reutterer, s.d.), exportado para CSV, com cada linha a representar uma transação e as respetivas colunas os produtos adquiridos; a ligação para o conjunto de dados está indicada no repositório do projeto (secção 4.2). A comparação com a documentação oficial do pacote arules, que indica 9 835 transações e 169 categorias de produtos (Hahsler, Hornik & Reutterer, s.d.), mostrou que a exportação obtida continha o conjunto de transações em duplicado (19 670 linhas, com todas as contagens absolutas a dobrar). O ficheiro foi, por isso, reduzido às 9 835 transações do conjunto de dados original, que são as utilizadas neste capítulo e pelo motor de recomendação. Os cestos repetidos dentro do conjunto de dados original não foram eliminados, porque podem corresponder a compras legítimas.

O primeiro conjunto de dados contém 9 835 transações e 169 itens distintos, num total de 43 367 ocorrências de itens (Tabela 3). Cada transação representa um talão de compra, com a lista de produtos adquiridos numa única visita. Os 169 itens correspondem a categorias agregadas de produtos (por exemplo, whole milk ou tropical fruit) e não a produtos individuais (SKU, *Stock Keeping Unit*); as recomendações obtidas com este conjunto são, por isso, recomendações de categorias, o que limita a sua especificidade.

Tabela 3 – Estatísticas do Dataset 1

Métrica	**Valor**
Número de transações	9 835
Número de itens únicos	169
Total de ocorrências de itens	43 367
Média de itens por transação	4,41

Este conjunto de dados caracteriza-se por (Tabela 4):

- elevada esparsidade, uma vez que cada transação contém apenas uma pequena fração do total de itens possíveis;

- presença de produtos muito frequentes, como leite, pão e outros bens de primeira necessidade;
- forte concentração de compras em poucos itens dominantes.

Após o pré-processamento, ambos os algoritmos foram aplicados com os parâmetros indicados na secção 5.1 (suporte mínimo de 0,005, confiança mínima de 0,25 e conjuntos de, no máximo, 2 itens), obtendo-se 725 conjuntos de itens frequentes e 161 regras.

Tabela 4 – Produtos mais frequentes (Dataset 1)

Produto	**Frequência**	**Percentagem (%)**
whole milk	2 513	25,55
other vegetables	1 903	19,35
rolls/buns	1 809	18,39
soda	1 715	17,44
yogurt	1 372	13,95
bottled water	1 087	11,05
root vegetables	1 072	10,90
tropical fruit	1 032	10,49
shopping bags	969	9,85
sausage	924	9,40
pastry	875	8,90
citrus fruit	814	8,28
bottled beer	792	8,05
newspapers	785	7,98
canned beer	764	7,77
pip fruit	744	7,56
fruit/vegetable juice	711	7,23
whipped/sour cream	705	7,17
brown bread	638	6,49
domestic eggs	624	6,34
frankfurter	580	5,90
margarine	576	5,86
coffee	571	5,81
pork	567	5,77

Produto	Frequência	Percentagem (%)
butter	545	5,54
curd	524	5,33
beef	516	5,25
napkins	515	5,24
chocolate	488	4,96
frozen vegetables	473	4,81

Na Tabela 4, a frequência corresponde ao número de transações que contêm o item (cada item conta uma vez por cesto) e a percentagem é calculada sobre as 9 835 transações. As Tabelas 5 e 6 apresentam as dez regras com maior confiança e as dez com maior elevação, respetivamente, entre as 161 regras obtidas; em caso de empate na confiança, as regras são ordenadas pela elevação. O suporte e a confiança são apresentados em proporção (entre 0 e 1).

Tabela 5 – Regras com maior confiança (Dataset 1)

Antecedente	Consequente	Suporte	Confiança	Elevação
baking powder	whole milk	0,0093	0,5230	2,05
butter	whole milk	0,0276	0,4972	1,95
curd	whole milk	0,0261	0,4905	1,92
flour	whole milk	0,0084	0,4854	1,90
herbs	other vegetables	0,0077	0,4750	2,45
herbs	whole milk	0,0077	0,4750	1,86
domestic eggs	whole milk	0,0300	0,4728	1,85
detergent	whole milk	0,0089	0,4656	1,82
onions	other vegetables	0,0142	0,4590	2,37
whipped/sour cream	whole milk	0,0322	0,4496	1,76

Tabela 6 – Regras com maior elevação (Dataset 1)

Antecedente	Consequente	Suporte	Confiança	Elevação
herbs	root vegetables	0,0070	0,4313	3,96
berries	whipped/sour cream	0,0090	0,2722	3,80
sliced cheese	sausage	0,0070	0,2863	3,05
beef	root vegetables	0,0174	0,3314	3,04
onions	root vegetables	0,0095	0,3049	2,80
grapes	tropical fruit	0,0061	0,2727	2,60
pip fruit	tropical fruit	0,0204	0,2702	2,57
soft cheese	yogurt	0,0060	0,3512	2,52
herbs	other vegetables	0,0077	0,4750	2,45
onions	other vegetables	0,0142	0,4590	2,37

A análise das regras de associação revela que as regras com maior confiança (Tabela 5) envolvem predominantemente produtos de consumo básico, em especial *whole milk* e other vegetables, refletindo hábitos de compra recorrentes.

Por outro lado, as regras com maior elevação (Tabela 6) identificam associações mais específicas e informativas, como a compra conjunta de *herbs* e *root vegetables* ou a associação entre diferentes tipos de fruta, indicando padrões de complementaridade entre produtos.

Em todas as configurações testadas (Tabela 7), o Apriori e o FP-Growth produziram exatamente os mesmos conjuntos de itens frequentes e as mesmas regras, com os mesmos valores de suporte, confiança e elevação, o que confirma que ambos são métodos exatos.

Tabela 7 – Equivalência dos resultados e tempo de extração dos conjuntos frequentes (Dataset 1)

Suporte mínimo	**max_len**	**Conjuntos frequentes**	**Regras (conf. ≥ 0,25)**	**Resultados iguais**	**Apriori (s)**	**FP-Growth (s)**
0,02	2	120	43	Sim	0,065	1,18
0,01	2	301	96	Sim	0,143	3,96
0,005	2	725	161	Sim	0,257	11,14
0,02	sem limite	122	49	Sim	0,090	1,30
0,01	sem limite	333	170	Sim	0,338	5,12
0,005	sem limite	1 001	673	Sim	1,009	17,26

Quanto ao tempo de execução, o resultado contraria a expectativa da literatura (Han, Pei & Yin, 2000): o Apriori foi entre cerca de 15 e 45 vezes mais rápido do que o FP-Growth, sendo a diferença maior com a restrição max_len = 2. A diferença resulta das implementações e não dos algoritmos em si. Na biblioteca mlxtend, a função apriori() avalia os candidatos com operações vetorizadas da NumPy, enquanto fpgrowth() constrói e percorre a FP-tree em Python puro. Além disso, o Dataset 1 tem apenas 169 itens: com um suporte mínimo de 0,005 há 120 itens frequentes e, portanto, no máximo 7 140 pares candidatos, que o Apriori avalia rapidamente. A vantagem teórica do FP-Growth manifesta-se sobretudo em bases de dados densas ou com muitos itens frequentes, onde o número de candidatos cresce de forma combinatória. Para este conjunto de dados e com esta biblioteca, o Apriori é, portanto, a opção mais eficiente.

As regras extraídas refletem maioritariamente associações entre produtos básicos, o que justifica o recurso a métricas adicionais, como a elevação, para filtrar associações pouco informativas causadas pela elevada popularidade de determinados produtos.

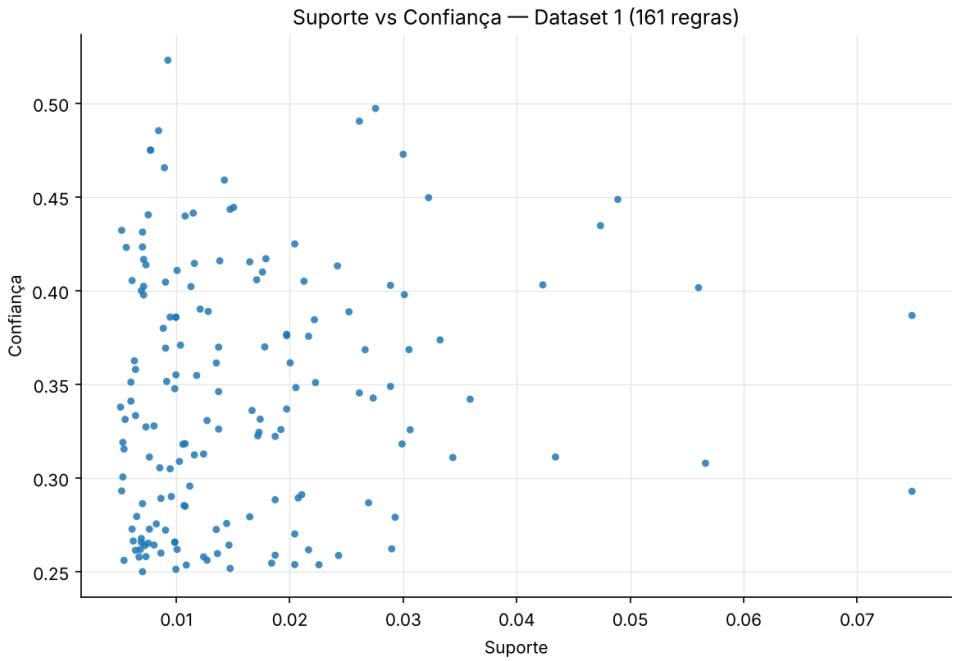


Figura 6 – Suporte versus confiança das 161 regras (Dataset 1; resultados idênticos com Apriori e FP-Growth)

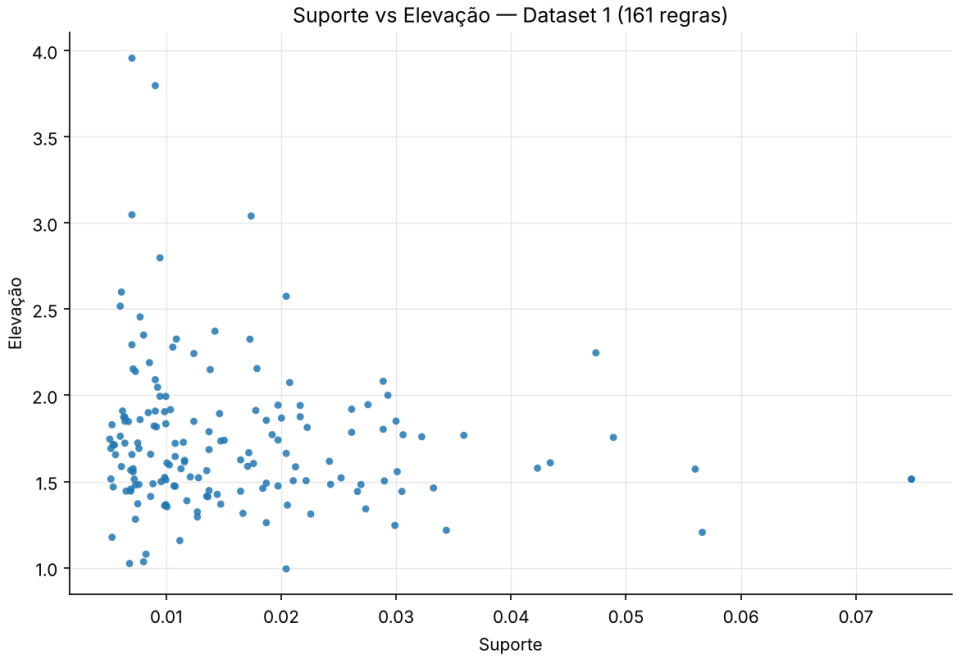


Figura 7 – Suporte versus elevação das 161 regras (Dataset 1; resultados idênticos com Apriori e FP-Growth)

A análise gráfica da relação entre suporte e confiança (Figura 6) evidencia que a maioria das regras apresenta valores de suporte reduzidos, concentrando-se abaixo de 0,02, o que é consistente com a elevada esparsidade do conjunto de dados. Observa-se, no entanto, um conjunto de regras com valores de confiança moderados a elevados, que indicam associações fortes nesta amostra; a sua estabilidade ao longo do tempo ou noutras amostras não foi avaliada.

No gráfico de suporte versus elevação (Figura 7), verifica-se que 160 das 161 regras têm elevação superior a 1, o que indica dependência positiva entre os itens envolvidos. Este resultado decorre em parte do próprio limiar de confiança: como a elevação é a confiança dividida pelo suporte do consequente, uma regra com confiança de pelo menos 0,25 só pode ter elevação inferior a 1 se o consequente estiver presente em mais de 25% das transações, o que apenas acontece com *whole milk* (25,55%). Para os restantes consequentes, a elevação é, por construção, de pelo menos 1,29 (0,25/0,1935, sendo 0,1935 o suporte de *other vegetables*, o segundo produto mais frequente). As únicas regras com elevação abaixo deste valor têm, de facto, *whole milk* como consequente, e a única com elevação inferior a 1 é *bottled beer* ⇒ *whole milk* (0,99). As regras com maior elevação tendem a apresentar suporte mais reduzido, sugerindo associações fortes, mas pouco frequentes, enquanto regras com suporte mais elevado tendem a apresentar valores de elevação mais próximos de 1, correspondendo a padrões mais genéricos.

5.3 Dataset 2: Online Retail II

O segundo conjunto de dados corresponde ao *Online Retail II*, disponibilizado por Daqing Chen, da School of Engineering da London South Bank University, através do UCI Machine Learning Repository (Chen, 2012). Foi utilizada a folha «Year 2009-2010» do ficheiro, que reúne as transações ocorridas entre 1 de dezembro de 2009 e 9 de dezembro de 2010 numa loja de comércio eletrónico registada no Reino Unido, especializada na venda de artigos de oferta e decoração, sendo grande parte dos seus clientes revendedores por grosso (Chen, Sain & Guo, 2012). Ao contrário do primeiro conjunto de dados, cada linha do ficheiro original corresponde a uma linha de fatura, pelo que foi necessário agregar as linhas por número de fatura para reconstituir cada cesto de compras como uma transação única, adequada à aplicação dos algoritmos Apriori e FP-Growth.

O tratamento aplicado foi o seguinte: das 525 461 linhas originais (28 816 faturas), removeram-se as 2 928 linhas sem descrição de produto, as linhas das faturas de cancelamento, identificadas por um número de fatura iniciado pela letra «C» (10 206 linhas), as linhas com quantidade nula ou negativa (294), as linhas com preço nulo ou negativo (468) e as linhas com códigos que não correspondem a produtos (2 305), como portes de envio (POSTAGE e DOTCOM POSTAGE), registos manuais (Manual), ajustes contabilísticos, taxas bancárias e da Amazon, descontos, amostras, vales de oferta e produtos de teste. Obtiveram-se 509 260 linhas, agregadas em 20 573 faturas, com 4 477 produtos distintos, identificados pela descrição. Optou-se pela descrição, e não pelo código do produto (StockCode), por ser a forma como o produto é apresentado ao utilizador. As duas opções não são equivalentes: 426 códigos surgem com mais do que uma descrição e 199 descrições estão associadas a mais do que um código, pelo que a escolha pode separar ou juntar artigos; o seu impacto não foi avaliado neste trabalho. Em cada fatura, cada produto foi contado uma única vez, mesmo que surgisse em várias linhas.

Após a agregação, cada transação contém, em média, 24,09 produtos distintos, contra 4,41 itens no primeiro conjunto (Tabela 8). Importa, contudo, distinguir o tamanho médio do cesto da densidade relativa da matriz binária transação × item, que corresponde ao tamanho médio do cesto dividido pelo número de itens distintos: no primeiro conjunto é de 4,41/169 ≈ 2,61%, e no segundo de 24,09/4 477 ≈ 0,54%. O segundo conjunto tem, assim, cestos maiores, mas é relativamente mais esparso.

Tabela 8 – Estatísticas do Dataset 2

Métrica	**Valor**
Número de transações (faturas)	20 573
Número de itens únicos	4 477
Total de ocorrências de itens	495 661
Média de itens por transação	24,09

Os produtos mais frequentes (Tabela 9) são maioritariamente artigos decorativos e de cozinha. Uma possível interpretação para os padrões de compra conjunta observados é a afinidade estética ou funcional entre produtos; trata-se, porém, de uma hipótese interpretativa e não de um comportamento comprovado, tanto mais que muitos dos clientes deste retalhista são grossistas (Chen, Sain & Guo, 2012). Na Tabela 9, a frequência corresponde ao número de faturas que contêm o produto e a percentagem é calculada sobre as 20 573 faturas.

Tabela 9 – Produtos mais frequentes (Dataset 2)

Produto	**Frequência**	**Percentagem (%)**
WHITE HANGING HEART T-LIGHT HOLDER	3 315	16,11
REGENCY CAKESTAND 3 TIER	2 019	9,81
STRAWBERRY CERAMIC TRINKET BOX	1 640	7,97
ASSORTED COLOUR BIRD ORNAMENT	1 412	6,86
PACK OF 72 RETRO SPOT CAKE CASES	1 410	6,85
60 TEATIME FAIRY CAKE CASES	1 336	6,49
HOME BUILDING BLOCK WORD	1 331	6,47
JUMBO BAG RED RETROSPOT	1 246	6,06
LUNCH BAG RED SPOTTY	1 218	5,92
JUMBO STORAGE BAG SUKI	1 179	5,73
PACK OF 60 PINK PAISLEY CAKE CASES	1 155	5,61

Produto	Frequência	Percentagem (%)
WOODEN FRAME ANTIQUE WHITE	1 140	5,54
LUNCH BAG BLACK SKULL.	1 115	5,42
LUNCH BAG SUKI DESIGN	1 114	5,41
HEART OF WICKER LARGE	1 092	5,31
LOVE BUILDING BLOCK WORD	1 091	5,30
REX CASH+CARRY JUMBO SHOPPER	1 074	5,22
RED HANGING HEART T-LIGHT HOLDER	1 066	5,18
JUMBO BAG STRAWBERRY	1 057	5,14
JUMBO SHOPPER VINTAGE RED PAISLEY	1 057	5,14

Com os parâmetros indicados na secção 5.1, obtiveram-se 272 conjuntos de itens frequentes e 31 regras; tal como no primeiro conjunto de dados, o Apriori e o FP-Growth produziram exatamente os mesmos resultados, o que se verificou em todas as configurações da Tabela 12. As regras com maior elevação (Tabela 11) indicam associações fortes entre produtos visualmente semelhantes ou pertencentes à mesma categoria, enquanto as regras com maior confiança (Tabela 10) indicam combinações frequentes quando o antecedente ocorre. As regras recíprocas (A ⇒ B e B ⇒ A) têm a mesma elevação, pelo que surgem aos pares na Tabela 11.

Tabela 10 – Regras com maior confiança (Dataset 2)

Antecedente	Consequente	Suporte	Confiança	Elevação
SWEETHEART CERAMIC TRINKET BOX	STRAWBERRY CERAMIC TRINKET BOX	0,0387	0,7952	9,98
RED HANGING HEART T-LIGHT HOLDER	WHITE HANGING HEART T-LIGHT HOLDER	0,0375	0,7233	4,49
WOODEN PICTURE FRAME WHITE FINISH	WOODEN FRAME ANTIQUE WHITE	0,0294	0,6389	11,53
SINGLE HEART ZINC T-LIGHT HOLDER	HANGING HEART ZINC T-LIGHT HOLDER	0,0202	0,6265	13,64
VINTAGE HEADS AND TAILS CARD GAME	VINTAGE SNAP CARDS	0,0216	0,5968	12,27
PACK OF 60 DINOSAUR CAKE CASES	60 TEATIME FAIRY CAKE CASES	0,0204	0,5868	9,04

Antecedente	Consequente	Suporte	Confiança	Elevação
72 SWEETHEART FAIRY CAKE CASES	60 TEATIME FAIRY CAKE CASES	0,0229	0,5419	8,34
LOVE BUILDING BLOCK WORD	HOME BUILDING BLOCK WORD	0,0284	0,5362	8,29
HEART OF WICKER SMALL	HEART OF WICKER LARGE	0,0266	0,5337	10,05
LUNCH BAG PINK RETROSPOT	LUNCH BAG RED SPOTTY	0,0209	0,5321	8,99

Tabela 11 – Regras com maior elevação (Dataset 2)

Antecedente	Consequente	Suporte	Confiança	Elevação
SINGLE HEART ZINC T-LIGHT HOLDER	HANGING HEART ZINC T-LIGHT HOLDER	0,0202	0,6265	13,64
HANGING HEART ZINC T-LIGHT HOLDER	SINGLE HEART ZINC T-LIGHT HOLDER	0,0202	0,4402	13,64
VINTAGE HEADS AND TAILS CARD GAME	VINTAGE SNAP CARDS	0,0216	0,5968	12,27
VINTAGE SNAP CARDS	VINTAGE HEADS AND TAILS CARD GAME	0,0216	0,4436	12,27
WOODEN PICTURE FRAME WHITE FINISH	WOODEN FRAME ANTIQUE WHITE	0,0294	0,6389	11,53
WOODEN FRAME ANTIQUE WHITE	WOODEN PICTURE FRAME WHITE FINISH	0,0294	0,5307	11,53
CHOCOLATE HOT WATER BOTTLE	HOT WATER BOTTLE TEA AND SYMPATHY	0,0233	0,5162	10,45
HOT WATER BOTTLE TEA AND SYMPATHY	CHOCOLATE HOT WATER BOTTLE	0,0233	0,4715	10,45
HEART OF WICKER SMALL	HEART OF WICKER LARGE	0,0266	0,5337	10,05

Antecedente	Consequente	Suporte	Confiança	Elevação
HEART OF WICKER LARGE	HEART OF WICKER SMALL	0,0266	0,5009	10,05

As regras de associação revelam padrões fortes entre produtos da mesma categoria ou com afinidade funcional e estética, como molduras, caixas decorativas e artigos de pastelaria (Figuras 8 e 9). As regras com maior elevação indicam associações altamente específicas, enquanto as regras com maior confiança indicam combinações frequentes quando o antecedente ocorre: por exemplo, 72,3% das faturas que contêm RED HANGING HEART T-LIGHT HOLDER contêm também WHITE HANGING HEART T-LIGHT HOLDER, embora esta combinação surja em apenas 3,7% do total de faturas.

Note-se, contudo, que neste conjunto o limiar de confiança implica, por si só, valores de elevação elevados: como o produto mais frequente (WHITE HANGING HEART T-LIGHT HOLDER) está presente em 16,11% das faturas, qualquer regra com confiança de pelo menos 0,40 tem elevação de pelo menos $0{,}40/0{,}1611 \approx 2{,}5$. Os valores de elevação das Tabelas 10 e 11 resultam, assim, em parte do próprio filtro de confiança e não devem ser lidos como uma propriedade independente dos dados.

Muitas das regras mais fortes ligam variantes do mesmo artigo, que diferem na cor (RED e WHITE HANGING HEART T-LIGHT HOLDER), no tamanho (HEART OF WICKER SMALL e LARGE) ou no modelo dentro da mesma coleção (VINTAGE SNAP CARDS e VINTAGE HEADS AND TAILS CARD GAME). Estas associações refletem, provavelmente, a compra de coleções ou de sortidos por revendedores, mais do que a complementaridade entre produtos. Numa aplicação de listas de compras, sugerir uma variante de um produto já escolhido teria pouco valor, pelo que este tipo de regras deverá ser filtrado ou tratado de forma diferente.

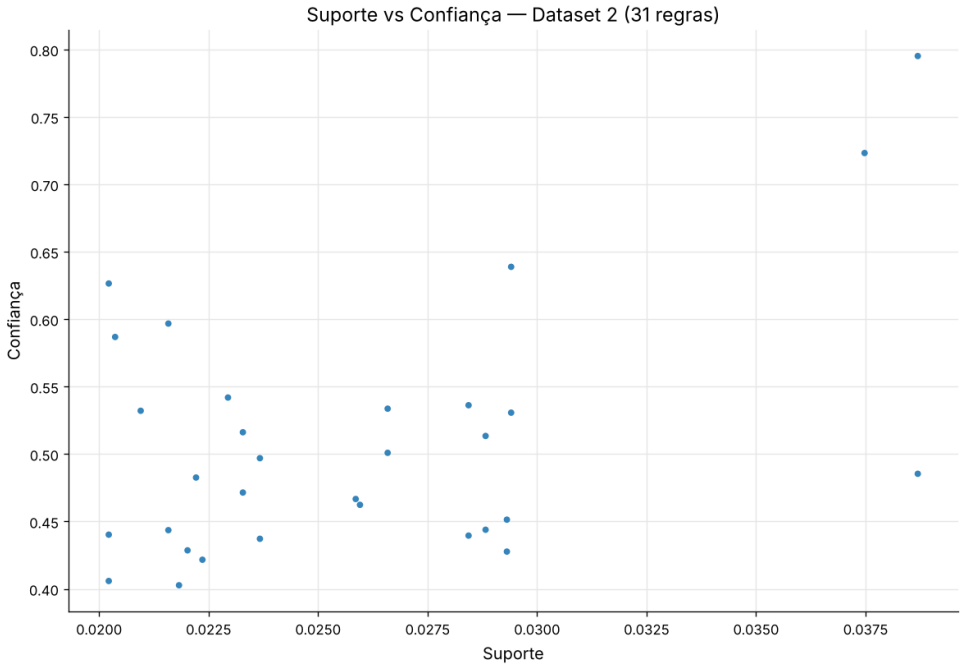


Figura 8 – Suporte versus confiança das 31 regras (Dataset 2)

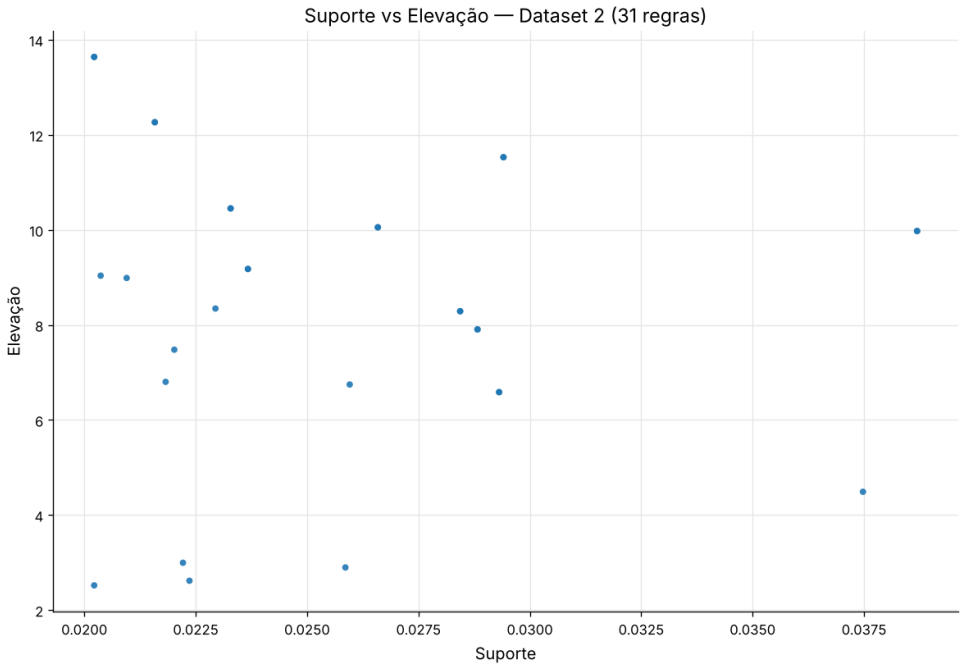


Figura 9 – Suporte versus elevação das 31 regras (Dataset 2)

A Tabela 12 resume a verificação da equivalência e as medições de tempo e de memória no Dataset 2, para vários valores de suporte mínimo e com max_len = 2.

Tabela 12 – Equivalência dos resultados, tempo e memória na extração dos conjuntos frequentes (Dataset 2, max_len = 2)

Suporte mín.	Itens freq.	Pares cand.	Conj. freq.	Regras (conf. ≥ 0,40)	Iguais	Apriori	Apriori (low_memory)	FP-Growth
0,03	94	4 371	96	3	Sim	0,98 s 258 MB	0,23 s 14 MB	2,22 s 2 196 MB
0,02	245	29 890	272	31	Sim	5,71 s 1 760 MB	0,34 s 14 MB	3,80 s 2 196 MB
0,015	423	89 253	529	91	Sim	16,71 s 5 254 MB	0,76 s 14 MB	8,47 s 2 196 MB
0,01	725	262 450	1 131	263	Sim	não executado (≈ 10,1 GB)	0,89 s 15 MB	22,10 s 2 196 MB

Ao contrário do que sucedeu no Dataset 1, a implementação por omissão do Apriori foi aqui mais lenta do que o FP-Growth a partir de um suporte mínimo de 0,02, e o seu consumo de memória cresceu rapidamente: 258 MB com suporte de 0,03, 1,7 GB com 0,02 e 5,1 GB com 0,015; com 0,01 seriam necessários cerca de 10,1 GB, pelo que não foi executada. Este comportamento resulta do número de pares candidatos, que cresce de forma quadrática com o número de itens frequentes (de 94 para 725) e que a função apriori() avalia todos de uma vez, numa matriz em memória. É precisamente esta explosão combinatória de candidatos que o FP-Growth evita (Han, Pei & Yin, 2000): o seu tempo cresceu de forma mais moderada (de 2,2 s para 22,1 s) e o pico de memória manteve-se em cerca de 2,1 GB em todas as configurações, um valor dominado pelas cópias temporárias da matriz transação × item que a mlxtend cria na preparação dos dados, e não pela FP-tree.

A variante do Apriori com low_memory=True, que avalia os candidatos um a um em vez de os manter todos em memória, foi a mais rápida em todas as configurações (menos de 1 s) e usou apenas cerca de 15 MB. Este resultado mostra que, com max_len = 2, o desempenho depende mais da forma como a biblioteca gere a memória do que do algoritmo escolhido; a vantagem teórica do FP-Growth deverá acentuar-se com conjuntos de itens maiores, o que não foi avaliado neste trabalho.

5.4 Discussão comparativa

O primeiro conjunto de dados representa um contexto de supermercado, com cestos de compras compostos por produtos de consumo diário, permitindo a identificação de padrões frequentes entre bens essenciais, como leite, ovos e produtos hortícolas. As regras extraídas são de fácil interpretação e adequadas para recomendações genéricas.

O segundo conjunto de dados, após agregação por número de fatura, representa um cenário de comércio eletrónico com mais itens por cesto, em média, e maior

diversidade de produtos, sobretudo artigos decorativos e de cozinha. As regras obtidas são mais específicas e apresentam valores de elevação elevados, em parte por efeito do limiar de confiança (secção 5.3), indicando associações fortes entre produtos da mesma categoria ou com afinidade estética. O Dataset 2 não corresponde ao contexto da aplicação desenvolvida, que se destina a listas de compras de supermercado. Foi utilizado para contrastar com o Dataset 1, por ter um catálogo maior, cestos de maior dimensão e clientes grossistas, pelo que os seus resultados servem sobretudo para avaliar o comportamento dos algoritmos e o impacto da estrutura dos dados, e não para alimentar diretamente o motor de recomendação.

Em ambos os casos, o Apriori e o FP-Growth produziram exatamente os mesmos resultados. O desempenho, pelo contrário, dependeu do conjunto de dados: com 169 itens (Dataset 1), o Apriori foi entre 15 e 45 vezes mais rápido; com 4 477 itens (Dataset 2), a sua implementação por omissão tornou-se mais lenta do que o FP-Growth a partir de um suporte mínimo de 0,02 e exigiu vários gigabytes de memória, enquanto a variante com low_memory=True foi a mais rápida em todas as configurações. Estes resultados indicam que o Apriori é adequado para o motor de recomendação do sistema (secção 5.5), desde que se use a variante low_memory=True quando o catálogo tiver muitos itens.

Em síntese, nos casos analisados, o primeiro conjunto de dados produziu sobretudo regras entre produtos de consumo geral, enquanto o segundo produziu associações mais específicas entre produtos da mesma categoria ou com afinidade estética. Como os dois conjuntos diferem simultaneamente no domínio, na época, no catálogo e nos parâmetros utilizados, esta comparação é descritiva: não permite isolar o efeito de nenhum destes fatores nem generalizar para o retalho em geral.

5.5 Implicações para o motor de recomendação

O motor de recomendação do sistema (Capítulo 6) utiliza o Dataset 1, lido a partir do ficheiro dataSort.csv com as 9 835 transações, e a implementação por omissão do Apriori da biblioteca mlxtend. Esta escolha é coerente com os resultados da secção 5.2: com um catálogo de 169 itens, o Apriori foi a opção mais rápida e produz exatamente as mesmas regras do que o FP-Growth. O Dataset 2 não é utilizado pelo motor, pelo que a filtragem das regras entre variantes do mesmo artigo, discutida na secção 5.3, não é necessária nesta versão.

Ao contrário das experiências deste capítulo, o motor não limita o tamanho dos conjuntos de itens e usa, por omissão, um suporte mínimo de 0,01 e uma confiança mínima de 0,2, valores que o administrador pode alterar no *backoffice*; às transações do Dataset 1 juntam-se ainda as listas guardadas pelo utilizador, repetidas de acordo com o peso definido (secção 6.1.3). Só com o Dataset 1, estes parâmetros produzem 333 conjuntos de itens frequentes (88 itens, 213 pares e 32 triplos) e 234 regras, das quais 80 têm dois itens no antecedente, o que permite ter em conta combinações de produtos já presentes na lista. Segundo a Tabela 7, a extração dos conjuntos frequentes

com suporte de 0,01 e sem limite de tamanho demorou cerca de 0,34 s, um tempo compatível com o cálculo das regras a cada pedido, embora o seu pré-cálculo periódico seja uma melhoria prevista (Capítulo 7).

As recomendações são ordenadas pela elevação, para evitar sugerir apenas os produtos mais populares, como whole milk, que têm confiança elevada com quase qualquer antecedente, mas elevação próxima de 1 (secção 5.2). Três das 234 regras têm elevação inferior a 1 e ficam, por isso, no fim da ordenação. A secção 6.5.2 avalia as recomendações obtidas com esta ordenação numa avaliação offline; o seu efeito na utilidade percebida não foi avaliado com utilizadores.

6. Análise, desenho e implementação do sistema

O sistema desenvolvido, "Análise de Cesto de Compras", recomenda produtos a um consumidor enquanto este constrói uma lista de compras, com base em regras de associação extraídas de um histórico transacional. A técnica segue a formulação clássica de regras de associação em bases de dados de transações (Agrawal, Imieliński & Swami, 1993) e o algoritmo Apriori (Agrawal & Srikant, 1994).

O motor de recomendação utiliza o Apriori, e não o FP-Growth, apesar das vantagens de eficiência que a literatura atribui a este último (secção 2.5). A decisão resulta dos resultados medidos no Capítulo 5: com o catálogo de 169 itens do Dataset 1, a implementação do Apriori da biblioteca mlxtend foi a mais rápida, produz exatamente as mesmas regras e demora cerca de 0,34 s com os parâmetros por omissão do motor (secção 5.5), um tempo compatível com o cálculo das regras a cada pedido. A simplicidade do algoritmo facilita também a explicação das recomendações e a configuração dos seus parâmetros.

O sistema divide-se em três componentes:

- API backend (api_backend/), em Python com FastAPI (Ramírez, s.d.): é o único ponto do sistema onde as regras de associação e as recomendações são calculadas, e onde ficam guardados os utilizadores, as sessões, as listas de compras e os parâmetros do motor.
- Aplicação de backoffice (AppItensRecomendados.py), em Streamlit (Streamlit Inc., s.d.): destina-se ao administrador e permite consultar estatísticas globais do conjunto de dados (regras de associação e itens mais frequentes) e configurar os parâmetros do motor de recomendação.
- Aplicação móvel (mercado_app/mercado_app/), em Flutter (Flutter, s.d.): destina-se ao consumidor final e permite criar conta, autenticar-se, construir e guardar listas de compras e receber recomendações que incorporam o seu histórico.

6.1 Arquitetura geral

O sistema segue uma arquitetura cliente-servidor com um único serviço central: a API backend concentra a lógica de negócio, o motor de recomendação e a persistência, e os dois clientes (backoffice e aplicação móvel) comunicam com ela exclusivamente por REST (Fielding, 2000) sobre HTTP/JSON. Nenhum dos clientes calcula regras de associação localmente; assim, o cálculo das recomendações e das estatísticas segue uma lógica centralizada.

6.1.1 Camadas e componentes

A API está organizada em camadas, no sentido descrito por Fowler (2002):

- Apresentação/controladores — app/routers/(auth.py, lists.py, recommendations.py, admin.py). Cada router agrupa endpoints por área funcional e é registado em app/main.py.
- Contratos de dados — app/schemas.py. Os modelos Pydantic funcionam como objetos de transferência de dados (DTO) e validam a entrada e a saída de cada endpoint.
- Serviços transversais — app/dependencies.py (resolução do utilizador a partir do token) e app/security.py (hash e verificação de palavras-passe com bcrypt; Provos & Mazières, 1999).
- Domínio/motor — app/recommendation_engine.py (classe RecommendationEngine) e app/engine_singleton.py.
- Persistência — app/models.py e app/database.py, com SQLAlchemy sobre SQLite.

O backoffice (`AppItensRecomendados.py`) é um cliente de pequena dimensão. Quatro funções (`get_settings`, `update_settings`, `get_rules_raw` e `get_top_items`) fazem as chamadas à API; três funções auxiliares (`items_covered_by_rules`, `compute_items_without_rules` e `compute_item_rule_counts`) calculam, a partir das regras e dos itens já devolvidos pela API, que itens aparecem em alguma regra e em quantas; e a função `main` desenha as páginas Streamlit. Estas contagens são apenas comparações sobre os resultados da API: o backoffice não extrai regras de associação.

A aplicação móvel separa a interface do acesso à rede. Os ecrãs (`LoginScreen`, `RegisterScreen`, `HomeScreen`, `NewListScreen` e o `_StartupGate` em `main.dart`) são widgets com estado (`StatefulWidget`), que guardam o estado local; todas as chamadas HTTP passam por `ApiService`, que também gere o token em `SharedPreferences`. Como a pesquisa de itens é feita a cada tecla e as recomendações a cada alteração da lista, as respostas podem chegar fora de ordem: o `NewListScreen` numera os pedidos de cada tipo e só aplica ao ecrã a resposta do pedido mais recente, ignorando as restantes, pelo que nunca mostra sugestões de uma versão anterior da lista. Não é aplicado um atraso (debounce) à pesquisa, pelo que cada tecla gera um pedido.

6.1.2 Padrões aplicados

- REST — recursos expostos por URL e HTTP (GET, POST, PUT e PATCH).
- Arquitetura em camadas
- Injeção de dependências — o FastAPI injeta a sessão de base de dados (Depends(get_db)) e o utilizador autenticado (Depends(get_current_user)) em cada endpoint. Os testes aproveitam este mecanismo para substituir a base de dados (app.dependency_overrides).
- Singleton — `get_engine()` é decorada com `@lru_cache`, pelo que o conjunto de dados só é lido e codificado uma vez por processo; do lado Flutter,

`ApiService.instance` é a única instância do serviço (Gamma et al., 1994). A cache evita apenas a leitura e a codificação repetidas do ficheiro: as regras de associação continuam a ser calculadas em cada pedido de recomendações e de estatísticas. A instância não é partilhada entre vários processos do servidor (por exemplo, vários workers do Uvicorn), que teriam cada um a sua cópia em memória, e não é invalidada se o ficheiro de dados mudar, sendo então necessário reiniciar a API.

- DTO — modelos Pydantic separam o formato de rede das entidades ORM (Fowler, 2002).
- Sessão por pedido — get_db() abre uma sessão SQLAlchemy por pedido e fecha-a sempre no fim, mesmo em erro.
- Tabela de linha única para configuração — Settings guarda os parâmetros globais do motor; get_or_create_settings cria a linha com os valores por omissão na primeira utilização.

6.1.3 Fluxo de dados

No arranque, a API cria as tabelas em `app.db` caso não existam. Na primeira chamada a `get_engine()`, o `RecommendationEngine` lê o ficheiro `dataSort.csv`, que contém o Dataset 1 (Groceries) já corrigido como descrito na secção 5.2, com as 9 835 transações, e que se encontra na pasta `Codigo`, ao lado da pasta `api_backend`. Cada linha do ficheiro corresponde a uma transação: a primeira coluna é um índice e as restantes contêm os itens do cesto, por exemplo `0,citrus fruit,margarine,ready soups,semi-finished bread`. Na leitura, os itens são convertidos para minúsculas, as células vazias são ignoradas e é construída uma matriz booleana one-hot com o `TransactionEncoder`; essa matriz fica em memória (`base_df`).

Quando a aplicação móvel pede recomendações, o router lê os parâmetros de `Settings` e as listas já guardadas pelo utilizador. O motor junta essas listas ao conjunto de dados base, repetidas `user_weight` vezes, executa o Apriori com `min_support` e gera as regras com `min_confidence`. São consideradas apenas as regras cujo antecedente está totalmente contido na lista atual (para uma regra {A, B} ⇒ C, a lista tem de conter A e B); os consequentes que ainda não estão na lista são ordenados pela elevação (lift), mantendo-se para cada item o valor mais alto e desempatando pelo nome do item. Se houver menos de cinco recomendações, a lista é completada com os itens mais frequentes do mesmo conjunto de dados ponderado, excluindo os que já estão na lista ou já foram recomendados; estas sugestões resultam apenas da popularidade dos itens e não de regras. Os pedidos do backoffice usam sempre só o conjunto de dados base, sem histórico de utilizadores. A Figura 10 resume os componentes envolvidos neste fluxo e as comunicações entre eles.

A personalização é feita por repetição das transações. Sendo N o número de transações do conjunto base, M o número de listas guardadas pelo utilizador e w o peso (`user_weight`), o suporte de um conjunto de itens X no conjunto ponderado é $s(X) = (n(X) + w \cdot m(X)) / (N + w \cdot M)$, em que n(X) e m(X) são, respetivamente, o número de

transações do conjunto base e de listas do utilizador que contêm X. O peso por omissão é 1000 e pode variar entre 1 e 1000. Com o suporte mínimo por omissão de 0,01 e N = 9 835, um conjunto de itens precisa de cerca de 99 transações para ser frequente: com w = 3, uma lista guardada representaria apenas três transações e teria um efeito praticamente nulo nas regras; com w = 1000, uma única lista corresponde a cerca de 9% do conjunto ponderado (1 000/10 835), pelo que as combinações de itens que o utilizador já escolheu passam a ser frequentes e geram regras próprias. Essas regras, porém, só são usadas quando a lista em construção contém itens das listas guardadas. Por exemplo, para um utilizador que guardou a lista «Jantar» (whipped/sour cream, other vegetables, yogurt, citrus fruit e tropical fruit), uma nova lista só com whipped/sour cream recebe, sem histórico, other vegetables, whole milk, root vegetables, yogurt e rolls/buns e, com o histórico, citrus fruit, other vegetables, tropical fruit, yogurt e whole milk. Já para bottled water, soda e root vegetables, que não constam dessa lista, as cinco recomendações são as mesmas com e sem histórico (other vegetables, whole milk, yogurt, rolls/buns e tropical fruit), mudando apenas a ordem: nesse caso, o efeito do histórico resume-se a reduzir o suporte das restantes combinações, cujo denominador passa de N para N + w·M, e a alterar a ordem dos itens mais frequentes usados para completar as recomendações. Estes exemplos foram obtidos com o motor da API, através do script da secção 6.5.2.

Esta abordagem tem limitações. As listas guardadas representam intenções de compra e não compras confirmadas, e o reforço do histórico pode levar o sistema a recomendar sobretudo o que o utilizador já escolheu. O histórico não é limitado no tempo nem no número de listas, pelo que, com várias listas, o histórico do utilizador pode dominar o conjunto base (dez listas com w = 1000 equivalem a cerca de 10 000 transações, mais do que o conjunto base), e o peso tem o mesmo valor para utilizadores com históricos de dimensões muito diferentes. Além disso, a repetição é física: cada pedido constrói uma matriz com N + w·M linhas, cujo custo em memória e em tempo cresce com o histórico; este custo não foi medido. Os itens desconhecidos, que não pertencem ao catálogo, são ignorados na codificação das listas do utilizador, e cada lista guardada conta como uma transação, mesmo que repita outra.

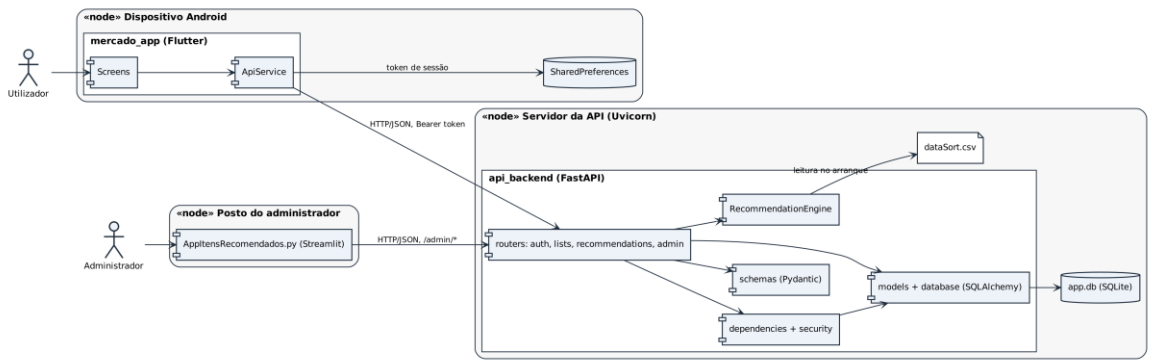


Figura 10 – Arquitetura de componentes do sistema

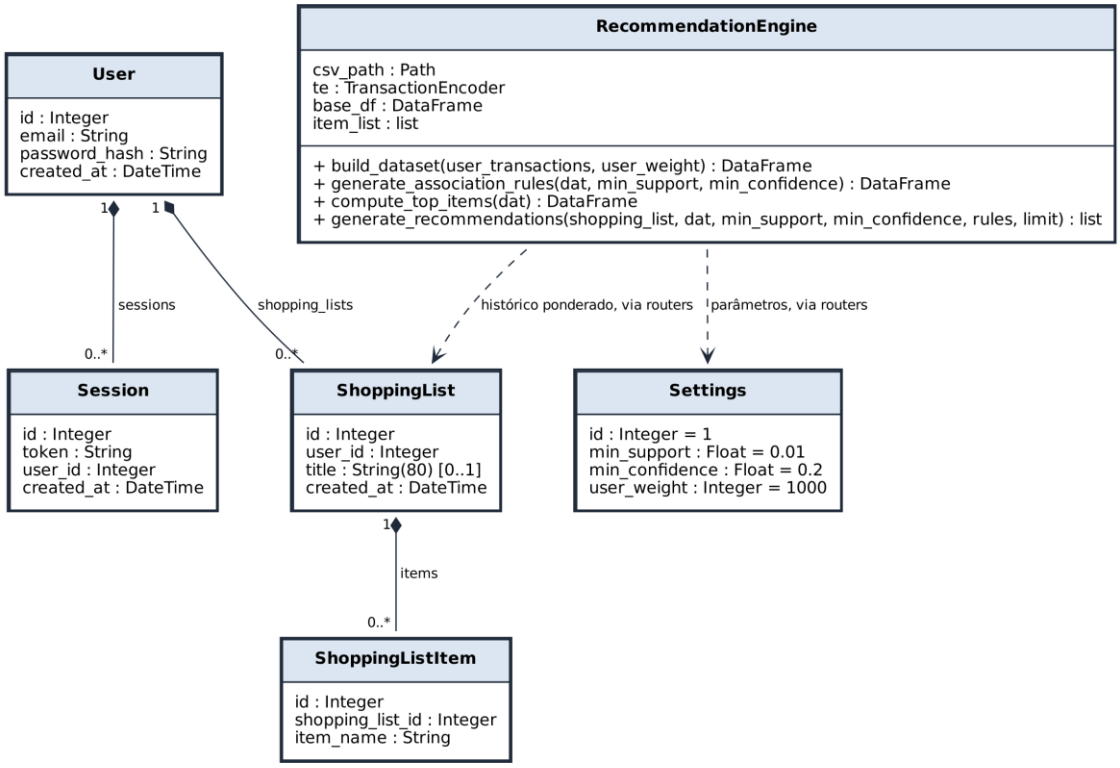


Figura 11 – Classes de domínio principais

A Figura 11 apresenta as classes de domínio principais e as relações entre elas. As composições refletem cascade="all, delete-orphan" em models.py: ao apagar um utilizador, apagam-se também as suas sessões e listas. O motor não acede diretamente à base de dados: recebe os parâmetros e o histórico já lidos pelos routers.

6.1.4 Interface REST

A Tabela 13 lista os endpoints expostos pela API, tal como definidos nos routers.

Tabela 13 – Endpoints da API

Método	Rota	Autenticação	Função	Cliente
POST	`/auth/register`	—	`auth.register`	Aplicação móvel
POST	`/auth/login`	—	`auth.login`	Aplicação móvel
POST	`/auth/logout`	Token	`auth.logout`	Aplicação móvel
GET	`/items?search=`	—	`lists.list_items`	Aplicação móvel
GET	`/lists`	Token	`lists.get_lists`	Aplicação móvel

Método	Rota	Autenticação	Função	Cliente
POST	`/lists`	Token	`lists.create_list`	Aplicação móvel
PATCH	`/lists/{list_id}`	Token	`lists.update_list`	Aplicação móvel
POST	`/recommendations`	Token	`recommendations.recommend`	Aplicação móvel
GET	`/admin/settings`	—	`admin.read_settings`	Backoffice
PUT	`/admin/settings`	—	`admin.update_settings`	Backoffice
GET	`/admin/stats/top-items`	—	`admin.top_items`	Backoffice
GET	`/admin/stats/rules`	—	`admin.association_rules_endpoint`	Backoffice
GET	`/health`	—	`main.health_check`	Infraestrutura

Os pedidos e respostas usam JSON. Por exemplo, para o utilizador com a lista «Jantar» da secção 6.1.3, a aplicação móvel envia `POST /recommendations` com o corpo `{"current_items": ["bottled water", "soda", "root vegetables"]}` e o cabeçalho `Authorization: Bearer <token>`, e a API responde 200 com `{"recommended_items": ["other vegetables", "whole milk", "rolls/buns", "yogurt", "tropical fruit"]}`. Os erros devolvem um objeto com o campo `detail` e os seguintes códigos: 401, quando o token falta ou já não é válido; 404, quando a lista a alterar não existe ou pertence a outro utilizador; 409, quando o email já está registado; e 422, quando os dados não respeitam o esquema Pydantic (por exemplo, email inválido, palavra-passe com menos de 8 caracteres, lista sem itens ou parâmetros fora dos limites).

Os endpoints `/admin/*` não exigem autenticação: no protótipo, «administrador» é apenas um papel de utilização, sem qualquer autorização técnica implementada, e qualquer cliente que alcance a API pode consultar as estatísticas e alterar os parâmetros do motor. Antes de disponibilizar o sistema fora de um ambiente controlado, estas operações têm de ser protegidas no servidor, com autenticação e verificação de permissões; restringir apenas a interface do backoffice não seria suficiente.

6.1.5 Interface da aplicação móvel

A interface da aplicação móvel foi desenhada antes da implementação, através de *mockups*, apresentados na Figura 12: (a) ecrã de autenticação, onde o utilizador

introduz o email e a palavra-passe; (b) ecrã de criação de conta; (c) ecrã inicial, com o histórico de listas de compras e o acesso à criação de uma nova lista; e (d) ecrã de construção da lista, onde o utilizador pesquisa e adiciona itens e recebe os itens recomendados com base nas regras de associação. Os *mockups* serviram de referência ao desenvolvimento, podendo a interface final diferir em pormenores.



Figura 12 – Mockups da aplicação móvel: (a) autenticação, (b) criação de conta, (c) ecrã inicial, (d) construção da lista

6.2 Atores e casos de uso

O sistema tem três atores e nove casos de uso, sete na aplicação móvel e dois no backoffice. A modelação segue a abordagem orientada a casos de uso de Jacobson et al. (1992), tal como adotada pelo ICONIX (Rosenberg & Stephens, 2007). Os diagramas seguem a notação UML (Object Management Group, 2017).

6.2.1 Requisitos

Os requisitos iniciais e os casos de uso foram definidos no início do Projeto II, a partir dos objetivos do projeto, e serviram de base, juntamente com os mockups, ao desenvolvimento. Durante a implementação surgiram alterações, como o título das listas, a marcação de itens e a passagem do cálculo das regras para a API, pelo que os requisitos, as fichas de casos de uso e os diagramas de robustez e de sequência foram atualizados e completados a partir do código final, de modo a documentar o sistema tal como foi implementado. Os requisitos funcionais (Tabela 14) correspondem diretamente aos casos de uso descritos na secção 6.2.4; os requisitos não funcionais (Tabela 15) descrevem as restrições técnicas e os critérios de qualidade do sistema.

Tabela 14 – Requisitos funcionais

ID	Requisito	Caso de uso
RF01	O utilizador deve poder criar uma conta com email e palavra-passe.	UC01
RF02	O utilizador deve poder autenticar-se com as suas credenciais.	UC02
RF03	O utilizador deve poder terminar a sessão.	UC03
RF04	O utilizador deve poder consultar o histórico das suas listas de compras, dar ou mudar o título de cada lista e marcar os itens já comprados.	UC04
RF05	O utilizador deve poder construir uma lista de compras, pesquisando e adicionando itens.	UC05
RF06	O sistema deve recomendar itens com base nas regras de associação e nos itens já escolhidos.	UC06
RF07	O utilizador deve poder guardar a lista de compras, com um título opcional.	UC07
RF08	O administrador deve poder consultar estatísticas sobre as regras de associação, os itens mais frequentes e os itens com e sem regras associadas.	UC08
RF09	O administrador deve poder configurar os parâmetros do motor de recomendação.	UC09

Tabela 15 – Requisitos não funcionais

ID	Requisito
RNF01	As palavras-passe são guardadas apenas sob a forma de hash bcrypt.
RNF02	Os endpoints protegidos exigem um token de autenticação do tipo Bearer.
RNF03	Os dados são persistidos numa base de dados SQLite, através do SQLAlchemy.
RNF04	A API segue o estilo REST, com dados em JSON validados com Pydantic.
RNF05	A aplicação móvel destina-se a Android e é desenvolvida em Flutter.
RNF06	As regras de associação e as recomendações são calculadas apenas no servidor.
RNF07	A API dispõe de testes automáticos executados com pytest.
RNF08	Cada utilizador só acede às suas próprias listas; um pedido sobre uma lista de outro utilizador é tratado como lista inexistente (404).
RNF09	Perante falhas de rede ou erros da API, os clientes mostram uma mensagem compreensível, sem terminar abruptamente, e distinguem uma sessão inválida (401) de uma falha de rede.

Os requisitos RNF01 a RNF07 são restrições tecnológicas, verificadas por inspeção do código. O RNF08 é verificado pelo teste test_list_title_create_and_rename (Tabela 26). O RNF09 foi implementado na aplicação móvel e no backoffice, mas não é verificado por testes automáticos.

6.2.2 Atores

A Tabela 16 descreve os atores identificados no código.

Tabela 16 – Atores do sistema

Ator	Descrição	Interface	Indício no código
Visitante	Pessoa que usa a aplicação móvel sem sessão iniciada. Pode criar conta ou autenticar-se.	Aplicação móvel	`_StartupGate` mostra o `LoginScreen` quando não há token guardado (`lib/main.dart`)
Utilizador autenticado	Consumidor com sessão válida. Consulta o histórico, constrói e guarda listas, recebe recomendações e termina a sessão.	Aplicação móvel	Endpoints com `Depends(get_current_user)` (`routers/lists.py`, `routers/recommendations.py`)
Administrador	Gestor do sistema. Consulta estatísticas globais e configura os parâmetros do motor. Não se autentica: os endpoints `/admin/*` estão abertos.	Backoffice	Comentário inicial de `AppItensRecomendados.py`; nota em `routers/admin.py`

A API backend não é tratada como ator: é o próprio sistema que os casos de uso descrevem.

6.2.3 Diagrama de casos de uso

A Figura 13 apresenta o diagrama geral, com uma única fronteira para o sistema, organizada em dois subsistemas, um por aplicação cliente; a API não aparece como ator, porque faz parte do sistema. O caso UC06 estende o UC05: sempre que a lista em construção tem pelo menos um item, cada alteração pede novas recomendações à API; com a lista vazia não há pedido, pelo que a relação não é de inclusão obrigatória.

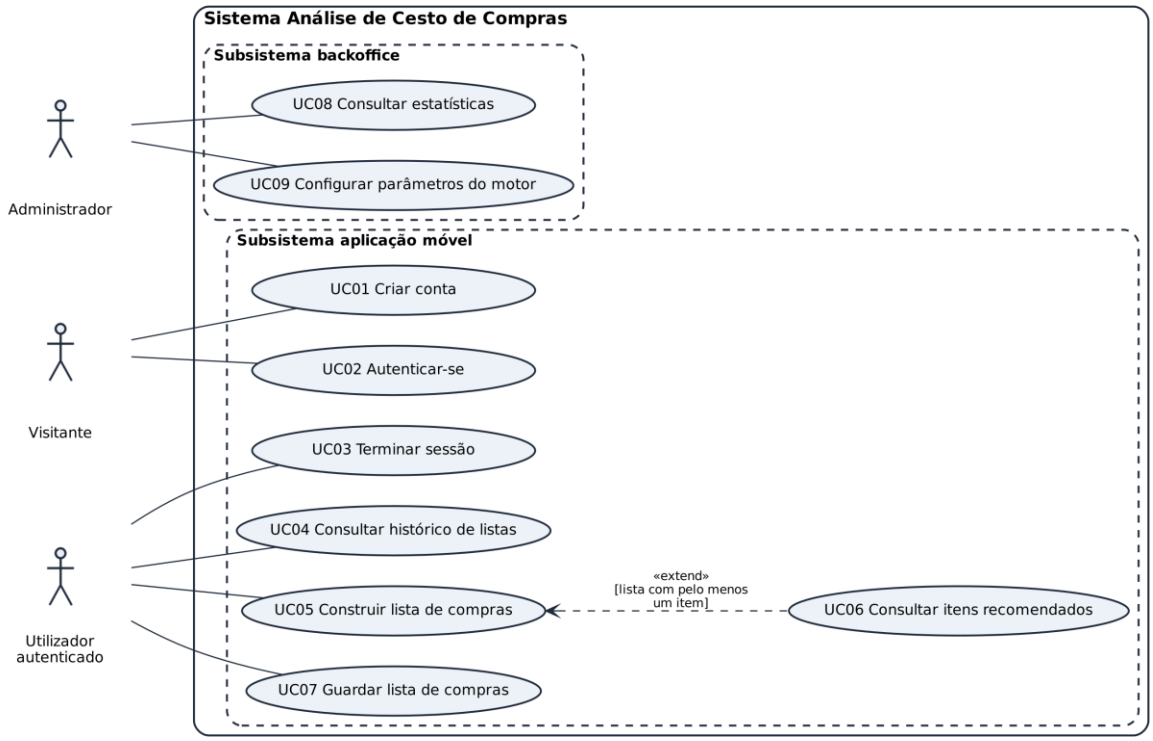


Figura 13 – Diagrama geral de casos de uso

6.2.4 Fichas de casos de uso

As Tabelas 17 a 25 descrevem cada caso de uso. Os passos dos fluxos são os mesmos que aparecem nos diagramas de robustez (secção 6.3) e de sequência (secção 6.4). Como as fichas foram completadas a partir do código final (secção 6.2.1), os fluxos indicam as classes e os métodos envolvidos em cada passo, para permitir a rastreabilidade até aos diagramas e ao código; numa especificação de requisitos destinada ao utilizador, estes pormenores técnicos seriam omitidos e os fluxos ficariam centrados na interação entre o ator e o sistema.

Tabela 17 – Ficha do caso de uso UC01 Criar conta

Campo	Descrição
Identificador	UC01
Nome	Criar conta
Ator(es)	Visitante
Descrição	O visitante regista-se com email e palavra-passe; a conta é criada e a sessão fica iniciada de imediato.
Pré-condições	A aplicação está aberta sem token guardado. A API está acessível.

Campo	Descrição
Pós-condições	Existe um utilizador com a palavra-passe em hash bcrypt e uma sessão associada. O token está guardado no dispositivo e é mostrado o HomeScreen.
Fluxo principal	1) No LoginScreen, o visitante toca em "Criar conta". 2) O RegisterScreen pede email, palavra-passe e confirmação. 3) O visitante preenche os campos e toca em "Criar conta". 4) _register confirma localmente que as palavras-passe coincidem. 5) ApiService.register envia POST /auth/register. 6) A API valida o RegisterRequest (email válido, mínimo de 8 caracteres, confirmação igual). 7) auth.register confirma que o email não existe. 8) Calcula o hash com hash_password e grava o User. 9) Cria a Session e devolve TokenResponse. 10) ApiService._saveToken guarda o token. 11) A aplicação navega para o HomeScreen.
Fluxos alternativos	4a) As palavras-passe não coincidem: aparece "As palavras-passe não coincidem." e não é feito qualquer pedido.
Fluxos de exceção	5a) Falha de rede: a mensagem da exceção é mostrada no ecrã. 6a) Email inválido ou palavra-passe com menos de 8 caracteres: a API responde 422 e a aplicação mostra o detalhe do erro. A API não define um limite máximo para a palavra-

Campo	Descrição
	passe; o bcrypt considera apenas os seus primeiros 72 bytes. 7a) Email já registado: 409 "Este email já está registado." A unicidade do email é garantida pela consulta prévia em auth.register e por uma restrição UNIQUE na base de dados. O domínio do email é normalizado pelo Pydantic, mas a parte local não é convertida para minúsculas, pelo que endereços que difiram apenas em maiúsculas nessa parte são tratados como contas distintas.
Classes / endpoints	`RegisterScreen, ApiService.register, ApiService._saveToken; POST /auth/register → auth.register, schemas.RegisterRequest, schemas.TokenResponse, security.hash_password, models.User, models.Session`

Tabela 18 – Ficha do caso de uso UC02 Autenticar-se

Campo	Descrição
Identificador	UC02
Nome	Autenticar-se
Ator(es)	Visitante
Descrição	O visitante introduz email e palavra-passe e obtém um novo token de sessão.
Pré-condições	A conta existe. Não há token guardado no dispositivo.
Pós-condições	Existe uma nova `Session` na base de dados. O token está guardado e é mostrado o `HomeScreen`.
Fluxo principal	1) No arranque, `_StartupGate` chama `ApiService.hasSession`, que devolve falso, e mostra o `LoginScreen`. 2) O visitante introduz email e palavra-passe e toca em "Entrar".

Campo	Descrição
	3) `_login` chama `ApiService.login`, que envia `POST /auth/login`. 4) `auth.login` procura o `User` pelo email. 5) `verify_password` compara a palavra-passe com o hash. 6) A API cria uma `Session` e devolve `TokenResponse`. 7) `ApiService._saveToken` guarda o token. 8) A aplicação navega para o `HomeScreen`.
Fluxos alternativos	1a) Já existe token guardado: `_StartupGate` mostra diretamente o `HomeScreen`, sem pedido à API. Ter um token guardado não garante uma sessão válida: o token só é validado no primeiro pedido protegido e, se este receber 401, o histórico informa que a sessão deixou de ser válida e oferece "Iniciar sessão", que apaga o token local e mostra o `LoginScreen` (UC04, exceção 3a).
Fluxos de exceção	3a) Falha de rede: a mensagem da exceção é mostrada. 4a/5a) Email inexistente ou palavra-passe errada: 401 "Email ou palavra-passe inválidos."
Classes / endpoints	`_StartupGate`, `LoginScreen`, `ApiService.hasSession`, `ApiService.login`; `POST /auth/login` → `auth.login`, `schemas.LoginRequest`, `security.verify_password`, `models.User`, `models.Session`

Tabela 19 – Ficha do caso de uso UC03 Terminar sessão

Campo	Descrição
Identificador	UC03
Nome	Terminar sessão

Campo	Descrição
Ator(es)	Utilizador autenticado
Descrição	O utilizador termina a sessão; o token usado é invalidado no servidor e apagado do dispositivo. As sessões noutros dispositivos mantêm-se.
Pré-condições	Sessão iniciada; o utilizador está no `HomeScreen`.
Pós-condições	Em caso de sucesso: a Session deixa de existir na base de dados, o token é removido de SharedPreferences e o LoginScreen é mostrado com a pilha de navegação limpa. Garantia mínima (falha de rede, exceção 3a): o token é removido do dispositivo e o LoginScreen é mostrado, mas a Session continua válida no servidor; como as sessões não expiram, um token copiado continuaria a ser aceite (limitação indicada no Capítulo 7).
Fluxo principal	1) O utilizador toca no ícone "Terminar sessão". 2) `HomeScreen._logout` chama `ApiService.logout`. 3) É enviado `POST /auth/logout` com o token Bearer. 4) `get_current_session` obtém a `Session` correspondente. 5) `auth.logout` apaga a `Session` e responde. 6) `ApiService._clearToken` apaga o token local. 7) `pushAndRemoveUntil` abre o `LoginScreen`.
Fluxos alternativos	—
Fluxos de exceção	3a) Falha de rede: a exceção é ignorada e o token é apagado localmente na mesma; a sessão continua válida no servidor.

Campo	Descrição
	4a) Token já inválido: a API responde 401, a resposta é ignorada e o fluxo continua no passo 6.
Classes / endpoints	`HomeScreen._logout`, `ApiService.logout`, `ApiService._clearToken`; `POST /auth/logout` → `dependencies.get_current_session`, `auth.logout`, `models.Session`

Tabela 20 – Ficha do caso de uso UC04 Consultar histórico de listas

Campo	Descrição
Identificador	UC04
Nome	Consultar histórico de listas
Ator(es)	Utilizador autenticado
Descrição	O utilizador vê as suas listas guardadas, da mais recente para a mais antiga, e pode expandir cada uma para ver os itens, marcar os que já comprou e dar ou mudar o título da lista.
Pré-condições	Sessão iniciada.
Pós-condições	Na consulta, nenhuma alteração de estado. Se o título for alterado, a ShoppingList fica com o novo título.
Fluxo principal	1) Ao abrir o `HomeScreen`, `initState` chama `ApiService.getLists`. 2) É enviado `GET /lists` com o token. 3) `get_current_user` resolve o utilizador. 4) `lists.get_lists` lê as `ShoppingList` desse utilizador por ordem decrescente de `created_at`. 5) A API devolve uma lista de `ShoppingListOut`. 6) `SavedList.fromJson` converte cada elemento.

Campo	Descrição
	7) O ecrã mostra cada lista como `ExpansionTile`, com o número de itens e a data. 8) Ao tocar numa lista, esta expande e mostra os itens.
Fluxos alternativos	5a) Sem listas: aparece "Ainda não tem listas guardadas." 1a) Puxar para atualizar, ou regressar do `NewListScreen`, chama `_refresh` e repete os passos 1 a 7. 8a) Ao marcar ou desmarcar um item, `_toggleItem` guarda a marcação em `SharedPreferences`, apenas no dispositivo. 8b) Ao tocar em "Dar título" (ou "Mudar título"), `_renameList` abre um diálogo; `ApiService.renameList` envia `PATCH /lists/{id}` com o título, `lists.update_list` grava-o (um título vazio remove-o) e `_refresh` recarrega o histórico.
Fluxos de exceção	3a) Token inválido (401): aparece "A sessão terminou ou deixou de ser válida." e o botão "Iniciar sessão", que apaga o token local e mostra o LoginScreen; repetir o pedido não resolveria. 3b) Falha de rede ou outro erro: aparece "Não foi possível carregar o histórico." com o botão "Tentar novamente"; a aplicação não volta ao login. 8b1) Lista inexistente ou de outro utilizador: a API responde 404 "Lista não encontrada". Em caso de erro, aparece "Não foi possível mudar o título".
Classes / endpoints	`HomeScreen`, `ApiService.getLists`, `SavedList`; `GET /lists` → `dependencies.get_current_user`, `lists.get_lists`, `schemas.ShoppingListOut`, `models.ShoppingList`, `models.ShoppingListItem`;

Campo	Descrição
	`HomeScreen._toggleItem`, `HomeScreen._renameList`, `ApiService.renameList`; `PATCH /lists/{list_id}` → `lists.update_list`, `schemas.ShoppingListUpdate`

Tabela 21 – Ficha do caso de uso UC05 Construir lista de compras

Campo	Descrição
Identificador	UC05
Nome	Construir lista de compras
Ator(es)	Utilizador autenticado
Descrição	O utilizador pesquisa itens do conjunto de dados e adiciona-os ou remove-os da lista em construção. Sempre que a lista tem pelo menos um item, cada alteração atualiza as recomendações (o UC06 estende este caso de uso).
Pré-condições	Sessão iniciada, o utilizador está no `HomeScreen`.
Pós-condições	A lista em construção (`_currentItems`) está atualizada em memória. Nada é gravado até ao UC07.
Fluxo principal	1) O utilizador toca em "Nova lista" e abre o `NewListScreen`. 2) Escreve no campo "Pesquisar item". 3) A cada alteração, `_search` chama `ApiService.searchItems`, que envia `GET /items?search=`. 4) `lists.list_items` filtra `engine.item_list` pelos itens que contêm o termo. 5) A aplicação mostra os resultados. 6) O utilizador toca num resultado e `_addItem` é chamado.

Campo	Descrição
	7) O item é acrescentado à lista e a pesquisa é limpa. 8) É executado o UC06 (`_refreshRecommendations`).
Fluxos alternativos	2a) Campo vazio: os resultados são limpos e não há pedido. 6a) Item já presente: é ignorado. 6b) O utilizador toca em "+" num item recomendado: segue para o passo 7. 6c) O utilizador remove um item (ícone do `Chip`): o item sai da lista e é executado o UC06.
Fluxos de exceção	3a) Falha de rede: a mensagem de erro aparece no fundo do ecrã.
Classes / endpoints	`NewListScreen` (`_search`, `_addItem`), `ApiService.searchItems`; `GET /items` → `lists.list_items`, `engine_singleton.get_engine`, `RecommendationEngine.item_list`

Tabela 22 – Ficha do caso de uso UC06 Consultar itens recomendados

Campo	Descrição
Identificador	UC06
Nome	Consultar itens recomendados
Ator(es)	Utilizador autenticado (através do UC05, que este caso de uso estende)
Descrição	A API calcula até cinco itens recomendados para a lista em construção, ponderando o histórico do próprio utilizador.
Pré-condições	Sessão válida, a lista em construção tem pelo menos um item.
Pós-condições	São mostradas até cinco recomendações. Nada é gravado.

Campo	Descrição
Fluxo principal	1) `_refreshRecommendations` chama `ApiService.getRecommendations`, que envia `POST /recommendations` com `current_items`. 2) `get_current_user` resolve o utilizador. 3) `get_or_create_settings` lê os parâmetros. 4) `get_engine` devolve o motor. 5) O router lê as `ShoppingList` do utilizador e forma `user_transactions`. 6) `build_dataset` junta-as ao conjunto de dados base, repetidas `user_weight` vezes. 7) Os itens atuais são normalizados (sem espaços nas pontas, minúsculas). 8) `generate_recommendations` chama `generate_association_rules` (Apriori e regras por confiança), seleciona as regras cujo antecedente está totalmente contido na lista, recolhe os seus consequentes, retira os que já estão na lista, mantém para cada item o maior lift e ordena por lift decrescente, desempatando pelo nome do item. 9) Se houver menos de 5, completa com os itens mais frequentes do mesmo conjunto de dados ponderado (`compute_top_items`), excluindo os itens da lista e os já recomendados; estas sugestões resultam da popularidade dos itens e não de regras. 10) A API devolve `RecommendationResponse` 200. 11) A aplicação aplica a resposta apenas se corresponder ao pedido mais recente, retira os itens já presentes e mostra a lista com botões "+".
Fluxos alternativos	1a) Lista vazia: as recomendações são limpas e aparece "Adicione pelo menos um item para ver recomendações.", sem pedido.

Campo	Descrição
	5a) Sem histórico: `build_dataset` devolve uma cópia do conjunto de dados base. 8a) Sem conjuntos frequentes: não há regras e as recomendações vêm só dos itens mais frequentes. 11a) Resposta vazia: "Sem recomendações para já."
Fluxos de exceção	2a) Token inválido (401): aparece no ecrã a mensagem da API "Sessão inválida ou expirada. Autentica-te novamente."; ao voltar ao histórico, é proposto iniciar sessão (UC04, exceção 3a). 2b) Falha de rede: a mensagem de erro aparece no ecrã e as recomendações anteriores mantêm-se.
Classes / endpoints	`NewListScreen._refreshRecommendations`, `ApiService.getRecommendations`; `POST /recommendations` → `recommendations.recommend`, `admin.get_or_create_settings`, `RecommendationEngine.build_dataset`, `generate_recommendations`, `generate_association_rules`, `compute_top_items`, `models.Settings`, `models.ShoppingList`

Tabela 23 – Ficha do caso de uso UC07 Guardar lista de compras

Campo	Descrição
Identificador	UC07
Nome	Guardar lista de compras
Ator(es)	Utilizador autenticado
Descrição	O utilizador grava a lista em construção, que passa a fazer parte do seu histórico e das suas recomendações futuras.

Campo	Descrição
Pré-condições	Sessão iniciada, a lista em construção tem pelo menos um item.
Pós-condições	Existe uma nova `ShoppingList` com os seus `ShoppingListItem` (em minúsculas). O `HomeScreen` volta a ser mostrado e recarrega o histórico.
Fluxo principal	1) O utilizador toca em "Guardar lista". 2) `_save` chama `ApiService.saveList`, que envia `POST /lists` com `items` e, se tiver sido preenchido, `title`. 3) `get_current_user` resolve o utilizador. 4) A API valida `ShoppingListCreate` (pelo menos um item; título com até 80 caracteres). 5) `lists.create_list` normaliza os itens (sem espaços nas pontas, minúsculas), ignora os vazios e remove os duplicados, e cria a `ShoppingList` e um `ShoppingListItem` por item. 6) Grava e devolve `ShoppingListOut` 201. 7) `Navigator.pop` volta ao `HomeScreen`. 8) O `HomeScreen` chama `_refresh` (UC04).
Fluxos alternativos	1a) Lista vazia: o botão "Guardar lista" está desativado.
Fluxos de exceção	3a) Token inválido (401): aparece a mensagem "Sessão inválida ou expirada. Autentica-te novamente." e o utilizador fica no ecrã, sem a lista ser gravada. 3b) Falha de rede: a mensagem de erro aparece e o utilizador fica no ecrã, podendo tentar guardar de novo. 4a) Corpo inválido: 422. 5a) Depois da normalização não resta nenhum item (por exemplo, só textos vazios): 422 "A lista tem de ter pelo menos um item." A aplicação só permite escolher

Campo	Descrição
	itens do catálogo, mas um pedido direto à API pode guardar itens desconhecidos, que são ignorados pelo motor (secção 6.1.3).
Classes / endpoints	`NewListScreen._save`, `ApiService.saveList`, `HomeScreen._refresh`; `POST /lists → lists.create_list`, `schemas.ShoppingListCreate`, `schemas.ShoppingListOut`, `models.ShoppingList`, `models.ShoppingListItem`

Tabela 24 – Ficha do caso de uso UC08 Consultar estatísticas

Campo	Descrição
Identificador	UC08
Nome	Consultar estatísticas
Ator(es)	Administrador
Descrição	O administrador vê as regras de associação do conjunto de dados global, o número de itens com e sem regras, o top 20 de itens mais frequentes, os itens que aparecem em mais regras, a lista dos itens sem regras associadas e um gráfico de suporte × confiança.
Pré-condições	A API está em execução e o backoffice foi iniciado (streamlit run AppItensRecomendados.py).
Pós-condições	Nenhuma alteração de estado (consulta).
Fluxo principal	1) O administrador abre o backoffice; a página por omissão é "Estatísticas". 2) `main` chama `get_settings` (`GET /admin/settings`) para confirmar a ligação. 3) `get_rules_raw` envia `GET /admin/stats/rules`. 4) `association_rules_endpoint` lê `Settings`, obtém o motor, constrói o

Campo	Descrição
	conjunto de dados base e chama `generate_association_rules`. 5) A API devolve a lista de `AssociationRuleOut` 200. 6) `get_top_items` envia `GET /admin/stats/top-items`; `admin.top_items` chama `compute_top_items`. 7) `format_rules_for_display` converte antecedentes e consequentes em texto e o backoffice mostra a tabela de regras. 8) `items_covered_by_rules` determina os itens que aparecem em alguma regra; o backoffice mostra o total de itens, os itens com regras e os itens sem regras. 9) O backoffice mostra os 20 primeiros itens com contagem e percentagem. 10) `compute_item_rule_counts` conta em quantas regras aparece cada item e o backoffice mostra a tabela "Itens que Mais Aparecem nas Regras". 11) `compute_items_without_rules` obtém os itens sem nenhuma regra e o backoffice mostra a sua lista. 12) Desenha o gráfico de dispersão suporte × confiança (seaborn) com `st.pyplot`.
Fluxos alternativos	5a) Sem regras para os limiares atuais: aparece um aviso a sugerir que se reduzam os limiares; a tabela e o gráfico de regras não aparecem, a tabela do passo 10 dá lugar a "Não há regras de associação para contar." e todos os itens passam a contar como itens sem regras. 11a) Todos os itens aparecem em alguma regra: em vez da lista, aparece "Todos os itens do dataset aparecem em pelo menos uma regra de associação."

Campo	Descrição
Fluxos de exceção	2a) API inacessível: aparece um erro com instruções para iniciar a API e a página termina. 3a/6a) Erro HTTP ou de ligação (incluindo timeout) depois do passo 2: aparece "Não foi possível obter as estatísticas da API. Verifique se a API está em execução e tente novamente." e a página termina, sem mostrar detalhes internos.
Classes / endpoints	`AppItensRecomendados.main`, `get_settings`, `get_rules_raw`, `format_rules_for_display`, `get_top_items`, `items_covered_by_rules`, `compute_item_rule_counts`, `compute_items_without_rules`; `GET /admin/settings` → `admin.read_settings`; `GET /admin/stats/rules` → `admin.association_rules_endpoint`; `GET /admin/stats/top-items` → `admin.top_items`; `RecommendationEngine.build_dataset`, `generate_association_rules`, `compute_top_items`

Tabela 25 – Ficha do caso de uso UC09 Configurar parâmetros do motor

Campo	Descrição
Identificador	UC09
Nome	Configurar parâmetros do motor
Ator(es)	Administrador
Descrição	O administrador altera o suporte mínimo, a confiança mínima e o peso do histórico do utilizador. Os novos valores passam a ser usados de imediato pelo UC06 e pelo UC08.
Pré-condições	A API está em execução e o backoffice está aberto.
Pós-condições	A linha única de `Settings` tem os novos valores.

Campo	Descrição
Fluxo principal	1) O administrador escolhe a página "Opções" na barra lateral. 2) `main` chama `get_settings` (`GET /admin/settings` → `admin.read_settings`). 3) O backoffice mostra três sliders com os valores atuais: "Suporte Mínimo" (0,01 a 1,0), "Confiança Mínima" (0,01 a 1,0) e "Peso do histórico do utilizador" (1 a 1000). 4) O administrador ajusta os valores e toca em "Guardar". 5) `update_settings` envia `PUT /admin/settings`. 6) A API valida `SettingsSchema`: suporte e confiança maiores do que 0 e no máximo 1, e peso entre 1 e 1000, os mesmos limites dos sliders. 7) `admin.update_settings` atualiza e grava a linha de `Settings` e devolve o código HTTP 200. 8) Aparece "Parâmetros atualizados."
Fluxos alternativos	4a) O administrador sai sem tocar em "Guardar": nada é gravado.
Fluxos de exceção	2a) API inacessível: aparece o erro de ligação e a página termina. 5a) Erro HTTP ou de ligação no PUT: aparece "Não foi possível guardar os parâmetros. Verifique se a API está em execução e tente novamente." e nada é gravado. 6a) Valores fora dos limites, só possíveis com pedidos diretos à API (por exemplo, suporte 0 ou peso 1001): 422 e os parâmetros não são alterados; este comportamento é verificado em test_admin_settings_roundtrip.
Classes / endpoints	`AppItensRecomendados.main`, `get_settings`, `update_settings`; `GET`

Campo	Descrição
	`/admin/settings` → `admin.read_settings`; `PUT /admin/settings` → `admin.update_settings`, `admin.get_or_create_settings`, `schemas.SettingsSchema`, `models.Settings`

6.3 Diagramas de robustez

Os diagramas de robustez fazem a ponte entre a descrição textual de cada caso de uso e o desenho detalhado, segundo a metodologia ICONIX (Rosenberg & Stephens, 2007). Usam três estereótipos: boundary (ecrãs e páginas com que o ator interage), control (funções e métodos que executam a lógica) e entity (dados persistidos ou mantidos em memória). Procurou-se respeitar as regras de interação do ICONIX: o ator só comunica com objetos boundary; os boundary só comunicam com control; os entity só comunicam com control. Neste projeto, os diagramas serviram sobretudo para identificar os controladores e as entidades envolvidas em cada caso de uso e para verificar a correspondência entre os passos das fichas e o código; o uso destes estereótipos não significa, por si só, que todo o processo ICONIX tenha sido seguido antes da implementação (secção 6.2.1).

Cada seta tem o número do passo correspondente na ficha do caso de uso. Os controladores do lado do servidor usam o nome real da função no router.

6.3.1 UC01 Criar conta

A Figura 14 mostra o registo, desde o `LoginScreen` até à gravação do token. A validação da confirmação acontece duas vezes, no cliente (passo 4) e na API (passo 6).

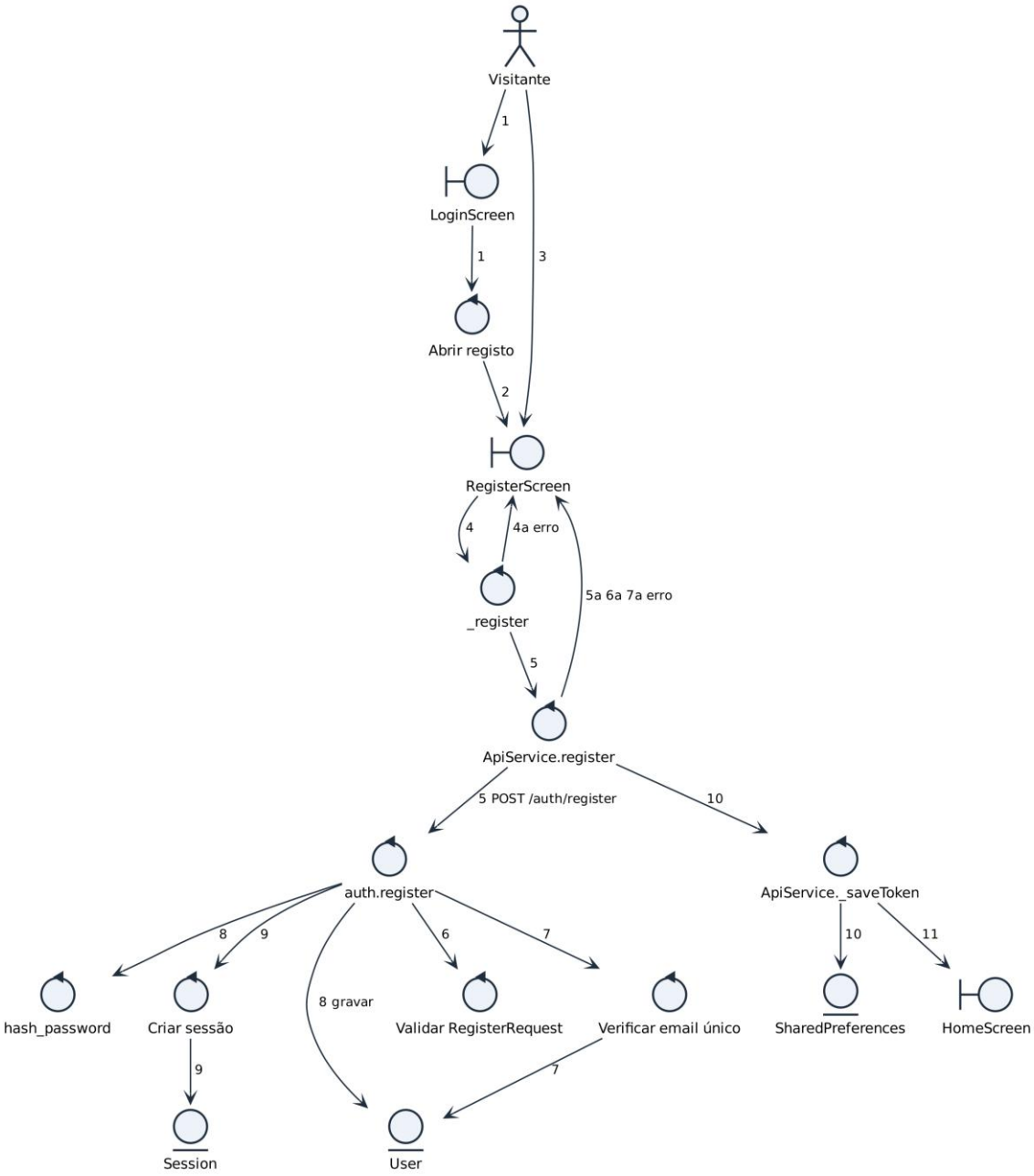


Figura 14 – Diagrama de robustez do UC01 Criar conta

6.3.2 UC02 Autenticar-se

A Figura 15 inclui a decisão do `_StartupGate`, que lê o token guardado e escolhe entre o `LoginScreen` e o HomeScreen.

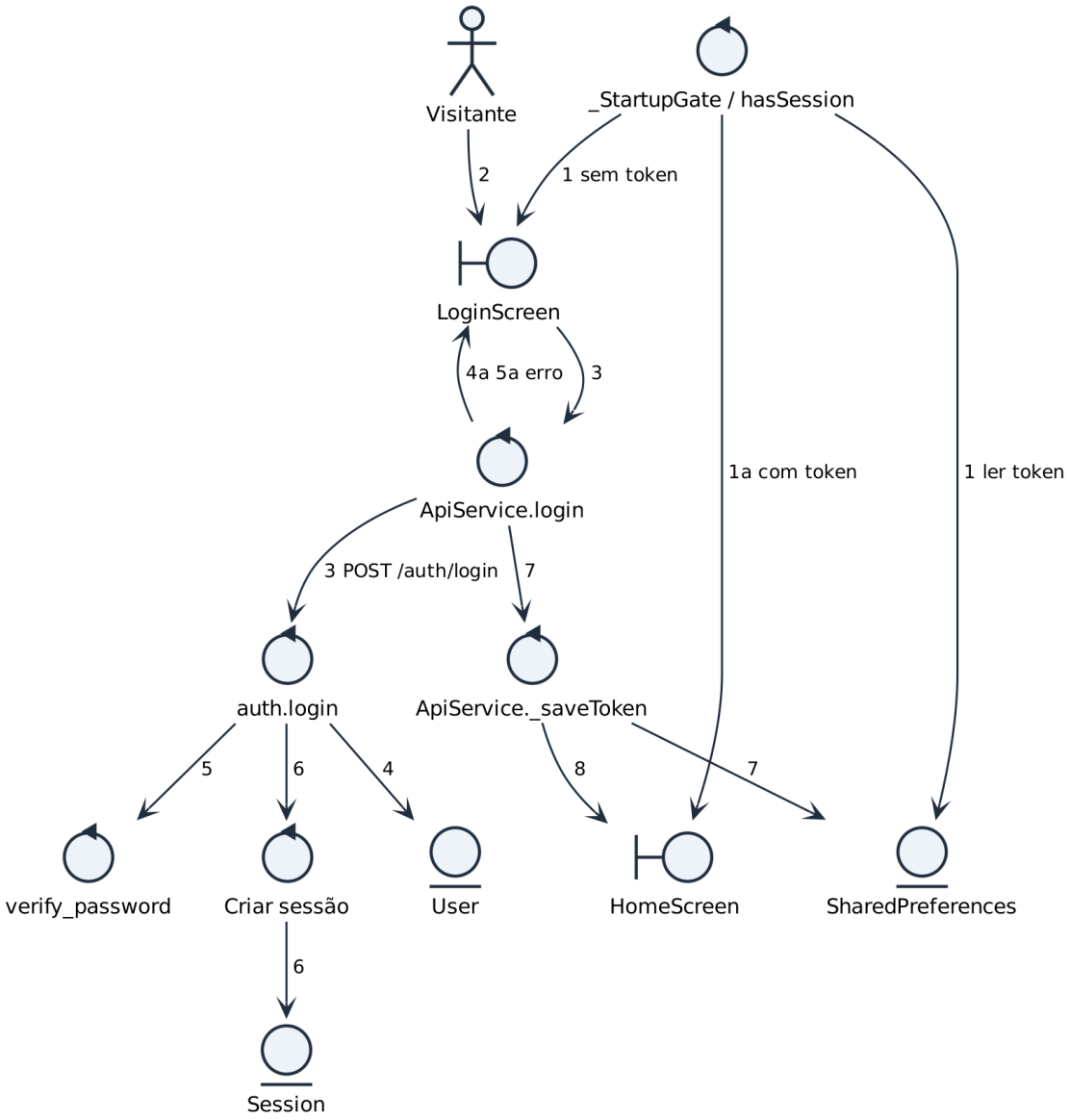


Figura 15 – Diagrama de robustez do UC02 Autenticar-se

6.3.3 UC03 Terminar sessão

A Figura 16 mostra que o token é apagado em dois sítios: na tabela `sessions` (passo 5) e no dispositivo (passo 6).

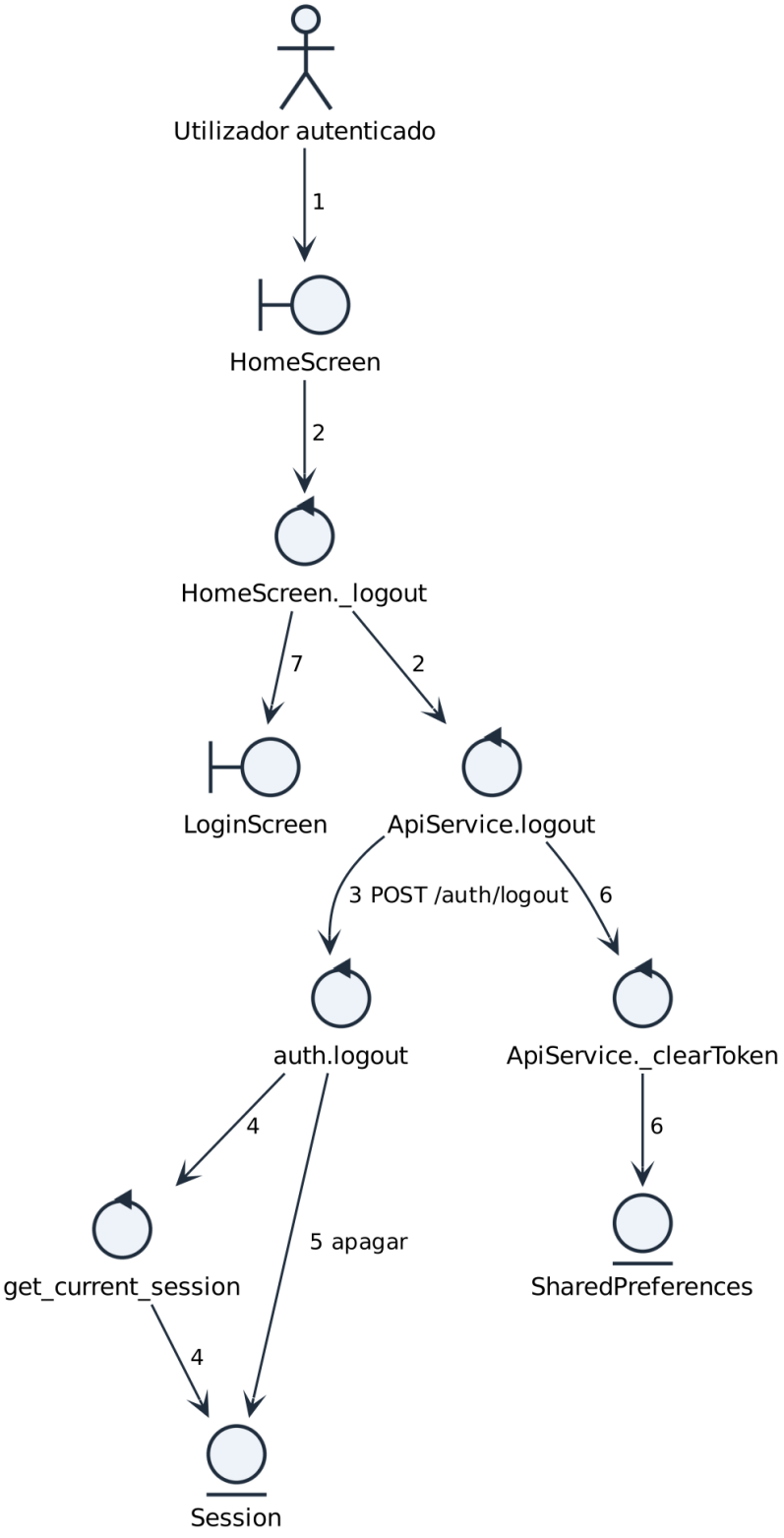


Figura 16 – Diagrama de robustez do UC03 Terminar sessão

6.3.4 UC04 Consultar histórico de listas

A Figura 17 mostra a leitura das listas do utilizador e a sua conversão em `SavedList` para apresentação. Os fluxos alternativos 8a e 8b (Tabela 20), que tratam da marcação de itens e da alteração do título, não estão representados.

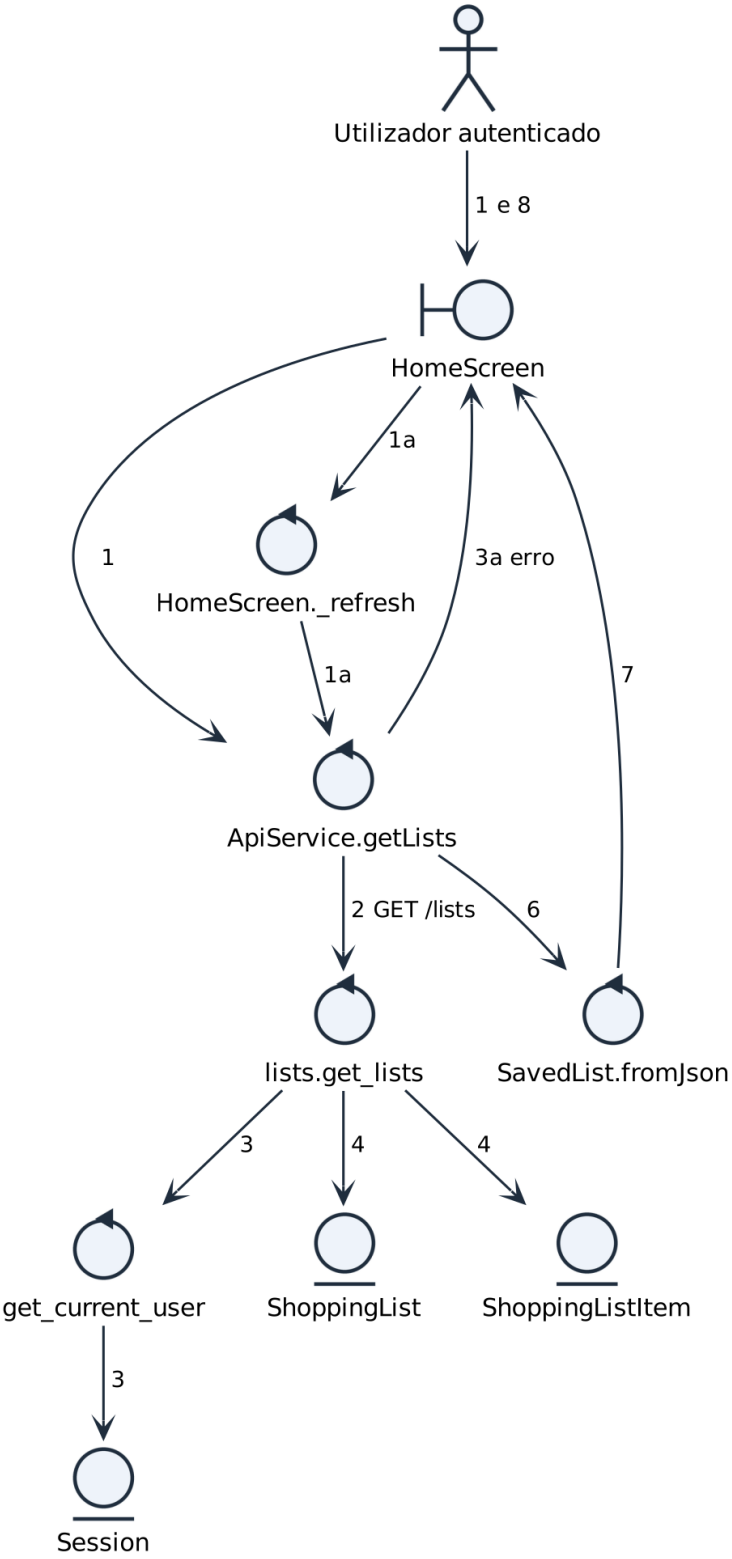


Figura 17 – Diagrama de robustez do UC04 Consultar histórico de listas

6.3.5 UC05 Construir lista de compras

Na Figura 18, a lista em construção é um objeto entity em memória (`_currentItems`); só passa a ser persistida no UC07. O controlador `_refreshRecommendations` corresponde ao UC06, que estende o UC05 quando a lista tem pelo menos um item.

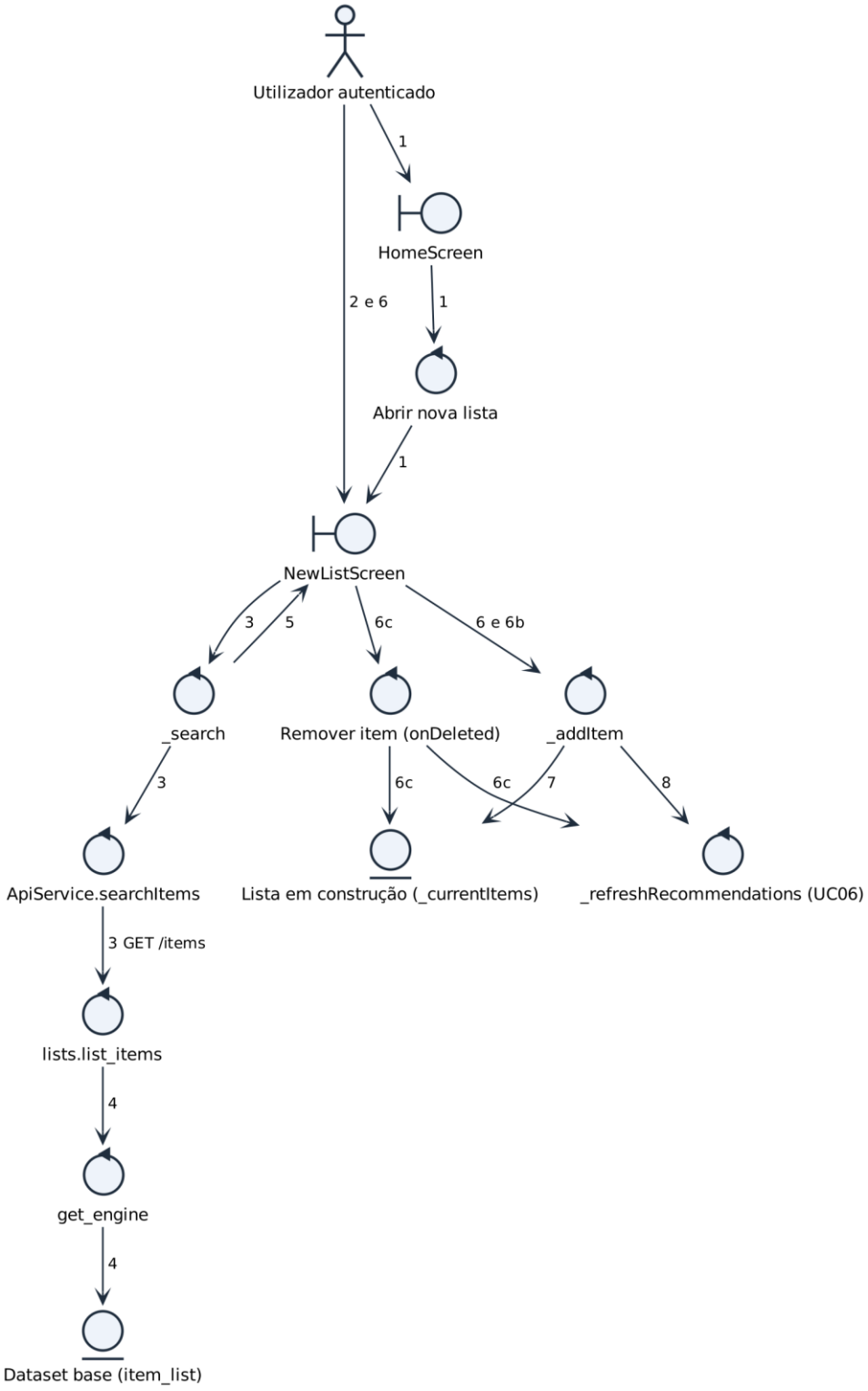


Figura 18 – Diagrama de robustez do UC05 Construir lista de compras

6.3.6 UC06 Consultar itens recomendados

A Figura 19 representa o fluxo de cálculo das recomendações: o pedido atravessa a autenticação, a leitura de parâmetros, a construção do conjunto de dados ponderado e o Apriori.

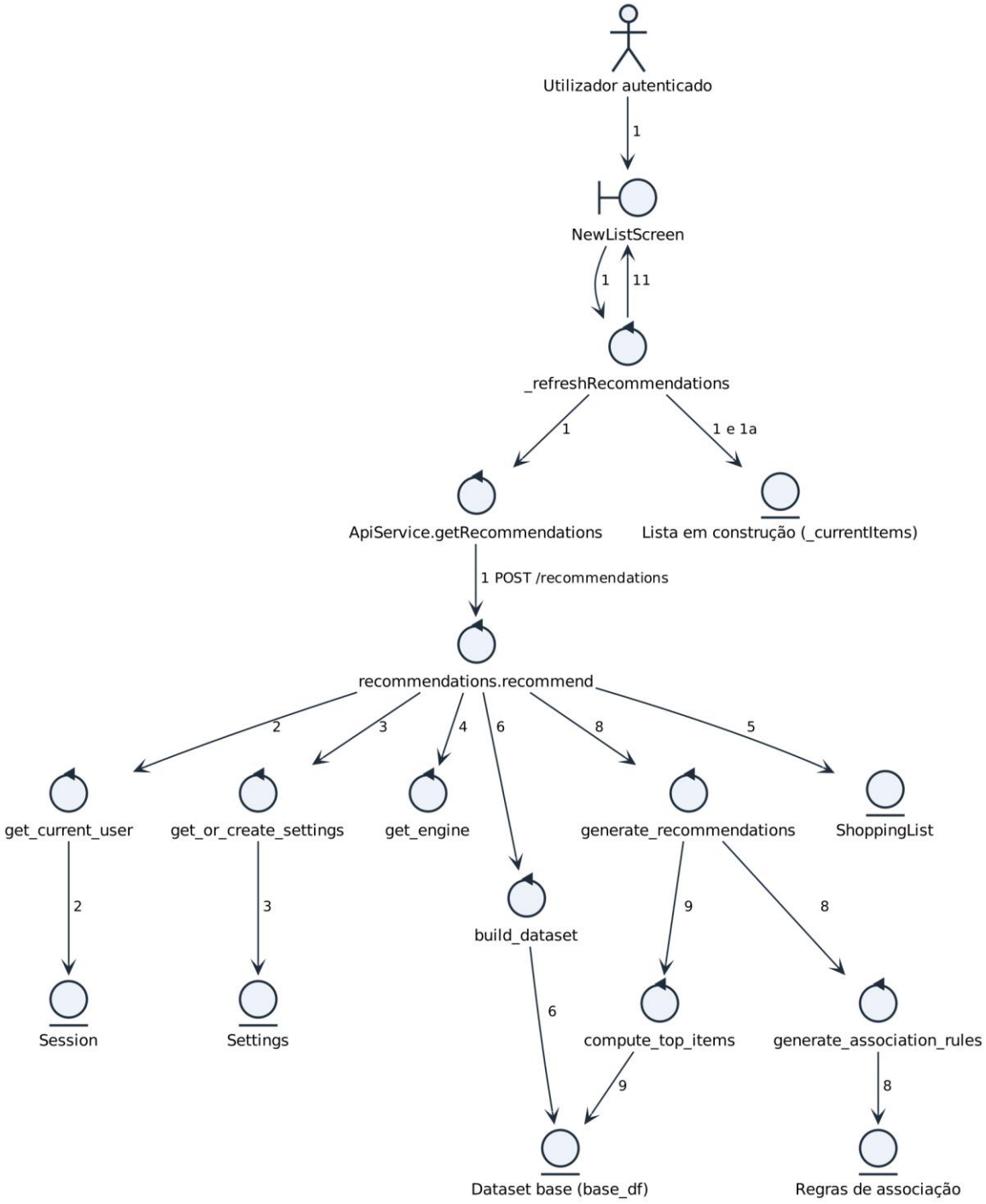


Figura 19 – Diagrama de robustez do UC06 Consultar itens recomendados

6.3.7 UC07 Guardar lista de compras

A Figura 20 mostra a passagem da lista em memória para as entidades persistidas e o regresso ao histórico.

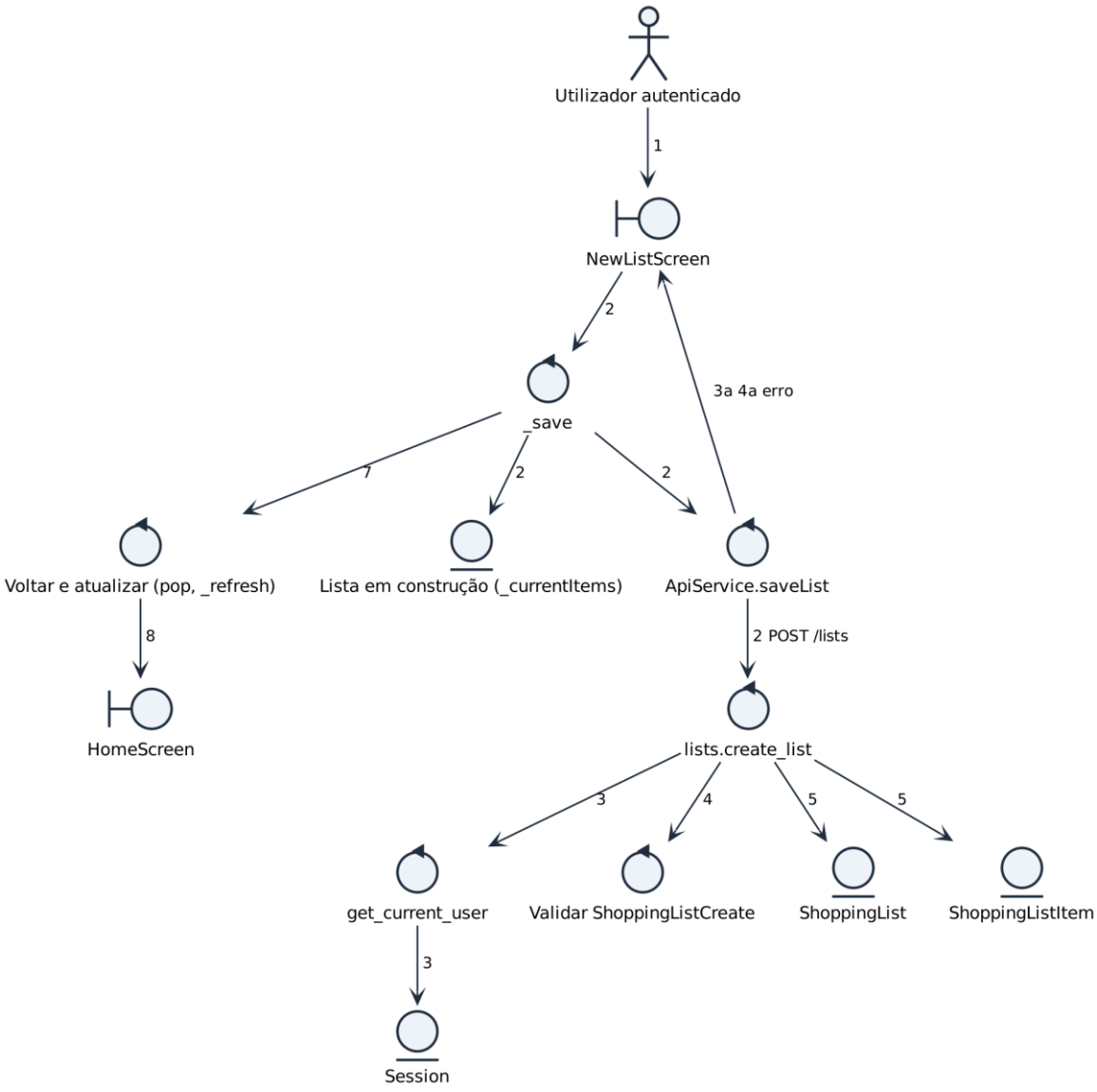


Figura 20 – Diagrama de robustez do UC07 Guardar lista de compras

6.3.8 UC08 Consultar estatísticas

Na Figura 21, o boundary é a página "Estatísticas" do Streamlit. Os controladores do backoffice (`get_settings`, `get_rules_raw` e `get_top_items`) chamam os controladores da API; o conjunto de dados usado é sempre o base, sem histórico de utilizadores. No diagrama, `get_rules_raw` aparece com o nome anterior, `get_rules`, e não estão representadas as contagens de itens com e sem regras (passos 8, 10 e 11 da Tabela 24), que o backoffice calcula localmente, sem novas chamadas à API.

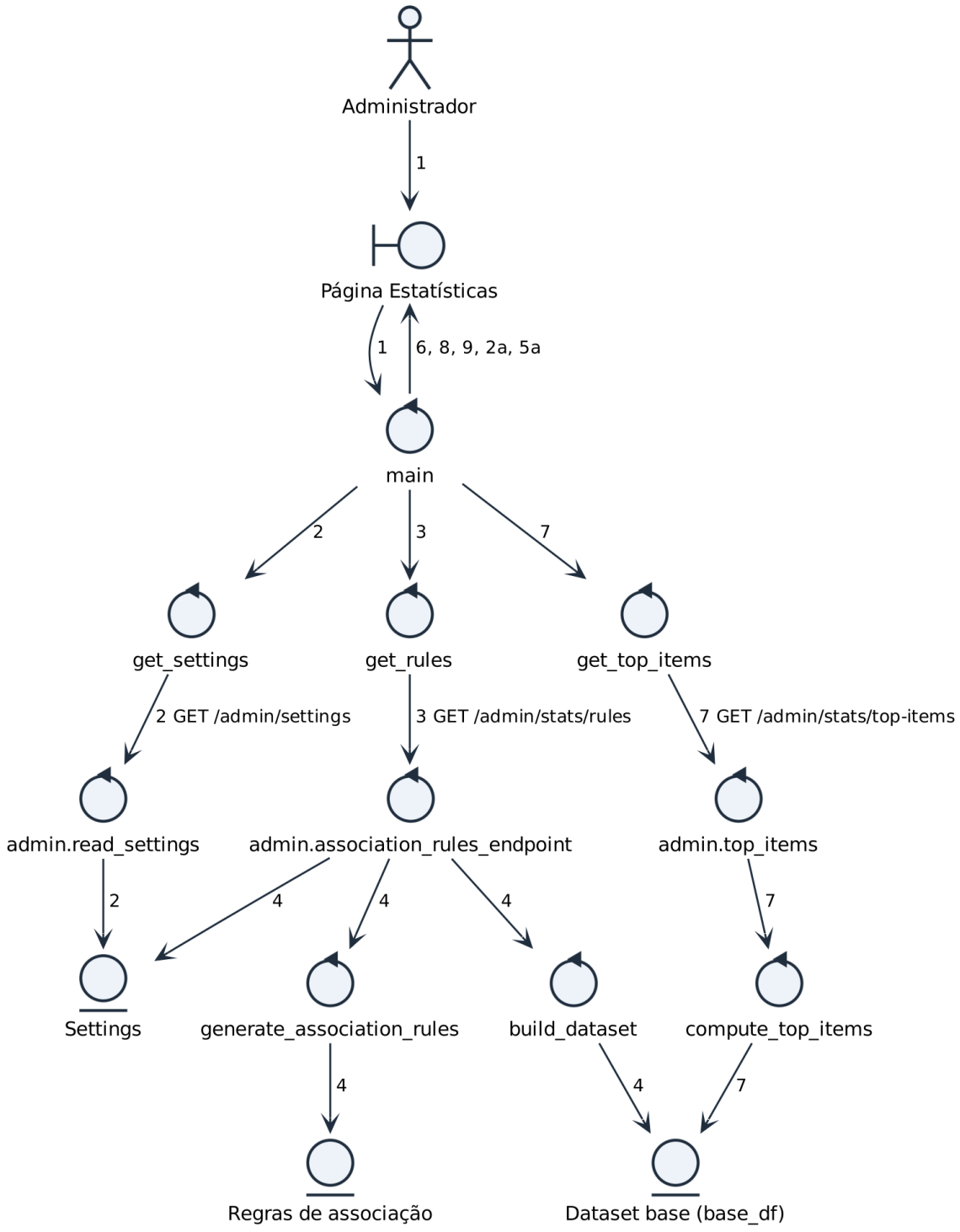


Figura 21 – Diagrama de robustez do UC08 Consultar estatísticas

6.3.9 UC09 Configurar parâmetros do motor

A Figura 22 mostra a leitura e a escrita da linha única de `Settings`, partilhada pelo backoffice e pela aplicação móvel.

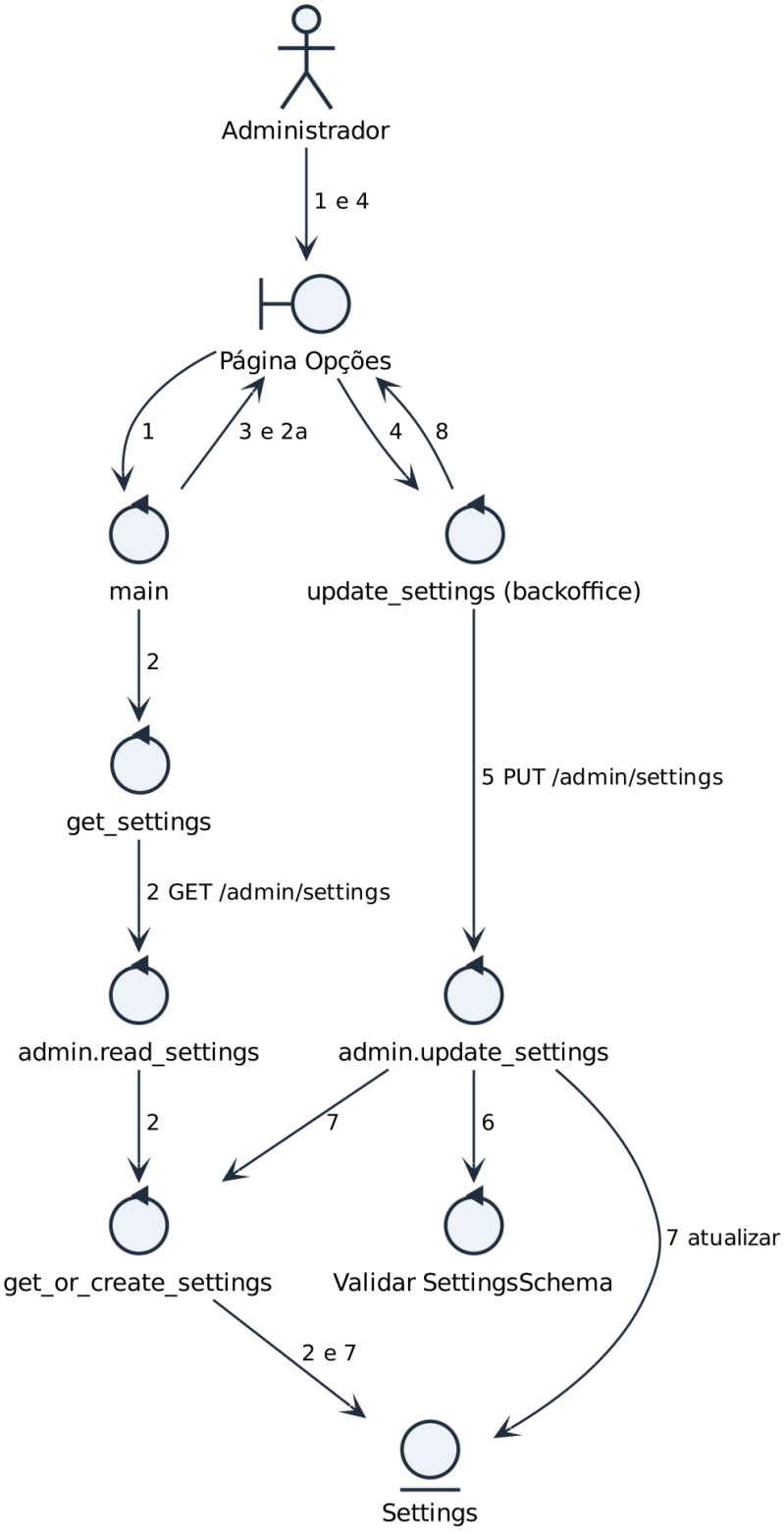


Figura 22 – Diagrama de robustez do UC09 Configurar parâmetros do motor

6.4 Diagramas de sequência

Os diagramas de sequência detalham, para cada caso de uso, a ordem das mensagens entre os objetos, usando os nomes reais das classes, funções e métodos do código. Os participantes mantêm os estereótipos dos diagramas de robustez: boundary para ecrãs e páginas, control para serviços e funções, entity para modelos e database para os armazenamentos (`app.db` e `SharedPreferences`). Cada mensagem começa pelo número do passo da ficha; os fluxos alternativos e de exceção aparecem em blocos `alt` e `opt`. As Figuras 23 a 31 apresentam os nove diagramas.

6.4.1 UC01 Criar conta

A Figura 23 detalha o registo e a emissão do primeiro token de sessão.

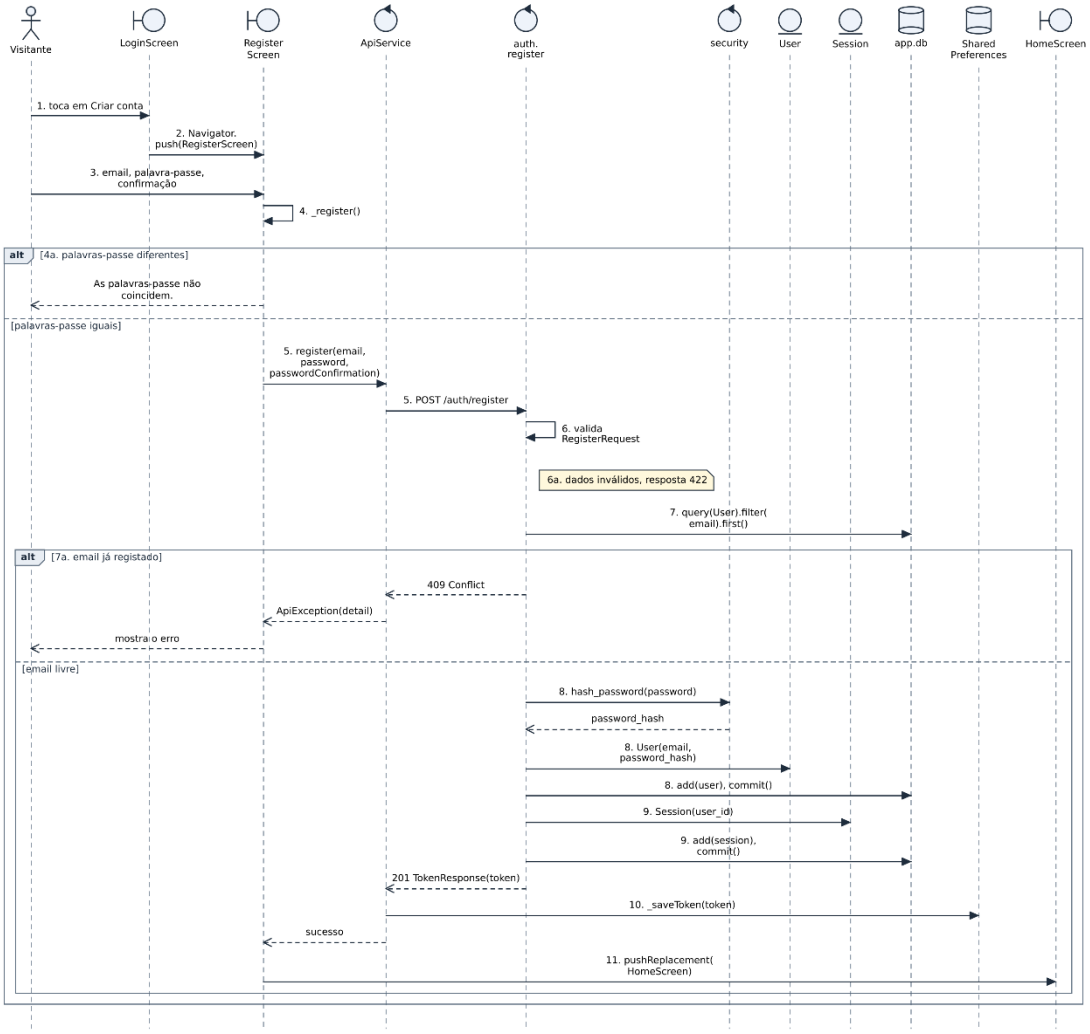


Figura 23 – Diagrama de sequência do UC01 Criar conta

6.4.2 UC02 Autenticar-se

A Figura 24 começa no arranque da aplicação, onde o `_StartupGate` decide o ecrã inicial.

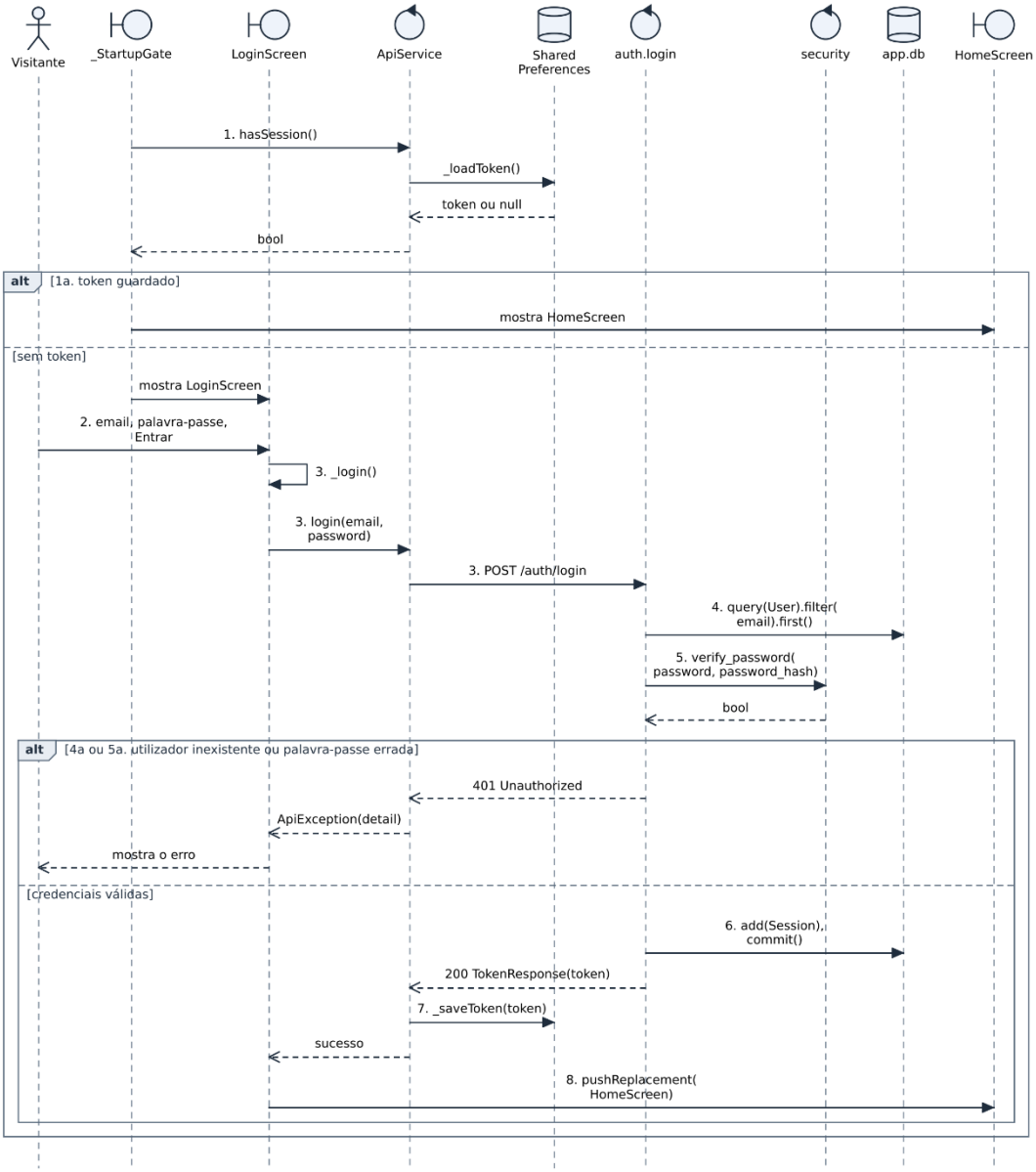


Figura 24 – Diagrama de sequência do UC02 Autenticar-se

6.4.3 UC03 Terminar sessão

A Figura 25 mostra que o token local é apagado em qualquer caso, mesmo que o pedido à API falhe.

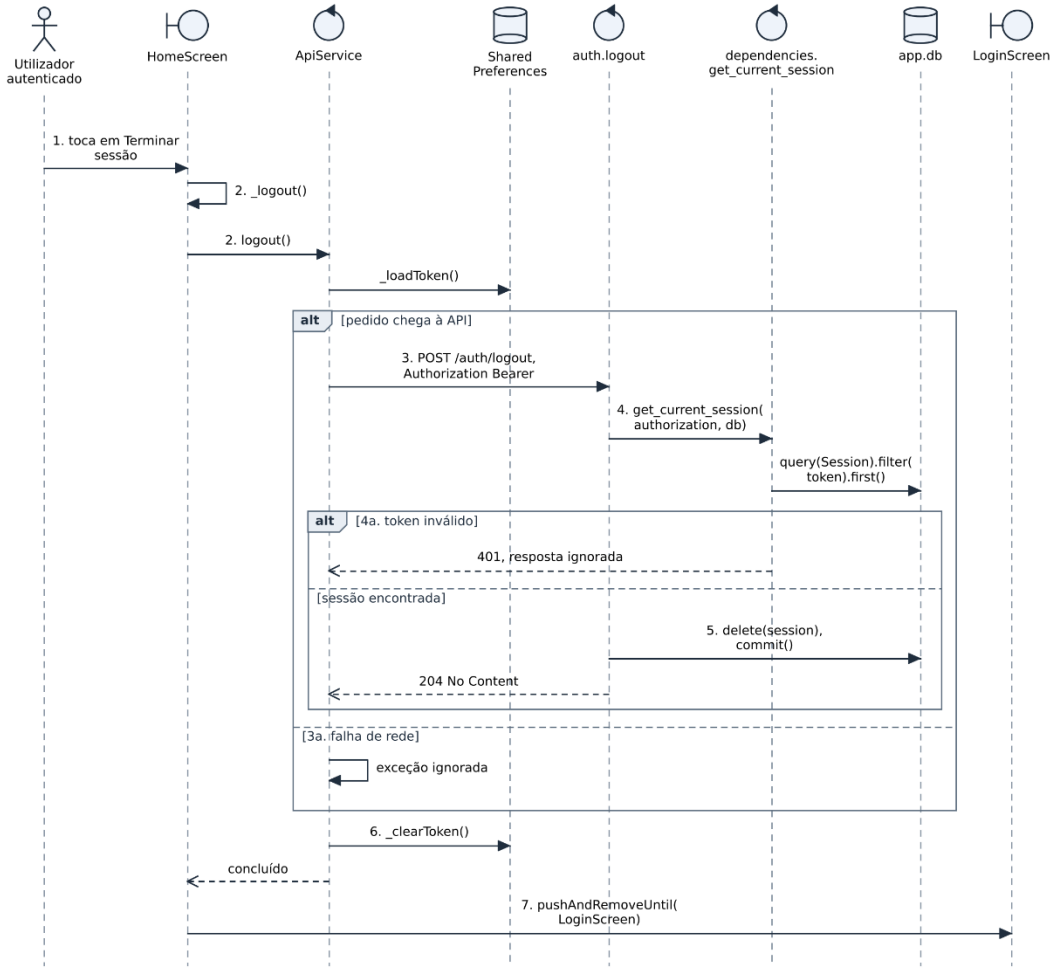
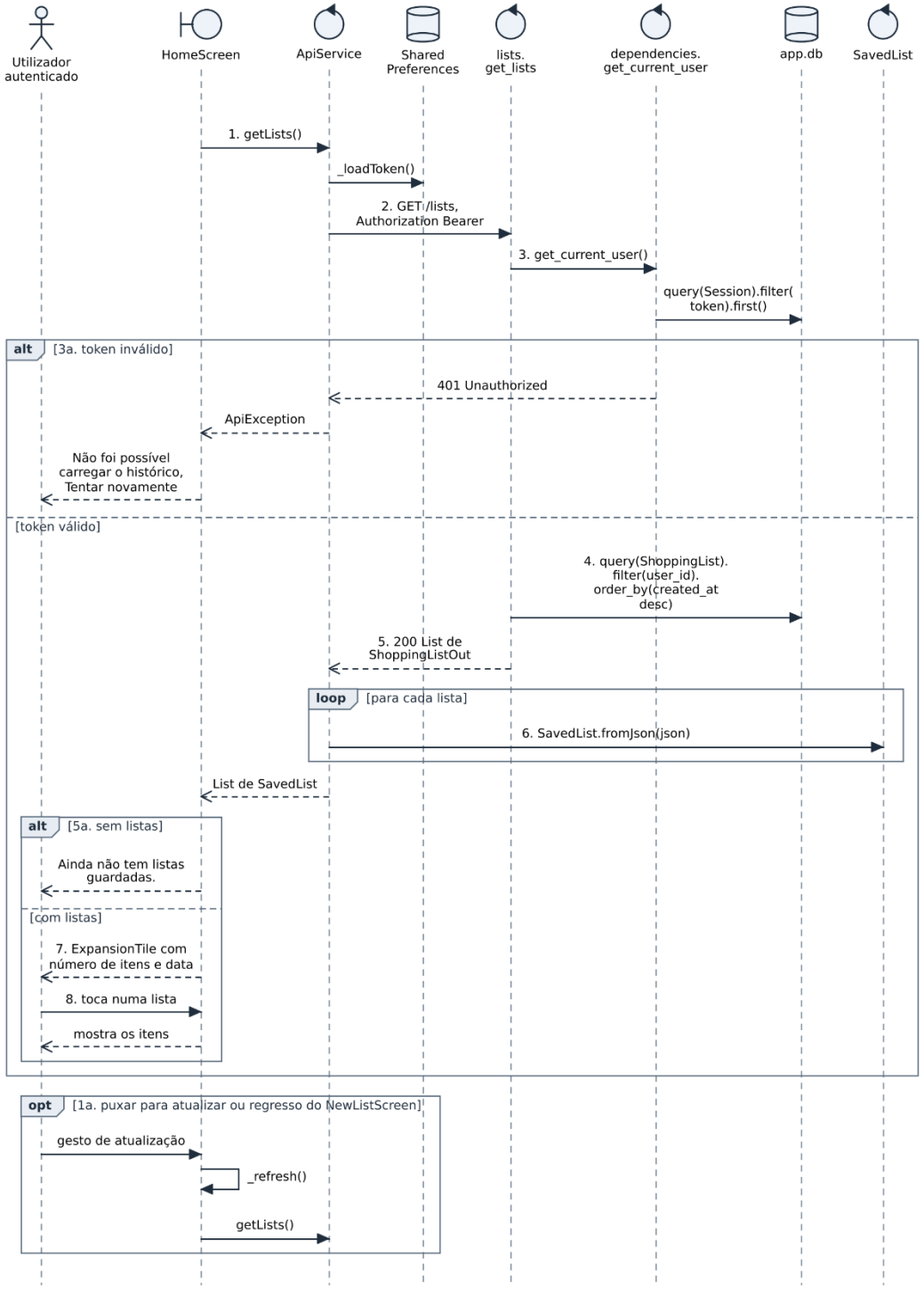


Figura 25 – Diagrama de sequência do UC03 Terminar sessão

6.4.4 UC04 Consultar histórico de listas

A Figura 26 mostra o carregamento do histórico no `initState` do `HomeScreen` e a atualização manual; tal como no diagrama de robustez, os fluxos alternativos 8a e 8b



não estão representados.

Figura 26 – Diagrama de sequência do UC04 Consultar histórico de listas

6.4.5 UC05 Construir lista de compras

A Figura 27 mostra a pesquisa de itens, que é feita a cada alteração do campo de texto, e a atualização da lista em memória.

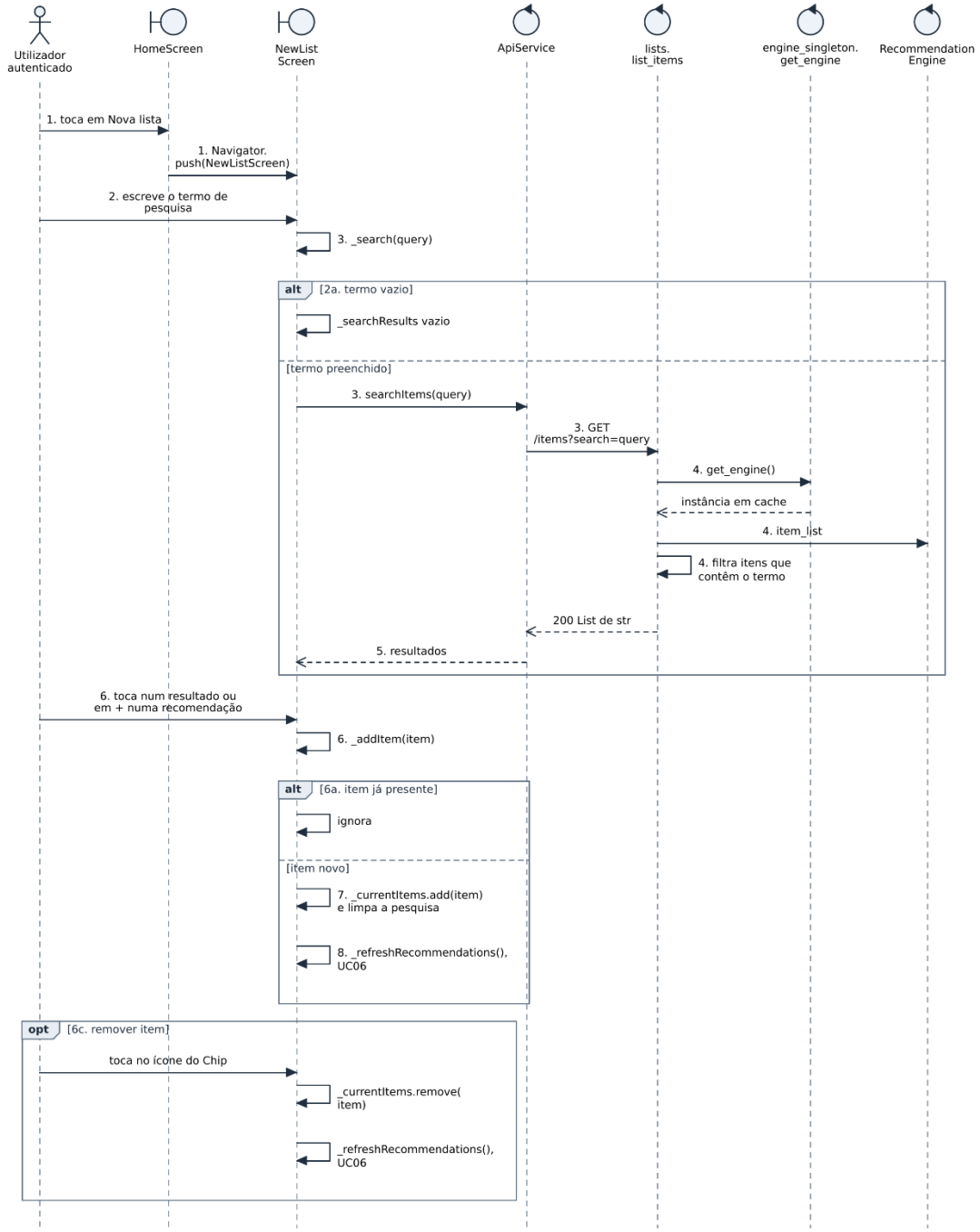


Figura 27 – Diagrama de sequência do UC05 Construir lista de compras

6.4.6 UC06 Consultar itens recomendados

A Figura 28 detalha o cálculo das recomendações no servidor, incluindo a construção do conjunto de dados ponderado e as chamadas ao mlxtend.

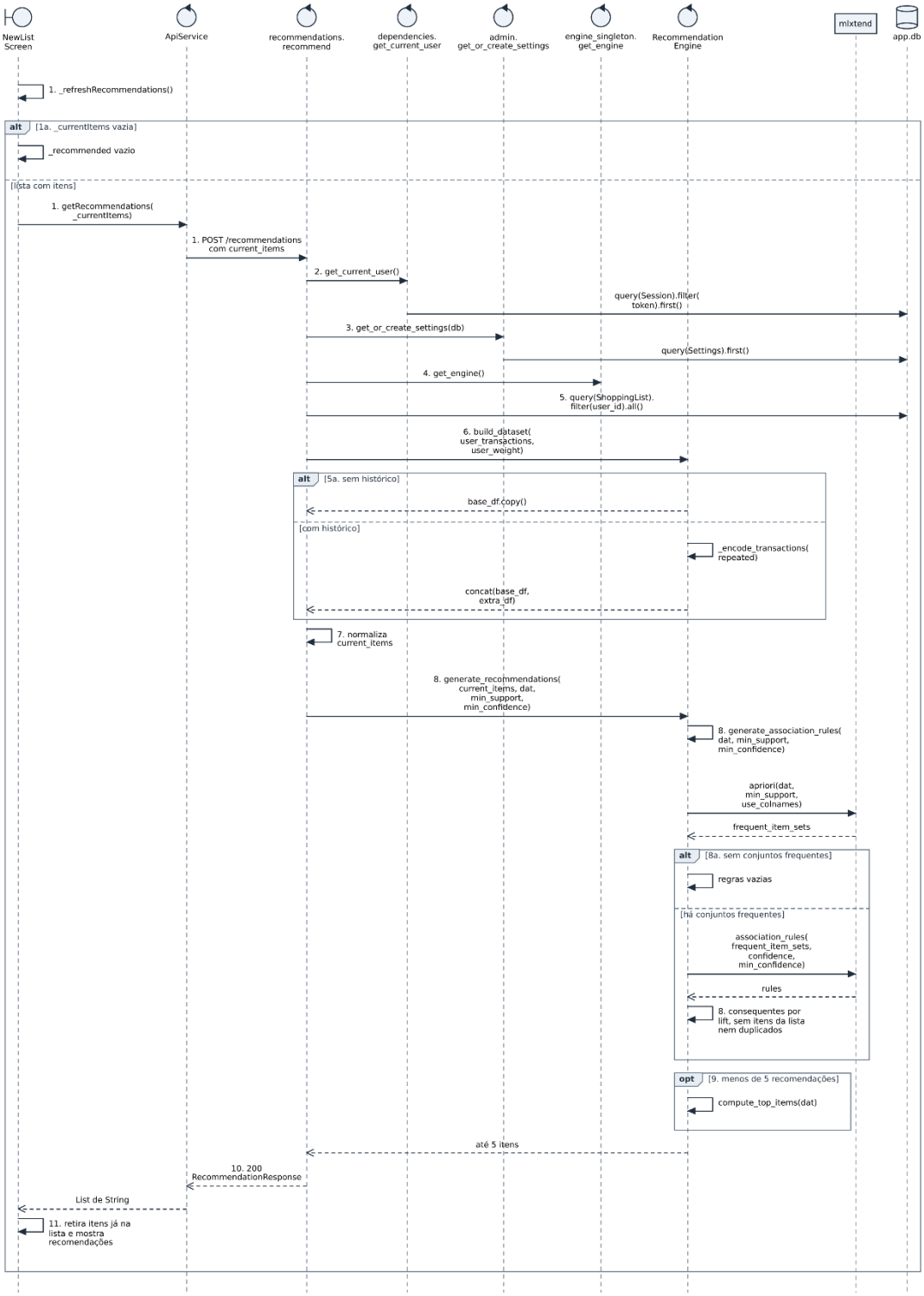


Figura 28 – Diagrama de sequência do UC06 Consultar itens recomendados

6.4.7 UC07 Guardar lista de compras

A Figura 29 mostra a gravação da lista e o regresso ao histórico atualizado.

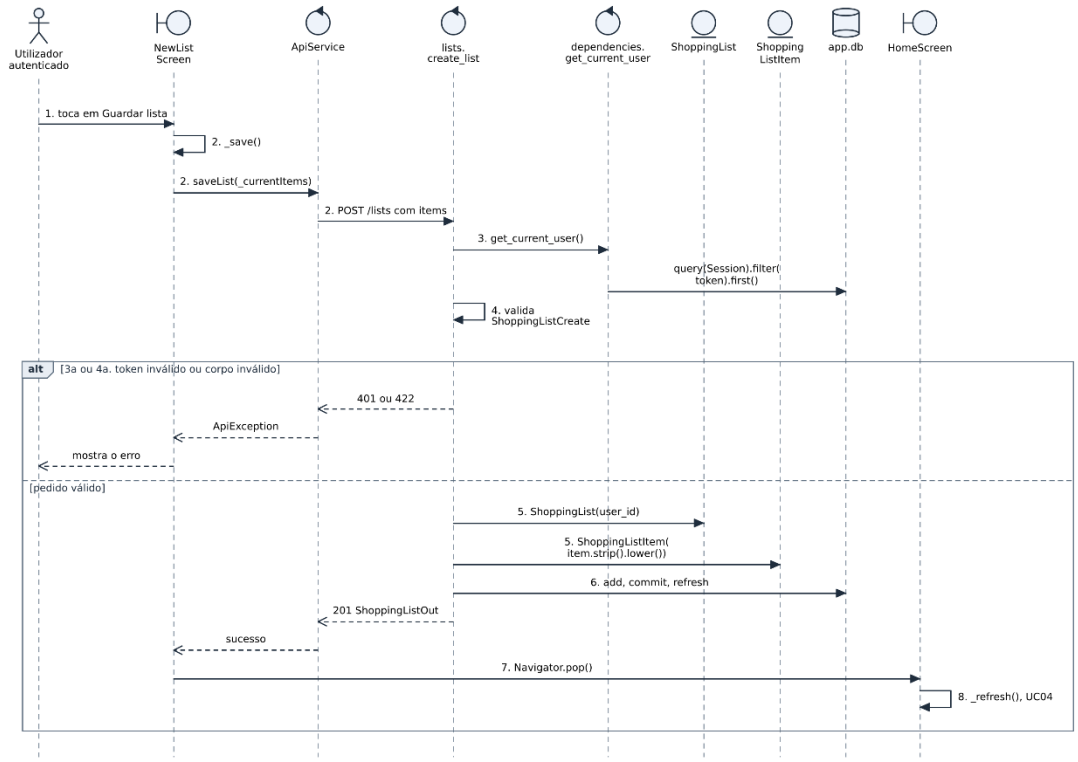
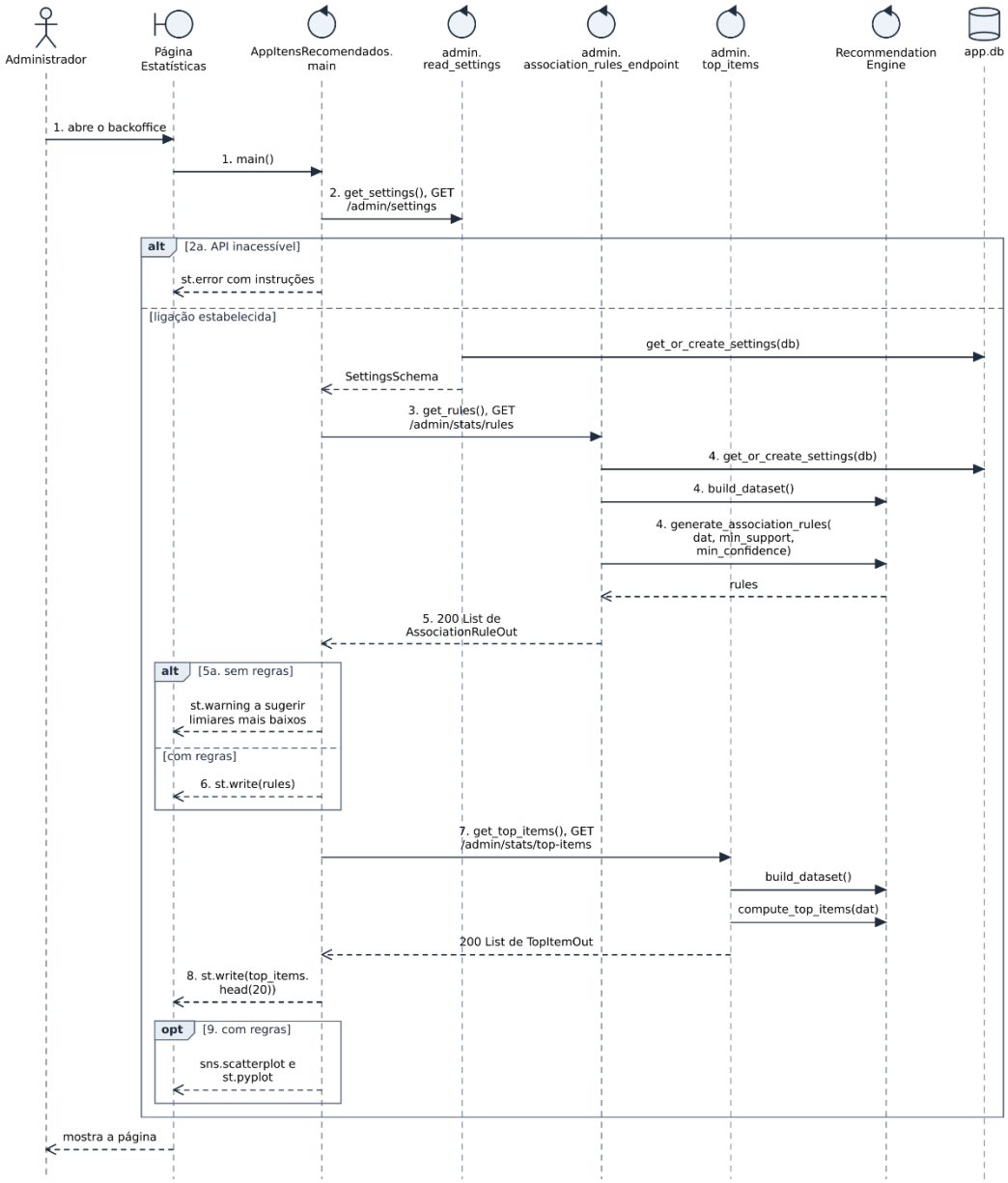


Figura 29 – Diagrama de sequência do UC07 Guardar lista de compras

6.4.8 UC08 Consultar estatísticas

A Figura 30 mostra as três chamadas do backoffice à API e a construção da página "Estatísticas". Tal como na Figura 21, `get_rules_raw` aparece com o nome anterior,



`get_rules`, e as contagens locais dos passos 8, 10 e 11 não estão representadas.

Figura 30 – Diagrama de sequência do UC08 Consultar estatísticas

6.4.9 UC09 Configurar parâmetros do motor

A Figura 31 mostra a leitura dos valores atuais, a edição nos sliders e a gravação.

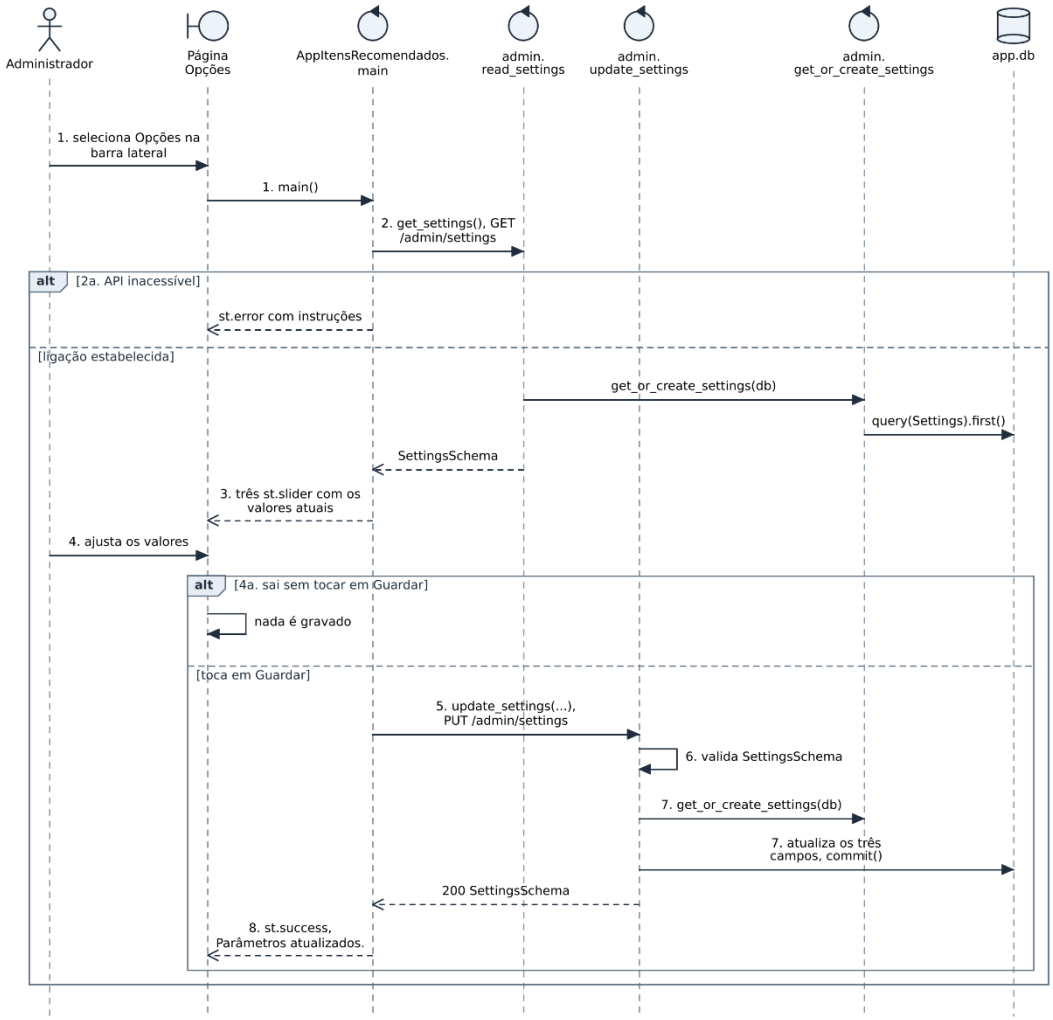


Figura 31 – Diagrama de sequência do UC09 Configurar parâmetros do motor

6.5 Testes e avaliação

6.5.1 Testes automáticos da API

A API foi testada com um conjunto de testes automáticos escritos com pytest (pytest-dev, s.d.), recorrendo ao TestClient do FastAPI (Ramírez, s.d.). Cada teste é executado de forma isolada: é criada uma base de dados SQLite temporária, que substitui a base de dados da aplicação através do mecanismo de injeção de dependências (app.dependency_overrides), e o motor de recomendação é instanciado sobre um pequeno ficheiro CSV de teste, para que os resultados não dependam do conjunto de dados real.

A Tabela 26 descreve os 14 testes implementados e os casos de uso a que se referem. Os testes do motor usam resultados esperados definidos manualmente: o teste test_recommendations_require_complete_antecedent fornece ao motor duas regras conhecidas, {milk, bread} ⇒ butter (elevação 3,0) e {milk} ⇒ eggs (elevação 1,5), e verifica que, só com milk na lista, butter não é recomendado e que, com milk e bread, butter surge antes de eggs.

Tabela 26 – Testes automáticos da API

Teste	O que verifica	UC
test_register_login_logout_flow	Registo, autenticação e terminação de sessão; após o logout, o token deixa de ser aceite.	UC01–UC03
test_login_with_wrong_password_is_rejected	Rejeição da autenticação com palavra-passe errada.	UC02
test_register_rejects_mismatched_passwords	Rejeição do registo quando as palavras-passe não coincidem.	UC01
test_register_rejects_duplicate_email	Rejeição do registo com um email já registado.	UC01
test_protected_routes_require_token	Recusa de acesso a rotas protegidas sem token ou com token inválido.	UC04
test_items_endpoint_lists_and_searches_dataset_items	Listagem e pesquisa dos itens do conjunto de dados.	UC05
test_save_list_and_history	Gravação de uma lista e a sua presença no histórico.	UC04, UC07

Teste	O que verifica	UC
test_recommendations_exclude_current_items_and_respect_limit	As recomendações não incluem itens já escolhidos e respeitam o limite máximo.	UC06
test_admin_settings_roundtrip	Leitura e alteração dos parâmetros do motor.	UC09
test_admin_stats_top_items_and_rules	Estatísticas dos itens mais frequentes e das regras de associação.	UC08
test_list_title_create_and_rename	Criação de listas com e sem título, alteração do título e proteção contra alterações por outro utilizador.	UC04, UC07
test_register_password_policy_and_email_validation	Limites do registo: palavra-passe com 7 caracteres rejeitada (422), com 8 aceite (201), e email com formato inválido rejeitado (422).	UC01
test_recommendations_require_complete_antecedent	Uma regra só é aplicada se todo o antecedente estiver na lista; ordenação pela elevação.	UC06
test_save_list_rejects_blank_items_and_removes_duplicates	Rejeição de uma lista só com itens vazios (422) e remoção de itens duplicados após a normalização.	UC07

A execução dos testes (`python -m pytest -v`) terminou com sucesso nos 14 testes, em 3,85 segundos, num computador com Windows 10, Python 3.9.5, pytest 8.4.2 e as versões da Tabela 28, sobre a versão do código entregue. Este tempo corresponde à execução de todo o conjunto de testes e não ao tempo de resposta do motor. Os testes automáticos cobrem apenas a API; a aplicação móvel e o backoffice não foram objeto de testes automáticos, tendo sido verificados manualmente através do percurso apresentado na secção 6.7. Os testes também não cobrem o efeito do peso do histórico, os empates na elevação nem o preenchimento com os itens mais frequentes, e não avaliam a qualidade das recomendações, que é objeto da secção 6.5.2.

6.5.2 Avaliação offline das recomendações

Para avaliar a qualidade das recomendações (quinto objetivo, secção 1.2), realizou-se uma avaliação offline com o script avaliacao_offline.py, incluído na pasta api_backend, que usa o próprio motor da API. As 9 835 transações do Dataset 1 foram divididas aleatoriamente em treino (80%) e teste (20%), em cinco divisões diferentes, e as regras foram extraídas apenas com o treino. Em cada cesto de teste com pelo

menos dois itens (1 542 por divisão, em média), escondeu-se um item escolhido ao acaso e pediram-se ao motor cinco recomendações para os restantes. Mediu-se a taxa de acerto nas cinco recomendações (HR@5), isto é, a proporção de casos em que o item escondido está entre as recomendações; com um único item escondido, esta medida coincide com a revocação nas cinco primeiras recomendações, sendo a precisão correspondente igual a HR@5/5 (Ricci, Rokach & Shapira, 2022). Como termo de comparação, usou-se uma recomendação baseada apenas na popularidade: os cinco itens mais frequentes do treino que ainda não estão no cesto. A Tabela 27 apresenta a média e o desvio-padrão nas cinco divisões, para os parâmetros por omissão e para três combinações de limiares mais baixos.

Tabela 27 – Resultados da avaliação offline das recomendações (Dataset 1, 80/20, cinco divisões)

Suporte mín.	Confiança mín.	Regras	HR@5 do motor	HR@5 da popularidade	Itens distintos recomendados
0,01	0,2	243	0,265 ± 0,009	0,265 ± 0,010	13,8
0,005	0,2	929	0,257 ± 0,008	0,265 ± 0,010	18,0
0,005	0,1	1 804	0,223 ± 0,012	0,265 ± 0,010	39,2
0,003	0,1	4 755	0,222 ± 0,013	0,265 ± 0,010	48,2

O número de regras é a média nas cinco divisões, extraídas com 80% das transações; por isso difere das 234 regras obtidas com o conjunto completo (secção 5.5).

Com os parâmetros por omissão, o motor acertou no item escondido em 26,5% dos casos, o mesmo valor obtido pela recomendação por popularidade; a diferença está dentro da variação entre divisões, de cerca de um ponto percentual. Este resultado explica-se pela estrutura das regras: com um suporte mínimo de 0,01 e uma confiança mínima de 0,2, as 234 regras do Dataset 1 têm apenas nove consequentes distintos, e whole milk e other vegetables são o consequente de 137 delas. Por isso, ao longo de todos os casos de teste, o motor recomendou apenas cerca de 14 dos 169 itens do catálogo, praticamente os mesmos que a recomendação por popularidade. Ao baixar os limiares, o número de regras e a variedade das recomendações aumentam (até 48 itens distintos), mas a taxa de acerto desce para cerca de 22%: as regras adicionais, menos frequentes, ficam à frente na ordenação pela elevação e substituem os itens populares, que são os que mais vezes estão nos cestos.

Nesta avaliação, as regras de associação não melhoraram, face à popularidade, a capacidade de recuperar o item em falta, embora permitam recomendações mais variadas com limiares mais baixos. A avaliação tem limitações: mede a capacidade de recuperar um item de compras passadas, que é apenas uma aproximação da utilidade percebida pelo utilizador; usa o Dataset 1, cujos itens são categorias agregadas; e não

inclui o histórico dos utilizadores, porque o conjunto de dados não identifica os clientes. A avaliação com utilizadores reais continua por realizar (Capítulo 7).

6.6 Instalação e execução

Esta secção descreve os passos necessários para reproduzir o sistema numa máquina de desenvolvimento. As três componentes executam-se de forma independente, mas a API tem de estar em execução antes de se iniciar o *backoffice* ou a aplicação móvel.

Pré-requisitos

- Python 3.9.5 (a versão utilizada no desenvolvimento e nos testes), com o gestor de pacotes pip; as versões das bibliotecas estão na Tabela 28 e, de forma completa, no ficheiro api_backend/requirements-lock.txt;
- Flutter SDK (canal stable, versão 3.44 ou superior, com Dart 3.12, conforme o ficheiro pubspec.lock) e Android Studio, com um emulador Android configurado;
- o ficheiro dataSort.csv na pasta Codigo, ao lado da pasta api_backend e do ficheiro AppItensRecomendados.py.

A Tabela 28 indica as versões utilizadas no desenvolvimento e nos testes do sistema.

Tabela 28 – Versões utilizadas no desenvolvimento e nos testes do sistema

Ferramenta	Versão	Utilização
Python	3.9.5	API, backoffice e testes
FastAPI	0.128.8	API REST
Uvicorn	0.39.0	Servidor da API
Pydantic	2.13.5	Validação dos dados
SQLAlchemy	2.0.54	Acesso à base de dados SQLite
bcrypt	4.3.0	Hash das palavras-passe
pandas	2.2.3	Leitura e tratamento dos dados
NumPy	2.0.2	Matriz transação × item
mlxtend	0.23.4	Apriori e regras de associação
Streamlit	1.50.0	Backoffice
Requests	2.32.5	Pedidos do backoffice à API
Matplotlib / Seaborn	3.9.4 / 0.13.2	Gráfico do backoffice
pytest	8.4.2	Testes automáticos
Flutter / Dart	≥ 3.44 / ≥ 3.12	Aplicação móvel
http / shared_preferences	1.6.0 / 2.5.5	Bibliotecas da aplicação móvel

1. API

```
cd Codigo/api_backend
python -m venv .venv
.venv\Scripts\activate        (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
uvicorn app.main:app --reload
```

A API fica disponível em http://localhost:8000, com documentação interativa em http://localhost:8000/docs. No primeiro arranque é criado o ficheiro app.db, com as tabelas; a linha de parâmetros é criada com os valores por omissão (suporte mínimo 0,01, confiança mínima 0,2 e peso do histórico do utilizador 1000) na primeira vez que os parâmetros são lidos, através de get_or_create_settings (secção 6.1.2). A opção --reload, que reinicia a API sempre que o código muda, destina-se apenas ao desenvolvimento. Em alternativa, o script run.py, na pasta api_backend, cria o ambiente virtual, instala as dependências e arranca a API (python run.py), ou executa os testes (python run.py test).

2. Backoffice

As bibliotecas do *backoffice* (Streamlit, Matplotlib e Seaborn) estão incluídas no ficheiro requirements.txt, pelo que basta, noutro terminal e com o mesmo ambiente virtual ativo, executar:

```
cd Codigo
streamlit run AppItensRecomendados.py
```

Por omissão, o *backoffice* comunica com a API em http://localhost:8000. Para utilizar outro endereço, define-se a variável de ambiente API_BASE_URL antes de executar o Streamlit.

3. Aplicação móvel

```
cd Codigo/mercado_app/mercado_app
flutter pub get
flutter run
```

A aplicação deve ser executada com o emulador Android aberto. O endereço da API está definido em lib/services/api_service.dart como http://10.0.2.2:8000, que corresponde à máquina anfitriã vista a partir do emulador. A aplicação foi executada e verificada apenas no emulador. Num dispositivo físico, não basta substituir esse endereço pelo endereço IP da máquina na rede local: a API tem de escutar em todas as interfaces (uvicorn app.main:app --host 0.0.0.0), o dispositivo e o computador têm de estar na mesma rede, a firewall tem de permitir a porta 8000 e, como a comunicação é feita por HTTP, a aplicação Android pode ter de ser configurada para permitir tráfego não cifrado. Este procedimento não foi testado.

4. Testes

```
cd Codigo/api_backend
pip install pytest httpx
python -m pytest -v
```

6.7 Interfaces implementadas

Esta secção apresenta as interfaces das duas aplicações cliente, tal como foram implementadas, e a documentação interativa da API. As capturas de ecrã foram obtidas com a API em execução sobre o Dataset 1 e com os parâmetros por omissão do motor (suporte mínimo 0,01, confiança mínima 0,2 e peso do histórico do utilizador 1000).

6.7.1 Aplicação móvel

A Figura 32 mostra os ecrãs de autenticação e de criação de conta, que seguem de perto os mockups da Figura 12, e o histórico de listas (UC04). Em relação aos mockups, o histórico acrescenta duas funcionalidades: cada lista pode ter um título, dado ao guardá-la ou mais tarde, através do botão "Dar título" (ou "Mudar título"), e os itens de cada lista podem ser marcados à medida que são comprados. As marcações ficam guardadas apenas no dispositivo.

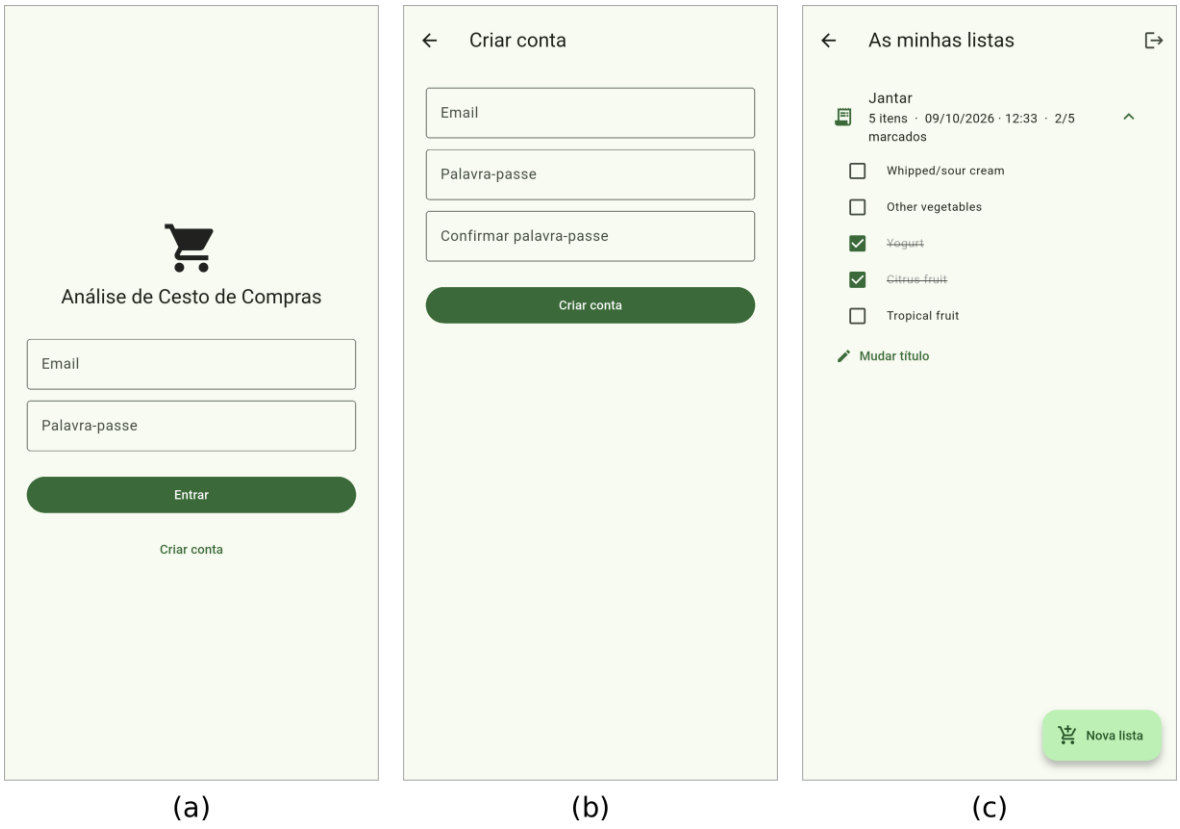


Figura 32 – Aplicação móvel: (a) autenticação, (b) criação de conta, (c) histórico de listas

A Figura 33 mostra a construção de uma nova lista (UC05 a UC07). Sem itens, a área de recomendações está vazia e o botão "Guardar lista" está desativado; depois de adicionados cinco itens (whipped/sour cream, root vegetables, rolls/buns, other vegetables e whole milk), a aplicação mostra cinco recomendações calculadas pela API, que podem ser acrescentadas à lista com um toque. A captura foi obtida com a versão final do motor e os parâmetros por omissão, para um utilizador com listas já guardadas; como estas entram no cálculo com peso 1000 (secção 6.1.3), as recomendações podem

incluir itens pouco frequentes no conjunto base, como artif. sweetener, que surgem por fazerem parte desse histórico.

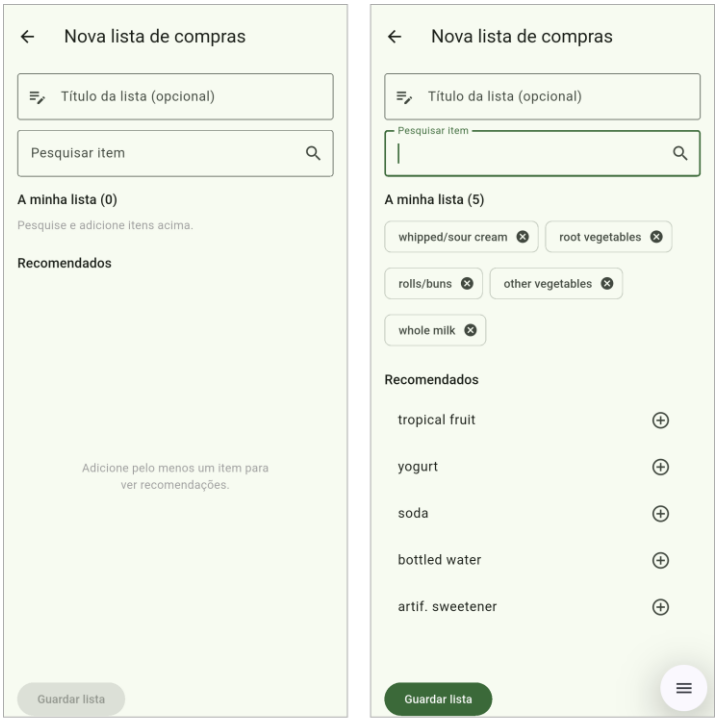


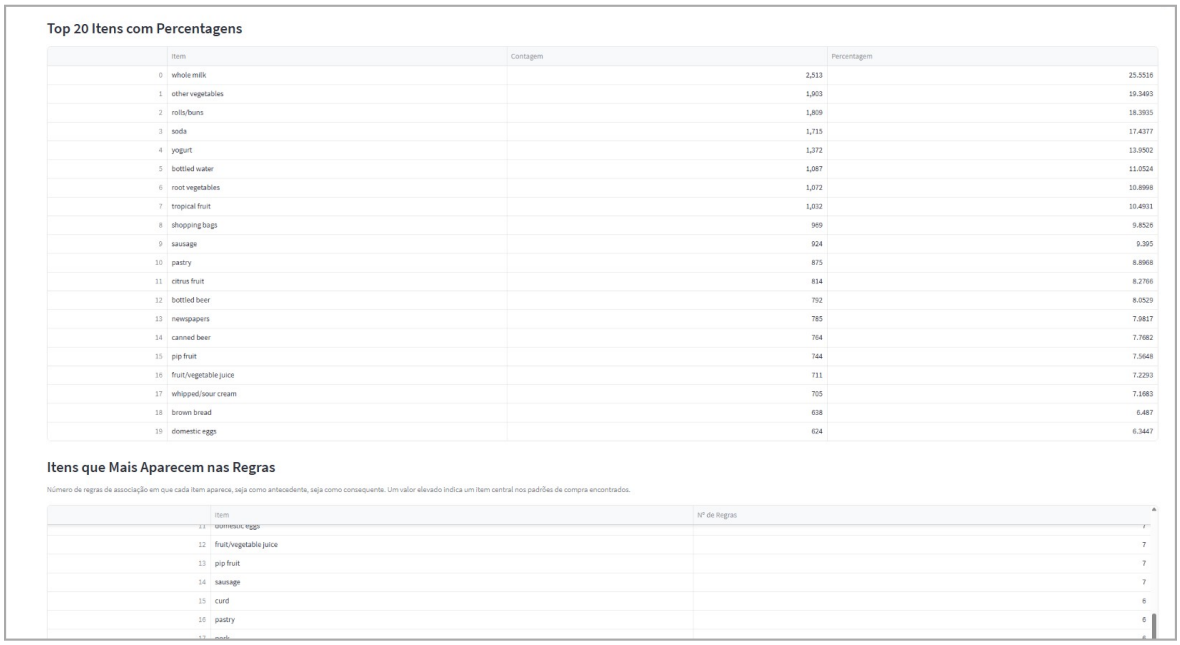
Figura 33 – Aplicação móvel: construção de uma nova lista, (a) sem itens e (b) com itens e recomendações

6.7.2 Backoffice

A Figura 34 e a Figura 35 (a) mostram a página "Estatísticas" (UC08). A página apresenta as regras de associação obtidas com os parâmetros atuais, o número total de itens do conjunto de dados e quantos aparecem ou não em alguma regra, os 20 itens mais frequentes, os itens que aparecem em mais regras e a lista dos itens sem nenhuma regra associada; no fim, mostra ainda um gráfico de dispersão do suporte versus confiança das regras, que não aparece nas figuras. Com os parâmetros por omissão, apenas 46 dos 169 itens aparecem em alguma regra.



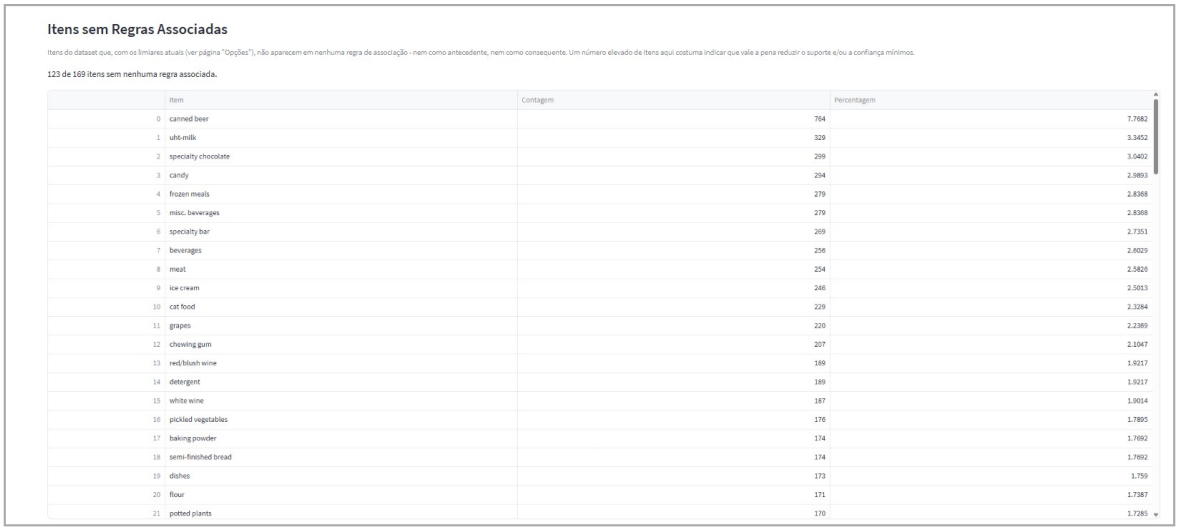
(a)



(b)

Figura 34 – Backoffice, página "Estatísticas": (a) regras de associação e totais de itens, (b) itens mais frequentes e itens que mais aparecem nas regras

A Figura 35 (b) mostra a página "Opções" (UC09), com os três parâmetros do motor e os respetivos valores por omissão.



(a)



(b)

Figura 35 – Backoffice: (a) itens sem regras associadas, na página "Estatísticas", (b) página "Opções"

6.7.3 Documentação da API

A Figura 36 mostra a documentação interativa gerada automaticamente pelo FastAPI, disponível em http://localhost:8000/docs, com os endpoints da Tabela 13 agrupados por área funcional.

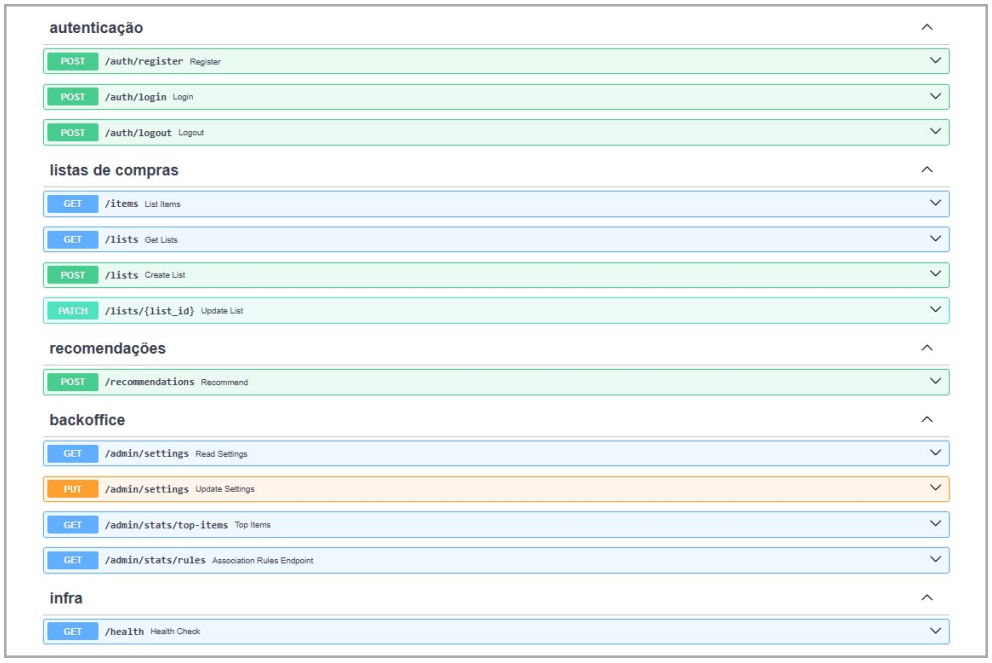


Figura 36 – Documentação interativa da API gerada pelo FastAPI (Swagger UI)

6.8 Síntese do capítulo

A decisão central da arquitetura é concentrar numa única API todo o cálculo de regras de associação e de recomendações. O backoffice e a aplicação móvel são clientes finos: nenhum executa o Apriori localmente e ambos leem os mesmos parâmetros da tabela `Settings`, mantendo uma configuração comum, embora usem dados distintos no cálculo das recomendações e das estatísticas (a aplicação móvel inclui o histórico do utilizador e o backoffice usa apenas o conjunto base).

7. Conclusão

O presente projeto teve como principal objetivo estudar e aplicar técnicas de extração de regras de associação a dados transacionais reais e integrá-las numa aplicação que sugere produtos durante a construção de uma lista de compras. Ao longo do trabalho, foram analisados conceitos fundamentais de mineração de dados e aprendizagem computacional, com especial enfoque na extração de regras de associação através dos algoritmos Apriori e FP-Growth.

A aplicação prática destes algoritmos permitiu verificar, em todas as configurações testadas, que ambos produzem exatamente os mesmos conjuntos frequentes e as mesmas regras, o que é consistente com o facto de serem métodos exatos. As medições de desempenho mostraram que a escolha do algoritmo depende dos dados e da implementação: com a biblioteca mlxtend, o Apriori foi mais rápido no Dataset 1, com apenas 169 itens, enquanto no Dataset 2, com 4 477 itens, a sua implementação por omissão se tornou mais lenta do que o FP-Growth e muito exigente em memória, o que está de acordo com a literatura; neste conjunto, a variante do Apriori com low_memory=True foi a mais rápida.

A análise experimental realizada sobre diferentes conjuntos de dados permitiu discutir a importância da estrutura e qualidade dos dados. No Dataset 1, em que cada registo corresponde diretamente a um talão de compra, foi possível extrair regras de associação relevantes e interpretáveis, associadas a produtos de consumo frequente. No Dataset 2, após um processo de pré-processamento e agregação das transações por número de fatura, foram igualmente identificadas regras fortes, sobretudo entre produtos da mesma categoria ou com afinidade funcional. Estes resultados evidenciam que o pré-processamento dos dados é uma etapa crítica para garantir a qualidade dos padrões extraídos.

Em particular, a definição da unidade de transação e a limpeza dos registos revelaram-se determinantes: no Dataset 2, os registos originais correspondem a linhas de fatura e incluem cancelamentos, linhas sem descrição, quantidades e preços não positivos e códigos que não correspondem a produtos (como portes de envio e ajustes contabilísticos), pelo que foi necessário filtrá-los e agregar as linhas por fatura para obter cestos de compras adequados. No Dataset 1, a comparação com a documentação oficial permitiu detetar que o ficheiro utilizado continha as transações em duplicado. Do ponto de vista teórico, conjuntos de dados com muito poucos itens por transação ou com uma estrutura inadequada tendem a limitar a geração de regras relevantes; esta limitação não foi, contudo, avaliada experimentalmente neste trabalho. Estas observações reforçam a necessidade de uma análise exploratória cuidada antes da aplicação de técnicas de mineração.

No Projeto II, os resultados deste estudo foram transpostos para um sistema funcional composto por uma API REST, desenvolvida com FastAPI, que integra o motor de recomendação baseado no algoritmo Apriori e a persistência dos dados em SQLite;

por uma aplicação móvel em Flutter, através da qual o utilizador constrói listas de compras e recebe sugestões de produtos; e por uma aplicação de backoffice em Streamlit, que permite ao administrador consultar estatísticas e ajustar os parâmetros do motor. O mecanismo de peso implementado permite ainda incorporar as escolhas anteriores do utilizador no cálculo das recomendações. A modelação segundo o processo ICONIX foi feita em duas fases: os casos de uso e os mockups foram definidos antes da implementação e orientaram o desenvolvimento, enquanto as fichas e os diagramas de robustez e de sequência foram completados a partir do código final, servindo sobretudo para documentar o sistema e verificar a correspondência entre os requisitos e a implementação. O motor aplica o Apriori ao Dataset 1, opção coerente com os resultados do Capítulo 5, e ordena as recomendações pela elevação (secção 5.5).

Relativamente aos objetivos definidos na secção 1.2, consideram-se cumpridos os quatro primeiros, cada um com uma evidência distinta: os conceitos da Análise de Cesto de Compras e das regras de associação foram estudados no Capítulo 2; os algoritmos Apriori e FP-Growth foram comparados quanto à equivalência dos resultados e ao desempenho computacional e aplicados a dois conjuntos de dados com características distintas, através de uma análise descritiva das regras (Capítulo 5); e foi desenvolvida uma aplicação que sugere produtos com base nos padrões identificados, cuja correção funcional da API foi verificada com testes automáticos (Capítulo 6). O quinto objetivo, avaliar a utilidade das recomendações, foi cumprido apenas em parte: a avaliação offline (secção 6.5.2) mostrou que, com os parâmetros por omissão, o motor recupera o item em falta em 26,5% dos casos, tal como uma recomendação baseada apenas na popularidade, e que limiares mais baixos aumentam a variedade das recomendações à custa da taxa de acerto; a utilidade percebida pelos utilizadores não foi avaliada.

Como limitações, destacam-se o facto de as medições de desempenho se limitarem a uma única biblioteca (mlxtend) e, no Dataset 2, a conjuntos de dois itens; a granularidade do Dataset 1 (categorias agregadas e não produtos individuais), que limita a especificidade das recomendações do sistema; o facto de a comparação entre os dois conjuntos de dados envolver simultaneamente domínios, épocas, catálogos e parâmetros diferentes; a ausência de validação das regras num conjunto de transações separado; as limitações de segurança da versão atual, em que os endpoints do backoffice não exigem autenticação, as sessões não expiram e a comunicação é feita por HTTP, sem cifra; o facto de, na avaliação offline, o motor não ter superado a recomendação por popularidade, porque as regras se concentram em poucos itens muito frequentes; e a ausência de avaliação das recomendações com utilizadores reais.

Como trabalho futuro, propõe-se: a melhoria do motor face à recomendação por popularidade, por exemplo combinando a elevação com a confiança na ordenação, usando limiares por item ou excluindo dos consequentes os itens demasiado frequentes, com nova avaliação offline; uma avaliação com utilizadores reais, de modo a medir a utilidade efetiva das recomendações e o efeito do peso do histórico; a comparação com outras implementações e algoritmos (por exemplo, o Eclat) e com

conjuntos de itens maiores; a introdução de autenticação no backoffice, de expiração de sessões, de armazenamento seguro do token e de comunicação por HTTPS; o pré-cálculo periódico das regras de associação e a medição do tempo de resposta do motor; testes automáticos da aplicação móvel e do backoffice; e a integração de outras abordagens de recomendação, como a filtragem colaborativa.

Em suma, o projeto demonstrou a viabilidade técnica de transpor os resultados de um estudo de Análise de Cesto de Compras para um sistema orientado ao utilizador final, estabelecendo uma base sólida para a sua evolução e validação em contexto real.

8. Bibliografia

Aggarwal, C. C. (2015). *Data mining: The textbook*. Springer. https://doi.org/10.1007/978-3-319-14142-8

Aggarwal, C. C. (2016). *Recommender systems: The textbook*. Springer.

Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. *ACM SIGMOD Record*, 22(2), 207–216. https://doi.org/10.1145/170036.170072

Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. Em *Proceedings of the 20th International Conference on Very Large Data Bases* (pp. 487–499). Morgan Kaufmann.

Alpaydin, E. (2020). *Introduction to machine learning* (4.ª ed.). MIT Press.

Bellazzi, R., & Zupan, B. (2008). Predictive data mining in clinical medicine: Current issues and guidelines. *International Journal of Medical Informatics*, 77(2), 81–97. https://doi.org/10.1016/j.ijmedinf.2006.11.006

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Brin, S., Motwani, R., Ullman, J. D., & Tsur, S. (1997). Dynamic itemset counting and implication rules for market basket data. Em *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data* (pp. 255–264). ACM. https://doi.org/10.1145/253260.253325

Chen, D. (2012). *Online Retail II* [Conjunto de dados]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CG6D

Chen, D., Sain, S. L., & Guo, K. (2012). Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. *Journal of Database Marketing & Customer Strategy Management*, 19(3), 197–208. https://doi.org/10.1057/dbm.2012.17

Dart. (s.d.). *Dart documentation*. Consultado a 5 de outubro de 2026, em https://dart.dev/guides

Dart Team. (s.d.). *http package*. Consultado a 5 de outubro de 2026, em https://pub.dev/packages/http

Demšar, J., Curk, T., Erjavec, A., Gorup, Č., Hočevar, T., Milutinovič, M., Možina, M., Polajnar, M., Toplak, M., Starič, A., Štajdohar, M., Umek, L., Žagar, L., Žbontar, J., Žitnik, M., & Zupan, B. (2013). Orange: Data mining toolbox in Python. *Journal of Machine Learning Research*, 14(71), 2349–2353. https://jmlr.org/papers/v14/demsar13a.html

Dogan, O. (2024). Enhancing e-commerce product recommendations through statistical settings and product-specific insights. *International Journal of Computational Science and Engineering*, *27*(6). https://doi.org/10.1504/IJCSE.2024.142831

Encode. (s.d.). *Uvicorn: An ASGI web server, for Python*. Consultado a 5 de outubro de 2026, em https://www.uvicorn.org/

Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, 17(3), 37–54.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tese de doutoramento, University of California, Irvine].

Flutter. (s.d.). *Flutter documentation*. Consultado a 5 de outubro de 2026, em https://docs.flutter.dev/

Flutter Team. (s.d.). *shared_preferences package*. Consultado a 5 de outubro de 2026, em https://pub.dev/packages/shared_preferences

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gaffney, K. P., Prammer, M., Brasfield, L., Hipp, D. R., Kennedy, D., & Patel, J. M. (2022). SQLite: Past, present, and future. *Proceedings of the VLDB Endowment*, 15(12), 3535–3547. https://vldb.org/pvldb/volumes/15/paper/SQLite%3A%20Past%2C%20Present%2C%20and%20Future

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Gino, H. L. S., Pedro, D. S., Ponciano, J. R., Linhares, C. D. G., & Traina, A. J. M. (2023). Exploratory analysis on market basket data using network visualization. Em *Anais do XII Brazilian Workshop on Social Network Analysis and Mining (BraSNAM 2023)* (pp. 19–30). Sociedade Brasileira de Computação. https://doi.org/10.5753/brasnam.2023.229505

GitHub. (s.d.). *About*. Consultado a 14 de julho de 2026, em https://github.com/about

Hahsler, M., Hornik, K., & Reutterer, T. (s.d.). *Groceries: Grocery store data* [Documentação do pacote arules]. CRAN. Consultado a 9 de outubro de 2026, em https://search.r-project.org/CRAN/refmans/arules/html/Groceries.html

Hahsler, M., Hornik, K., & Reutterer, T. (2006). Implications of probabilistic data modeling for mining association rules. Em M. Spiliopoulou, R. Kruse, C. Borgelt, A. Nuernberger, & W. Gaul (Eds.), *From data and information analysis to knowledge engineering* (pp. 598–605). Springer-Verlag.

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3.ª ed.). Morgan Kaufmann.

Han, J., Pei, J., & Yin, Y. (2000). Mining frequent patterns without candidate generation. Em *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (pp. 1–12). ACM. https://doi.org/10.1145/342009.335372

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2.ª ed.). Springer.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.

Kaur, M., & Kang, S. (2016). Market basket analysis: Identify the changing trends of market data using association rule mining. *Procedia Computer Science*, 85, 78–85. https://doi.org/10.1016/j.procs.2016.05.180

Kurniawan, F., Umayah, B., Hammad, J., Nugroho, S. M. S., & Hariadi, M. (2018). Market basket analysis to identify customer behaviours by way of transaction data. *Knowledge Engineering and Data Science*, 1(1), 20–25. https://doi.org/10.17977/um018v1i12018p20-25

Kutuzova, T., & Melnik, M. (2018). Market basket analysis of heterogeneous data sources for recommendation system improvement. *Procedia Computer Science*, 136, 246–254. https://doi.org/10.1016/j.procs.2018.08.263

McKinney, W. (2010). Data structures for statistical computing in Python. Em *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Mendeley. (s.d.). *About Mendeley*. Consultado a 14 de julho de 2026, em https://www.mendeley.com/about-us/

Mitchell, T. (1997). *Machine learning*. McGraw-Hill.

Montgomery, D. C., Peck, E. A., & Vining, G. G. (2021). *Introduction to linear regression analysis* (6.ª ed.). Wiley.

Murphy, K. P. (2022). *Probabilistic machine learning: An introduction*. MIT Press.

Mustakim, M., Herianda, D. M., Ilham, A., Daeng GS, A., Laumal, F. E., Kurniasih, N., Iskandar, A., Manulangga, G. C., Iswara, I. B. A. I., & Rahim, R. (2018). Market basket analysis using Apriori and FP-Growth for analysis consumer expenditure patterns at Berkah Mart in Pekanbaru Riau. *Journal of Physics: Conference Series*, 1114(1), Artigo 012131. https://doi.org/10.1088/1742-6596/1114/1/012131

NumPy. (s.d.). *About NumPy*. Consultado a 14 de julho de 2026, em https://numpy.org/about/

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

openpyxl. (s.d.). *openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files*. Consultado a 9 de outubro de 2026, em https://openpyxl.readthedocs.io/

Pandas. (s.d.). *About pandas*. Consultado a 14 de julho de 2026, em https://pandas.pydata.org/about/index.html

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. Em *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. USENIX Association. https://www.usenix.org/conference/1999-usenix-annual-technical-conference/presentation/future-adaptable-password-scheme

Pydantic. (s.d.). *Pydantic validation documentation (versão 2.13)*. Consultado a 5 de outubro de 2026, em https://pydantic.dev/docs/validation/latest/

pytest-dev. (s.d.). *pytest documentation*. Consultado a 5 de outubro de 2026, em https://docs.pytest.org/

Python Software Foundation. (s.d.). *Python documentation*. Consultado a 9 de outubro de 2026, em https://docs.python.org/3/

Ramírez, S. (s.d.). *FastAPI documentation*. Consultado a 5 de outubro de 2026, em https://fastapi.tiangolo.com/

Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. *Journal of Open Source Software*, 3(24), Artigo 638. https://doi.org/10.21105/joss.00638

Requests. (s.d.). *Requests: HTTP for Humans*. Consultado a 5 de outubro de 2026, em https://requests.readthedocs.io/

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). *Recommender systems handbook* (3.ª ed.). Springer.

Rosenberg, D., & Stephens, M. (2007). *Use case driven object modeling with UML: Theory and practice*. Apress.

Santos, A. C. P. C. (2020). *Técnicas de data mining: Análise ao cesto de compras e segmentação de clientes de um supermercado* [Trabalho de projeto de mestrado, NOVA Information Management School, Universidade Nova de Lisboa]. http://hdl.handle.net/10362/112038

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7.ª ed.). McGraw-Hill.

SQLAlchemy. (s.d.). *SQLAlchemy 2.0 documentation*. Consultado a 5 de outubro de 2026, em https://docs.sqlalchemy.org/en/20/

SQLite. (s.d.). *About SQLite*. Consultado a 5 de outubro de 2026, em https://www.sqlite.org/about.html

Streamlit Inc. (s.d.). *Streamlit documentation*. Consultado a 11 de setembro de 2026, em https://docs.streamlit.io/

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2.ª ed.). MIT Press.

Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to data mining* (2.ª ed.). Pearson.

Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), Artigo 3021. https://doi.org/10.21105/joss.03021

Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). *Data mining: Practical machine learning tools and techniques* (4.ª ed.). Morgan Kaufmann.

Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*, 12(3), 372–390. https://doi.org/10.1109/69.846291